
Python

unknown

Apr 23, 2024

CONTENT

1	Installation	3
2	Usage	5
3	atto-device-python	7
3.1	atto_device package	7
4	Indices and tables	181
	Python Module Index	183
	Index	185

Python API for interactions with Attocube devices

INSTALLATION

Preferrably, install the library using a Git branch tag to ensure it's not going to change unexpectedly. For this, look at the most recent version in the right sidebar, and use that, e.g. for v1.0.1:

```
pip install git+ssh://git@github.com/attocube-systems/atto-device-python.git@1.0.1
```

or, to install from master (which might break at any point):

```
pip install git+ssh://git@github.com/attocube-systems/atto-device-python.git
```

CHAPTER TWO

USAGE

```
from atto_device import AMC  
machine = AMC("192.168.53.12")
```


ATTO-DEVICE-PYTHON

3.1 atto_device package

3.1.1 Subpackages

atto_device.AMC package

Submodules

atto_device.AMC.ACS module

exception atto_device.AMC.ACS.**AttoException**(*errorText=None, errorNumber=0*)

Bases: Exception

class atto_device.AMC.ACS.**AttoResult**(*resultDict*)

Bases: object

class atto_device.AMC.ACS.**Device**(*address*)

Bases: object

TCP_PORT = 9090

close()

Closes the connection to the device.

connect()

Initializes and connects the selected AMC device.

static discover(*cls*)

getResponse(*request_id*)

handleError(*response, ignoreFunctionError=False*)

is_open = False

printError(*errorNumber*)

Converts the errorNumber into an error string and prints it to the console. :param errorNumber: :type errorNumber: int

request(*method, params=False*)

Synchronous request.

```
request_id = 466666

request_id_lock = <unlocked _thread.lock object>

response_buffer = {}

sendRequest(method, params=False)
```

atto_device.AMC.AMC module

```
class atto_device.AMC.AMC.Device(address)
    Bases: Device
atto_device.AMC.AMC.discover()
```

atto_device.AMC.about module

```
class atto_device.AMC.about.About(device)
    Bases: object

    getInstalledPackages() → str
        Get list of packages installed on the device :returns: errNo errorCode
        value_string: string: Comma separated list of packages

        Return type
            value_errNo

    getPackageLicense(pkg: str) → str
        Get the license for a specific package

        Parameters
            pkg – string: Package name

        Returns
            errNo errorCode value_string: string: License for this package

        Return type
            value_errNo
```

atto_device.AMC.access module

```
class atto_device.AMC.access.Access(device)
    Bases: object

    getLockStatus()
        This function returns if the device is locked and if the current client is authorized to use the device. :returns:
        errNo errorCode

        value_locked: locked Is the device locked? value_authorized: authorized Is the client authorized?

        Return type
            value_errNo
```

grantAccess(*password: str*)

Grants access to a locked device for the requesting IP by checking against the password

Parameters

password – string the current password

lock(*password: str*)

This function locks the device with a password, so the calling of functions is only possible with this password. The locking IP is automatically added to the devices which can access functions

Parameters

password – string the password to be set

unlock(*password: str*)

This function unlocks the device, so it will not be necessary to execute the grantAccess function to run any function

Parameters

password – string the current password

atto_device.AMC.amcids module

class atto_device.AMC.amcids.**Amcids**(*device*)

Bases: object

getLowerSoftLimit(*axis: int*) → float

Gets the lower boundary of the soft limit protection. This protection is needed if the IDS working range is smaller than the positioners travel range. It is no hard limit, so, it is possible to overshoot it!

Parameters

axis – Axis of the AMC to get the soft limit status from

Returns

Error number if one occurred, 0 in case of no error limit: Lower boundary in pm

Return type

errNo

getSoftLimitEnabled(*axis: int*) → bool

Gets whether the soft limit protection is enabled. This protection is needed if the IDS working range is smaller than the positioners travel range. It is no hard limit, so, it is possible to overshoot it!

Parameters

axis – Axis of the AMC to get the soft limit status from

Returns

Error number if one occurred, 0 in case of no error enabled: True, if the soft limit should be enabled on this axis

Return type

errNo

getSoftLimitReached(*axis: int*) → bool

Gets whether the current position is out of the soft limit boundaries. This protection is needed if the IDS working range is smaller than the positioners travel range. It is no hard limit, so, it is possible to overshoot it!

Parameters

axis – Axis of the AMC to get the soft limit status from

Returns

Error number if one occurred, 0 in case of no error enabled: True, if the position is not within the boundaries

Return type

errNo

getUpperSoftLimit(axis: int) → float

Gets the upper lower boundary of the soft limit protection. This protection is needed if the IDS working range is smaller than the positioners travel range. It is no hard limit, so, it is possible to overshoot it!

Parameters

axis – Axis of the AMC to get the soft limit status from

Returns

Error number if one occurred, 0 in case of no error limit: Upper boundary in pm

Return type

errNo

resetIdsAxis(axis: int)

Resets the position value to zero of a specific measurement axis. Use this for positioners with an IDS as sensor. This method does not work for NUM and RES sensors. Use `com.attocube.amc.control.resetAxis` instead.

Parameters

axis – Axis of the IDS to reset the position

setLowerSoftLimit(axis: int, limit: float)

Sets the lower boundary of the soft limit protection in pm. This protection is needed if the IDS working range is smaller than the positioners travel range. It is no hard limit, so, it is possible to overshoot it!

Parameters

- **axis** – Axis of the AMC where the soft limit should be changed
- **limit** – Lower boundary in pm

setSoftLimitEnabled(axis: int, enabled: bool)

Enables/disables the soft limit protection. This protection is needed if the IDS working range is smaller than the positioners travel range. It is no hard limit, so, it is possible to overshoot it!

Parameters

- **axis** – Axis of the AMC where the soft limit should be changed
- **enabled** – True, if the soft limit should be enabled on this axis

setUpperSoftLimit(axis: int, limit: float)

Sets the upper boundary of the soft limit protection in pm. This protection is needed if the IDS working range is smaller than the positioners travel range. It is no hard limit, so, it is possible to overshoot it!

Parameters

- **axis** – Axis of the AMC where the soft limit should be changed
- **limit** – Upper boundary in pm

atto_device.AMC.control module

class atto_device.AMC.control.**Control**(*device*)

Bases: object

MultiAxisPositioning(*set1: bool, set2: bool, set3: bool, target1: int, target2: int, target3: int*)

Simultaneously set 3 axes positions and get positions to minimize network latency

Parameters

- **set1** – axis1 otherwise pos1 target is ignored
- **set2** – axis2 otherwise pos2 target is ignored
- **set3** – axis3 otherwise pos3 target is ignored
- **target1** – target position of axis 1
- **target2** – target position of axis 2
- **target3** – target position of axis 3

Returns

errNo value_ref1: ref1 Status of axis 1 value_ref2: ref2 Status of axis 2 value_ref3: ref3 Status of axis 3 value_refpos1: refpos1 reference Position of axis 1 value_refpos2: refpos2 reference Position of axis 2 value_refpos3: refpos3 reference Position of axis 3 value_pos1: pos1 position of axis 1 value_pos2: pos2 position of axis 2 value_pos3: pos3 position of axis 3

Return type

errNo

getActorName(*axis: int*) → str

This function gets the name of the positioner of the selected axis.

Parameters

axis – [0|1|2]

Returns

errNo actor_name: actor_name

Return type

errNo

getActorParametersActorName(*axis: int*) → str

Control the actors parameter: actor name

Parameters

axis – [0|1|2]

Returns

errNo actorname: actorname

Return type

errNo

getActorSensitivity(*axis: int*) → int

Get the setting for the actor parameter sensitivity

Parameters

axis – [0|1|2]

Returns

errNo sensitivity: sensitivity

Return type

errNo

getActorType(*axis: int*) → int

This function gets the type of the positioner of the selected axis.

Parameters

axis – [0|1|2]

Returns

errNo value_actor_type: actor_type 0: linear, 1: rotator, 2: goniometer

Return type

errNo

getAutoMeasure(*axis: int*) → bool

This function returns if the automeasurement on axis enable is enabled

Parameters

axis – [0|1|2]

Returns

errNo value_enable: enable true: enable automeasurement, false: disable automeasurement

Return type

errNo

getControlAmplitude(*axis: int*) → int

This function gets the amplitude of the actuator signal of the selected axis.

Parameters

axis – [0|1|2]

Returns

errNo value_amplitude: amplitude in mV

Return type

errNo

getControlAutoReset(*axis: int*) → bool

This function resets the position every time the reference position is detected.

Parameters

axis – [0|1|2]

Returns

errNo value_enabled: enabled boolean

Return type

errNo

getControlFixOutputVoltage(*axis: int*) → int

This function gets the DC level output of the selected axis.

Parameters

axis – [0|1|2]

Returns

errNo value_amplitude_mv: amplitude_mv in mV

Return type

errNo

getControlFrequency(*axis: int*) → int

This function gets the frequency of the actuator signal of the selected axis.

Parameters**axis** – [0|1|2]**Returns**

errNo value_frequency: frequency in mHz

Return type

errNo

getControlMove(*axis: int*) → bool

This function gets the approach of the selected axis' positioner to the target position.

Parameters**axis** – [0|1|2]**Returns**

errNo value_enable: enable boolean true: closed loop control enabled, false: closed loop control disabled

Return type

errNo

getControlOutput(*axis: int*) → bool

This function gets the status of the output relays of the selected axis.

Parameters**axis** – [0|1|2]**Returns**

errNo value_enabled: enabled power status (true = enabled, false = disabled)

Return type

errNo

getControlReferenceAutoUpdate(*axis: int*) → bool

This function gets the status of whether the reference position is updated when the reference mark is hit.

Parameters**axis** – [0|1|2]**Returns**

errNo value_enabled: enabled boolean

Return type

errNo

getControlTargetRange(*axis: int*) → int

This function gets the range around the target position in which the flag "In Target Range" becomes active.

Parameters**axis** – [0|1|2]**Returns**

errNo value_targetrange: targetrange in nm

Return type

errNo

getCrosstalkThreshold(*axis: int*)

This function gets the threshold range and slip phase time which is used while moving another axis

Parameters

axis – [0|1|2]

Returns

errNo value_range: range in pm value_time: time after slip phase which is waited until the controller is acting again in microseconds

Return type

errNo

getCurrentOutputVoltage(*axis: int*) → int

This function gets the current Voltage which is applied to the Piezo

Parameters

axis – [0|1|2]

Returns

errNo value_amplitude: amplitude in mV

Return type

errNo

getExternalSensor(*axis: int*) → bool

This function gets whether the sensor source of closed loop is IDS It is only available when the feature AMC/IDS closed loop has been activated

Parameters

axis – [0|1|2]

Returns

errNo enabled: enabled

Return type

errNo

getFinePositioningRange(*axis: int*) → int

This function gets the fine positioning DC-range

Parameters

axis – [0|1|2]

Returns

errNo value_range: range in nm

Return type

errNo

getFinePositioningSlewRate(*axis: int*) → int

This function gets the fine positioning slew rate

Parameters

axis – [0|1|2]

Returns

errNo value_slewrates: slewrates [0|1|2|3]

Return type

errNo

getMotionControlThreshold(axis: int) → int

This function gets the threshold range within the closed-loop controlled movement stops to regulate.

Parameters

axis – [0|1|2]

Returns

errNo value_threshold: threshold in pm

Return type

errNo

getPositionsAndVoltages()

Simultaneously get 3 axes positions as well as the DC offset to maximize sampling rate over network
:returns: errNo

value_pos1: pos1 position of axis 1 value_pos2: pos2 position of axis 2 value_pos3: pos3 position of axis 3
value_val1: val1 dc voltage of of axis 1 in mV value_val2: val2 dc voltage of of axis 2 in mV
value_val3: val3 dc voltage of of axis 3 in mV

Return type

errNo

getReferencePosition(axis: int) → int

This function gets the reference position of the selected axis.

Parameters

axis – [0|1|2]

Returns

errNo value_position: position: For linear type actors the position is defined in nm for goniometer an rotator type actors it is μ° .

Return type

errNo

getSensorDirection(axis: int) → bool

This function gets whether the IDS sensor source of closed loop is inverted It is only available when the feature AMC/IDS closed loop has been activated

Parameters

axis – [0|1|2]

Returns

errNo value_inverted: inverted boolean

Return type

errNo

getSensorEnabled(axis: int) → bool

Get sensot power supply status

Parameters

axis – [0|1|2]

Returns

errNo value_value: value true if enabled, false otherwise

Return type

errNo

getStatusMovingAllAxes()

Get Status of all axes, see getStatusMoving for coding of the values :returns: errNo

value_moving1: moving1 status of axis 1 value_moving2: moving2 status of axis 2
value_moving3: moving3 status of axis 3

Return type

errNo

searchReferencePosition(axis: int)

This function searches for the reference position of the selected axis.

Parameters

axis – [0|1|2]

setActorParametersByName(axis: int, actorname: str)

This function sets the name for the positioner on the selected axis.

Parameters

- **axis** – [0|1|2]
- **actorname** – name of the actor

setActorParametersJson(axis: int, json_dict: str)

Select and override a positioner out of the Current default list only override given parameters set others default

Parameters

- **axis** – [0|1|2]
- **json_dict** – dict with override params

setActorSensitivity(axis: int, sensitivity: int)

Control the actor parameter closed loop sensitivity

Parameters

- **axis** – [0|1|2]
- **sensitivity** –

setAutoMeasure(axis: int, enable: bool)

This function enables/disables the automatic C/R measurement on axis enable

Parameters

- **axis** – [0|1|2]
- **enable** – true: enable automeasurement, false: disable automeasurement

setControlAmplitude(axis: int, amplitude: int)

This function sets the amplitude of the actuator signal of the selected axis.

Parameters

- **axis** – [0|1|2]
- **amplitude** – in mV

setControlAutoReset(*axis: int, enable: bool*)

This function resets the position every time the reference position is detected.

Parameters

- **axis** – [0|1|2]
- **enable** – boolean

setControlFixOutputVoltage(*axis: int, amplitude_mv: int*)

This function sets the DC level output of the selected axis.

Parameters

- **axis** – [0|1|2]
- **amplitude_mv** – in mV

setControlFrequency(*axis: int, frequency: int*)

This function sets the frequency of the actuator signal of the selected axis.

Parameters

- **axis** – [0|1|2]
- **frequency** – in mHz

setControlMove(*axis: int, enable: bool*)

This function sets the approach of the selected axis' positioner to the target position.

Parameters

- **axis** – [0|1|2]
- **enable** – boolean true: enable the approach, false: disable the approach

setControlOutput(*axis: int, enable: bool*)

This function sets the status of the output relays of the selected axis.

Parameters

- **axis** – [0|1|2]
- **enable** – true: enable drives, false: disable drives

setControlReferenceAutoUpdate(*axis: int, enable: bool*)

This function sets the status of whether the reference position is updated when the reference mark is hit.

Parameters

- **axis** – [0|1|2]
- **enable** – boolean

setControlTargetRange(*axis: int, range: int*)

This function sets the range around the target position in which the flag “In Target Range” (see VIII.7.a) becomes active.

Parameters

- **axis** – [0|1|2]
- **range** – in nm

setCrosstalkThreshold(*axis: int, threshold: int, slipphasetime: int*)

This function sets the threshold range and slip phase time which is used while moving another axis

Parameters

- **axis** – [0|1|2]
- **threshold** – in pm
- **slipphasetime** – time after slip phase which is waited until the controller is acting again in microseconds

setExternalSensor()

This function sets the sensor source of closed loop to the IDS when enabled.

Parameters

- **axis** – [0|1|2]
- **enabled** –

setFinePositioningRange(*axis: int, range: int*)

This function sets the fine positioning DC-range

Parameters

- **axis** – [0|1|2]
- **range** – in nm

setFinePositioningSlewRate(*axis: int, slewrate: int*)

This function sets the fine positioning slew rate

Parameters

- **axis** – [0|1|2]
- **slewrate** – [0|1|2|3]

setMotionControlThreshold(*axis: int, threshold: int*)

This function sets the threshold range within the closed-loop controlled movement stops to regulate.

Parameters

- **axis** – [0|1|2]
- **threshold** – in pm

setReset(*axis: int*)

This function resets the actual position of the selected axis given by the NUM sensor to zero and marks the reference position as invalid.

Parameters

axis – [0|1|2]

setSensorDirection(*axis: int, inverted: bool*)

This function sets the IDS sensor source of closed loop to inverted when true.

Parameters

- **axis** – [0|1|2]
- **inverted** –

setSensorEnabled(*axis: int, value: bool*)

Set sensor power supply status, can be switched off to save heat generated by sensor [NUM or RES] Positions retrieved will be invalid when activating this, so closed-loop control should be switched off beforehand

Parameters

- **axis** – [0|1|2]
- **value** – true if enabled, false otherwise

atto_device.AMC.description module

class atto_device.AMC.description.**Description**(*device*)

Bases: object

checkChassisNbr()

Get Chassis and Slot Number, only works when AMC is within a Rack :returns: errNo errorCode

slotNbr: slotNbr chassisNbr: chassisNbr

Return type

value_errNo

getDeviceType() → str

This function gets the device type based on its EEPROM configuration. :returns: errNo

value_devicetype: devicetype Device name (AMC100, AMC150, AMC300) with attached feature
(AMC100/NUM, AMC100/NUM/PRO)

Return type

errNo

getFeaturesActivated() → str

Get the activated features and return as a string :returns: errNo

value_features: features activated on device concatenated by comma e.g. Closed loop Operation,
Pro, Wireless Controller, IO

Return type

errNo

getPositionersList() → str

This function reads the actor names that can be connected to the device. :returns: errNo

PositionersList: PositionersList

Return type

errNo

atto_device.AMC.diagnostic module**class** atto_device.AMC.diagnostic.**Diagnostic**(*device*)

Bases: object

getDiagnosticPower(*axis: int*) → int

Returns the current power consumption

Parameters**axis** – [0|1|2]**Returns**

errNo power: power

Return type

errNo

getDiagnosticResults(*axis: int*)

Returns the results of the last diagnostic run and an error, if there was no run, it is currently running or the run failed

Parameters**axis** – [0|1|2]**Returns**

errNo value_capacity: capacity in nF value_resistance: resistance in Ohm

Return type

errNo

getDiagnosticStepSize(*axis: int*)

Performs 10 steps in forward and backward and calculates the average step size in both directions on a specific axis

Parameters**axis** – [0|1|2]**Returns**

errNo stepsize_fwd: stepsize_fwd stepsize_bwd: stepsize_bwd

Return type

errNo

getDiagnosticTemperature(*axis: int*) → int

Returns the current axis temperature

Parameters**axis** – [0|1|2]**Returns**

errNo temperature: temperature

Return type

errNo

startDiagnostic(*axis: int*)

Start the diagnosis procedure for the given axis

Parameters**axis** – [0|1|2]

atto_device.AMC.functions module

```
class atto_device.AMC.functions.Functions(device)
```

Bases: object

```
checkAMCinRack()
```

If AMC is on Rack position 0, use it as DHCP server, else use it as DHCP client

atto_device.AMC.move module

```
class atto_device.AMC.move.Move(device)
```

Bases: object

```
getControlContinuousBkwd(axis: int) → bool
```

This function gets the axis' movement status in backward direction.

Parameters

axis – [0|1|2]

Returns

errNo value_enabled: enabled true if movement backward is active , false otherwise

Return type

errNo

```
getControlContinuousFwd(axis: int) → bool
```

This function gets the axis' movement status in positive direction.

Parameters

axis – [0|1|2]

Returns

errNo value_enabled: enabled true if movement Fwd is active, false otherwise

Return type

errNo

```
getControlEotOutputDeactive(axis: int) → bool
```

This function gets the output applied to the selected axis on the end of travel.

Parameters

axis – [0|1|2]

Returns

errNo value_enabled: enabled If true, the output of the axis will be deactivated on positive EOT detection.

Return type

errNo

```
getControlTargetPosition(axis: int) → float
```

This function gets the target position for the movement on the selected axis.

Parameters

axis – [0|1|2]

Returns

errNo value_position: position defined in nm for goniometer an rotator type actors it is μ° .

Return type

errNo

getGroundAxis(axis: int) → bool

Checks if the axis piezo drive is actively grounded only in AMC300

Parameters**axis** – motion controller axis [0|1|2]**Returns**

errNo 0 or error value_ground: grounded true or false

Return type

value_errNo

getGroundAxisAutoOnTarget(axis: int) → bool

Pull axis piezo drive to GND if positioner is in ground target range only in AMC300

Parameters**axis** – motion controller axis [0|1|2]**Returns**

errNo 0 or error value_value: value true or false

Return type

value_errNo

getGroundTargetRange(axis: int) → int

Retrieves the range around the target position in which the auto grounding becomes active.

Parameters**axis** – [0|1|2]**Returns**

errNo value_targetrange: targetrange in nm

Return type

errNo

getNSteps(axis: int) → int

This function gets the number of Steps in desired direction.

Parameters**axis** – [0|1|2]**Returns**

errNo nbrstep: nbrstep

Return type

errNo

getPosition(axis: int) → float

This function gets the current position of the positioner on the selected axis.

Parameters**axis** – [0|1|2]**Returns**errNo value_position: position defined in nm for goniometer an rotator type actors it is μ° .**Return type**

errNo

moveReference(*axis: int*)

This function starts an approach to the reference position.

Parameters

axis – [0|1|2]

performNSteps(*axis: int, backward: bool*)

Perform the OL command for N steps

Parameters

- **axis** – [0|1|2]
- **backward** – Selects the desired direction. False triggers a forward step, true a backward step

setControlContinuousBkwd(*axis: int, enable: bool*)

This function sets a continuous movement on the selected axis in backward direction.

Parameters

- **axis** – [0|1|2]
- **enable** – If enabled a present movement in the opposite direction is stopped. The parameter “false” stops all movement of the axis regardless its direction

setControlContinuousFwd(*axis: int, enable: bool*)

This function sets a continuous movement on the selected axis in positive direction.

Parameters

- **axis** – [0|1|2]
- **enable** – If enabled a present movement in the opposite direction is stopped. The parameter “false” stops all movement of the axis regardless its direction.

setControlEotOutputDeactive(*axis: int, enable: bool*)

This function sets the output applied to the selected axis on the end of travel.

Parameters

- **axis** – [0|1|2]
- **enable** – if enabled, the output of the axis will be deactivated on positive EOT detection.

setControlTargetPosition(*axis: int, target: float*)

This function sets the target position for the movement on the selected axis.

Parameters

- **axis** – [0|1|2]
- **target** – absolute position : For linear type actors the position is defined in nm for goniometer an rotator type actors it is μ° .

setGroundAxis(*axis: int, enabled: bool*)

Pull axis piezo drive to GND actively only in AMC300 this is used in MIC-Mode

Parameters

- **axis** – motion controler axis [0|1|2]
- **enabled** – true or false

setGroundAxisAutoOnTarget(*axis: int, enabled: bool*)

Pull axis piezo drive to GND actively if positioner is in ground target range only in AMC300 this is used in MIC-Mode

Parameters

- **axis** – motion controller axis [0|1|2]
- **enabled** – true or false

setGroundTargetRange(*axis: int, range: int*)

Set the range around the target position in which the auto grounding becomes active.

Parameters

- **axis** – [0|1|2]
- **range** – in nm

setNSteps(*axis: int, backward: bool, step: int*)

This function triggers n steps on the selected axis in desired direction.

Parameters

- **axis** – [0|1|2]
- **backward** – Selects the desired direction. False triggers a forward step, true a backward step
- **step** – number of step

setSingleStep(*axis: int, backward: bool*)

This function triggers one step on the selected axis in desired direction.

Parameters

- **axis** – [0|1|2]
- **backward** – Selects the desired direction. False triggers a forward step, true a backward step

writeNSteps(*axis: int, step: int*)

Sets the number of steps to perform on stepwise movement.

Parameters

- **axis** – [0|1|2]
- **step** – number of step

atto_device.AMC.network module

class atto_device.AMC.network.**Network**(*device*)

Bases: object

apply()

Apply temporary IP configuration and load it

configureWifi(*mode: int, ssid: str, psk: str*)

Change the wifi configuration and applies it

Parameters

- **mode** – 0: Access point, 1: Wifi client
- **ssid** –
- **psk** – Pre-shared key

discard()

Discard temporary IP configuration

getDefaultGateway() → str

Get the default gateway of the device :returns: errNo errorCode

value_Default: Default gateway

Return type

value_errNo

getDnsResolver(priority: int) → str

Get the DNS resolver

Parameters

priority – of DNS resolver (Usually: 0 = Default, 1 = Backup)

Returns

errNo errorCode value_IP: IP address of DNS resolver

Return type

value_errNo

getEnableDhcpClient() → bool

Get the state of DHCP client :returns: errNo errorCode

value_boolean: boolean: true = DHCP client enable, false = DHCP client disable

Return type

value_errNo

getEnableDhcpServer() → bool

Get the state of DHCP server :returns: errNo errorCode

value_boolean: boolean: true = DHCP server enable, false = DHCP server disable

Return type

value_errNo

getIpAddress() → str

Get the IP address of the device :returns: errNo errorCode

value_IP: IP address as string

Return type

value_errNo

getProxyServer() → str

Get the proxy settings of the device :returns: errNo errorCode

value_Proxy: Proxy Server String, empty for no proxy

Return type

value_errNo

getRealIpAddress() → str

Get the real IP address of the device set to the network interface (br0, eth1 or eth0) :returns: errNo errorCode

value_IP: IP address as string

Return type

value_errNo

getSubnetMask() → str

Get the subnet mask of the device :returns: errNo errorCode

value_Subnet: Subnet mask as string

Return type

value_errNo

getWifiMode() → int

Get the operation mode of the wifi adapter :returns: errNo errorCode

value_mode: mode 0: Access point, 1: Wifi client

Return type

value_errNo

getWifiPassphrase() → str

Get the the passphrase of the network hosted (mode: Access point) or connected to (mode: client) :returns: errNo errorCode

value_psk: psk Pre-shared key

Return type

value_errNo

getWifiPresent() → bool

Returns is a Wifi interface is present :returns: errNo errorCode

value_True: True, if interface is present

Return type

value_errNo

getWifiSSID() → str

Get the the SSID of the network hosted (mode: Access point) or connected to (mode: client) :returns: errNo errorCode

SSID: SSID

Return type

value_errNo

setDefaultGateway(*gateway: str*)

Set the default gateway of the device

Parameters

gateway – Default gateway as string

setDnsResolver(*priority: int, resolver: str*)

Set the DNS resolver

Parameters

- **priority** – of DNS resolver (Usually: 0 = Default, 1 = Backup)
- **resolver** – The resolver's IP address as string

setEnabledDhcpClient(*enable: bool*)

Enable or disable DHCP client

Parameters

enable – boolean: true = enable DHCP client, false = disable DHCP client

setEnabledDhcpServer(*enable: bool*)

Enable or disable DHCP server

Parameters

enable – boolean: true = enable DHCP server, false = disable DHCP server

setIpAddress(*address: str*)

Set the IP address of the device

Parameters

address – IP address as string

setProxyServer(*proxyServer: str*)

Set the proxy server of the device

Parameters

proxyServer – Proxy Server Setting as string

setSubnetMask(*netmask: str*)

Set the subnet mask of the device

Parameters

netmask – Subnet mask as string

setWifiMode(*mode: int*)

Change the operation mode of the wifi adapter

Parameters

mode – 0: Access point, 1: Wifi client

setWifiPassphrase(*psk: str*)

Change the passphrase of the network hosted (mode: Access point) or connected to (mode: client)

Parameters

psk – Pre-shared key

setWifiSSID(*ssid: str*)

Change the SSID of the network hosted (mode: Access point) or connected to (mode: client)

Parameters

ssid –

verify()

Verify that temporary IP configuration is correct

atto_device.AMC.res module

class atto_device.AMC.res.Res(device)

Bases: object

getChainGain(axis: int) → int

Get chain gain, see setChainGain for parameter description

Parameters

axis – number of axis

Returns

errNo gaincoeff: gaincoeff

Return type

errNo

getLinearization(axis: int) → bool

Gets wether linearization is enabled or not

Parameters

axis – [0|1|2]

Returns

errNo value_enabled: enabled true when enabled

Return type

errNo

getLutSn(axis: int) → str

get the identifier of the loaded lookuptable (will be empty if disabled)

Parameters

axis – [0|1|2]

Returns

errNo value_string: string : identifier

Return type

errNo

getMode() → int

Get mode of RES application, see setMode for the description of possible parameters :returns: errNo

mode: mode

Return type

errNo

getSensorStatus(axis: int) → bool

Gets wether a valid RES position signal is present (always true for a disabled sensor and for rotators)

Parameters

axis – [0|1|2]

Returns

errNo value_present: present true when present

Return type

errNo

setChainGain(*axis: int, gainconfig: int*)

Set signal chain gain to control overall power

Parameters

- **axis** – number of axis
- **gainconfig** – 0: 0dB (power $600\text{mVpkpk}^2/R$), 1 : -10 dB , 2 : -15 dB , 3 : -20 dB

setConfigurationFile(*axis: int, content: str*)

Load configuration file which either contains a JSON dict with parameters for the positioner on the axis or the LUT file itself (as legacy support for ANC350 .aps files)

Parameters:

axis: [0|1|2] content: JSON Dictionary or .aps File.

The JSON Dictionary can/must contain the following keys: 'type': mandatory This field has to be one of the positioner list (see getPositionersList) 'lut': optional, contains an array of 1024 LUT values that are a mapping between ratio of the RES element travelled (0 to 1) and the corresponding absolute value at this ratio given in [nm]. Note: when generating these tables with position data in absolute units, the scaling of the travel ratio with the current sensor range has to be reversed. 'lut_sn': optional, a string to uniquely identify the loaded LUT

setLinearization(*axis: int, enable: bool*)

Control if linearization is enabled or not

Parameters

- **axis** – [0|1|2]
- **enable** – boolean (true: enable linearization)

setMode(*mode: int*)

Sets the mode of the RES position measurement This selects which frequency/ies are used for the lock-in measurement of the RES position, currently there are two possibilities: 1: Individual per axis: each axis is measured on a different frequency; this mode reduces noise coupling between axes, while requiring more wiring 2: Shared line/MIC-Mode: each axis is measured on the same frequency, which reduces the number of required wires while more coupling noise is expected 3: Same as 1, but with overall lower frequencies.

Parameters

mode – 1: Individual per axis, 2: Shared line mode, 3: Individual per axis (low frequency), 4: Shared line mode (low frequency)

atto_device.AMC.rotcomp module

class atto_device.AMC.rotcomp.**Rotcomp**(*device*)

Bases: object

getControlTargetRanges() → bool

Checks if all three axis are in target range. :returns: Error code, if there was an error, otherwise 0 for ok

in_target_range: true all three axes are in target range, false at least one axis is not in target range

Return type

errNo

getEnabled() → bool

Gets the enabled status of the rotation compensation :returns: Error code, if there was an error, otherwise 0 for ok

enabled: true Rotation compensation is enabled, false Rotation compensation is disabled

Return type

errNo

getLUT() → str

Gets the LUT file as JSON string :returns: Error code, if there was an error, otherwise 0 for ok

lut: JSON string of the LUT file for the rotation compensation

Return type

errNo

setEnabled(enabled: bool)

Enables and disables the rotation compensation

Parameters

enabled – true Rotation compensation is enabled, false Rotation compensation is disabled

setLUT(lut_string: str)

Sets the LUT file from a JSON string

Parameters

lut_string – JSON string of the LUT file for the rotation compensation

updateOffsets(offset_axis0: int, offset_axis1: int, offset_axis2: int)

Updates the start offsets of the axes

Parameters

- **offset_axis0** – Offset of axis 1 in [nm]
- **offset_axis1** – Offset of axis 2 in [nm]
- **offset_axis2** – Offset of axis 3 in [nm]

atto_device.AMC.rtin module**class** atto_device.AMC.rtin.**Rtin(device)**

Bases: object

apply()

Apply all realtime input function

discard()

Discard all values beting set and not yet applied

getControlAQuadBInResolution(*axis: int*) → int

This function gets the AQuadB input resolution for setpoint parameter.

Parameters

axis – [0|1|2]

Returns

errNo value_resolution: resolution in nm

Return type

errNo

getControlMoveGPIO(*axis: int*) → bool

This function gets the status for real time input on the selected axis in closed-loop mode.

Parameters

axis – [0|1|2]

Returns

errNo value_enable: enable boolean true: approach enabled , false: approach disabled

Return type

errNo

getGpioMode() → int

get the GPIO mode for Mic Mode feature :returns: errNo

value_gpio_mode: gpio_mode: 0: Standard GPIO 1: NSL-/Mic-Mode

Return type

errNo

getNslMux() → int

get the axis the NSL multiplexer is set to :returns: errNo

value_mux_mode: mux_mode 0: Off 1: Axis 1 2: Axis 2 3: Axis 3

Return type

errNo

getRealTimeInChangePerPulse(*axis: int*) → int

This function gets the change per pulse for the selected axis under real time input in the closed-loop mode.

Parameters

axis – [0|1|2]

Returns

errNo value_resolution: resolution to be added in current pos in nm

Return type

errNo

getRealTimeInFeedbackLoopMode(*axis: int*) → int

Get if the realtime function must operate in close loop operation or open loop operation

Parameters

axis – [0|1|2]

Returns

errNo value_mode: mode 0: open loop, 1 : close-loop

Return type

errNo

getRealTimeInMode(*axis: int*) → int

This function sets or gets the real time input mode for the selected axis.

Parameters**axis** – [0|1|2]**Returns**errNo value_mode: mode see *RT_IN_MODES***Return type**

errNo

getRealTimeInStepsPerPulse(*axis: int*) → int

Get the change in step per pulse of the realtime input when trigger and stepper mode is used

Parameters**axis** – [0|1|2]**Returns**

errNo value_steps: steps number of steps to applied

Return type

errNo

setControlAQuadBInResolution(*axis: int, resolution: int*)

This function sets the AQuadB input resolution for setpoint parameter.

Parameters

- **axis** – [0|1|2]
- **resolution** – ion nm

setControlMoveGPIO(*axis: int, enable: bool*)

This function sets the status for real time input on the selected axis in closed-loop mode.

Parameters

- **axis** – [0|1|2]
- **enable** – boolean true: enable the approach , false: disable the approach

setGpioMode(*gpio_mode: int*)

set the GPIO mode for Mic Mode feature

Parameters:

gpio_mode: [0|1]

0: Standard GPIO 1: NSL-/Mic-Mode

setNs1Mux(*mux_mode: int*)

set the axis the NSL multiplexer is set to

Parameters:

mux_mode: [0|1|2|3]

0: Off 1: Axis 1 2: Axis 2 3: Axis 3

setRealTimeInChangePerPulse(*axis: int, delta: int*)

This function sets the change per pulse for the selected axis under real time input in the closed-loop mode.

Parameters

- **axis** – [0|1|2]
- **delta** – to be added to current position in nm

setRealTimeInFeedbackLoopMode(*axis: int, mode: int*)

Set if the realtime function must operate in close loop operation or open loop operation

Parameters

- **axis** – [0|1|2]
- **mode** – 0: open loop, 1 : close-loop

setRealTimeInMode(*axis: int, mode: int*)

This function sets the real time input mode for the selected axis.

Parameters

- **axis** – [0|1|2]
- **mode** – see *RT_IN_MODES* @see realtime

setRealTimeInStepsPerPulse(*axis: int, steps: int*)

Set the change in step per pulse of the realtime input when trigger and stepper mode is used only used in open loop operation

Parameters

- **axis** – [0|1|2]
- **steps** – number of steps to applied

atto_device.AMC.rtout module

class atto_device.AMC.rtout.**Rtout**(*device*)

Bases: object

apply()

Apply for all rtout function

applyAxis(*axis: int*)

Apply for rtout function of specific axis

Parameters

axis – [0|1|2]

discard()

Discard all rtout value set by the set function(not applied yet)

discardAxis(*axis: int*)

Discard rtout value of specific axis set by the set function(not applied yet)

Parameters

axis – [0|1|2]

discardSignalMode()

Discard value set by setSignalMode

getControlAQuadBOut(*axis: int*) → bool

This function gets if of AQuadB output for position indication is enabled

Parameters

axis – [0|1|2]

Returns

errNo value_enabled: enabled boolean

Return type

errNo

getControlAQuadBOutClock(*axis: int*) → int

This function gets the clock for AQuadB output.

Parameters

axis – [0|1|2]

Returns

errNo value_clock_in_ns: clock_in_ns Clock in multiples of 20ns. Minimum 2 (40ns), maximum 65535 (1,310700ms)

Return type

errNo

getControlAQuadBOutResolution(*axis: int*) → int

This function gets the AQuadB output resolution for position indication.

Parameters

axis – [0|1|2]

Returns

errNo value_resolution: resolution in nm

Return type

errNo

getMode(*axis: int*) → int

Get Mode

Parameters

axis – [0|1|2]

Returns

errNo value_mode: mode 0: Off, 1: AquadB, 2: Trigger

Return type

errNo

getSignalMode(*tempvalue: int*) → int

This function gets the real time output mode for the selected axis.

Parameters

tempvalue –

Returns

errNo value_mode: mode 0: TTL, 1: LVDS

Return type

errNo

getTriggerConfig(*axis: int*)

Get the real time output trigger config

Parameters

axis – [0|1|2]

Returns

errNo value_higher: higher upper limit in nm / μ deg value_lower: lower lower limit in nm / μ deg value_epsilon: epsilon hysteresis in nm / μ deg value_polarity: polarity 0: active high, 1: active low

Return type

errNo

setControlAQuadBOutClock(*axis: int, clock: int*)

This function sets the clock for AQuadB output.

Parameters

- **axis** – [0|1|2]
- **clock** – Clock in multiples of 20ns. Minimum 2 (40ns), maximum 65535 (1,310700ms)

setControlAQuadBOutResolution(*axis: int, resolution: int*)

This function sets the AQuadB output resolution for position indication.

Parameters

- **axis** – [0|1|2]
- **resolution** – in nm

setMode(*axis: int, mode: int*)

Set the real time output signal mode

Parameters

- **axis** – [0|1|2]
- **mode** – 0: Off, 1: AQuadB, 2: Trigger

setSignalMode(*mode: int*)

This function sets the real time output mode for the selected axis.

Parameters

mode – 0: TTL, 1: LVDS

setTriggerConfig(*axis: int, higher: int, lower: int, epsilon: int, polarity: int*)

Control the real time output trigger config

Parameters

- **axis** – [0|1|2]
- **higher** – upper limit in nm / μ deg
- **lower** – lower limit in nm / μ deg
- **epsilon** – hysteresis in nm / μ deg
- **polarity** – 0: active high, 1: active low

atto_device.AMC.status module**class** atto_device.AMC.status.**Status**(*device*)

Bases: object

getFullCombinedStatus(*axis: int*) → str

Get the full combined status of a positioner axis and return the status as a string (to be used in the Webap-
plication)

Parameters**axis** – [0|1|2]**Returns**

errNo value_string: string can be “moving”, “in target range”, “backward limit reached”, “for-
ward limit reached”, “positioner not connected”, “grounded” (only AMC300), “output not
enabled”

Return type

errNo

getOlStatus(*axis: int*) → int

Get the Feedback status of the positioner

Parameters**axis** – [0|1|2]**Returns**

errNo value_sensorstatus: sensorstatus as integer 0: NUM Positioner connected 1: OL posi-
tioner connected 2: No positioner connected , 3: RES positione connected

Return type

errNo

getStatusConnected(*axis: int*) → bool

This function gets information about the connection status of the selected axis' positioner.

Parameters**axis** – [0|1|2]**Returns**

errNo value_connected: connected If true, the actor is connected

Return type

errNo

getStatusEot(*axis: int*) → bool

Retrieves the status of the end of travel (EOT) detection in backward direction or in forward direction.

Parameters**axis** – [0|1|2]**Returns**

errNo value_detected: detected true when EoT in either direction was detected

Return type

errNo

getStatusEotBkwd(*axis: int*) → bool

This function gets the status of the end of travel detection on the selected axis in backward direction.

Parameters**axis** – [0|1|2]**Returns**

errNo value_detected: detected true when EoT was detected

Return type

errNo

getStatusEotFwd(axis: int) → bool

This function gets the status of the end of travel detection on the selected axis in forward direction.

Parameters**axis** – [0|1|2]**Returns**

errNo value_detected: detected true when EoT was detected

Return type

errNo

getStatusMoving(axis: int) → int

This function gets information about the status of the stage output.

Parameters:

axis: [0|1|2]

Returns:

errNo: errNo value_status: status 0: Idle, i.e. within the noise range of the sensor, 1: Moving, i.e the actor is actively driven by the output stage either for closed-loop approach or continuous/single stepping and the output is active.

2 : Pending means the output stage is driving but the output is deactivated

getStatusReference(axis: int) → bool

This function gets information about the status of the reference position.

Parameters**axis** – [0|1|2]**Returns**

errNo value_valid: valid true = valid, false = not valid

Return type

errNo

getStatusTargetRange(axis: int) → bool

This function gets information about whether the selected axis' positioner is in target range or not.

Parameters**axis** – [0|1|2]**Returns**

errNo value_in_range: in_range true within the target range, false not within the target range

Return type

errNo

atto_device.AMC.system_service module**class** atto_device.AMC.system_service.**System_service**(device)

Bases: object

apply(key: int)

Apply temporary system configuration

Parameters**key** –**errorNumberToRecommendation**(language: int, errNbr: int) → str

Get a recommendation for the error code

Parameters

- **language** – integer: Language code
- **errNbr** – integer: Error code to translate

Returns

errNo errorCode value_string: string: Error recommendation (currently returning an int = 0 until we have recommendations)

Return type

value_errNo

errorNumberToString(language: int, errNbr: int) → str

Get a description of an error code

Parameters

- **language** – integer: Language code 0 for the error name, 1 for a more user friendly error message
- **errNbr** – integer: Error code to translate

Returns

errNo errorCode value_string: string: Error description

Return type

value_errNo

factoryReset()

Turns on the factory reset flag.

getDeviceName() → str

Get the actual device name :returns: errNo errorCode

value_string: string: actual device name

Return type

value_errNo

getFirmwareVersion() → str

Get the firmware version of the system :returns: errNo errorCode

value_string: string: The firmware version

Return type

value_errNo

getFluxCode() → str

Get the flux code of the system :returns: errNo errorCode

value_string: string: flux code

Return type

value_errNo

getHostname() → str

Return device hostname :returns: errNo errorCode

available: available

Return type

value_errNo

getMacAddress() → str

Get the mac address of the system :returns: errNo errorCode

value_string: string: Mac address of the system

Return type

value_errNo

getSerialNumber() → str

Get the serial number of the system :returns: errNo errorCode

value_string: string: Serial number

Return type

value_errNo

rebootSystem()

Reboot the system

setDeviceName(*name: str*)

Set custom name for the device

Parameters

name – string: device name

setTime(*day: int, month: int, year: int, hour: int, minute: int, second: int*)

Set system time manually

Parameters

- **day** – integer: Day (1-31)
- **month** – integer: Day (1-12)
- **year** – integer: Day (eg. 2021)
- **hour** – integer: Day (0-23)
- **minute** – integer: Day (0-59)
- **second** – integer: Day (0-59)

softReset()

Performs a soft reset (Reset without deleting the network settings).

updateTimeFromInternet()

Update system time by querying attocube.com

atto_device.AMC.test module

class `atto_device.AMC.test.Test(device)`

Bases: object

clearLog(*axis: int, testname: str*)

Resets the log For debugging only.

Parameters

- **axis** – Axis of the AMC
- **testname** – Name of the test

execute(*name: str, parameters: str*)

Starts a test run For debugging only. Error codes: `ERR_OK = 0`
`ERR_TEST_DOES_NOT_EXIST = -1` `ERR_TEST_RUNNING = -2`

Parameters:

name: Name of the test, see `getAllTest()` **parameters:** Parameters object as stringified JSON object, where “default” is the applied value

Example

```
“{
  “axis”: {
    “friendlyName”: “Axis”, “default”: “0”
  }, “mode”: {
    “friendlyName”: “Mode”, “default”: 0
  }, “amplitude”: {
    “friendlyName”: “Amplitude (V)”, “default”: “45”
  }, “frequency”: {
    “friendlyName”: “Frequency (Hz)”, “default”: “2000”
  }, “cycles”: {
    “friendlyName”: “Cycles”, “default”: “3”
  }, “random_range”: {
    “friendlyName”: “Random range”, “default”: 500000
  }, “buffer”: {
    “friendlyName”: “Buffer”,
    “default”: 2000
  }, “sample_period”: {
    “friendlyName”: “Sample period (ms)”, “default”: “100”
```

```
    }
  }”
```

getAllTests(axis: int)

Request all names of the registered tests For debugging only.

Parameters:

axis:

Returns:

error_code: Error code tests: Jsonified list with all automatic tests

Example “[

```
{
  “name”: “Velocity Test”, “parameters”: {
    “axis”: {
      “friendlyName”: “Axis”, “default”: “0”
    }, “mode”: {
      “friendlyName”: “Mode”, “default”: 0
    }, “amplitude”: {
      “friendlyName”: “Amplitude (V)”, “default”: “45”
    }, “frequency”: {
      “friendlyName”: “Frequency (Hz)”, “default”: “2000”
    }, “cycles”: {
      “friendlyName”: “Cycles”, “default”: “3”
    }, “random_range”: {
      “friendlyName”: “Random range”, “default”: 500000
    }, “buffer”: {
      “friendlyName”: “Buffer”,
      “default”: 2000
    }, “sample_period”: {
      “friendlyName”: “Sample period (ms)”, “default”: “100”
    }
  },
  “version”: “1.0.0”, “stoppable”: true
}
```

]”

, “version”: “1.0.0”, “stoppable”: true}]” “stoppable” tells the user if this test can be aborted while running

manualTests: Jsonified list with all manual tests

Example “[{“name”: “Capacity Test”, “parameters”: {“capacity”: “Capacity (nF)”}, “version”: “1.0.0”}]”

getLog(*axis: int*) → str

Gets the complete log For debugging only.

Parameters

axis – Axis of the AMC

Returns

Error code logs: Log string json-encoded ({testname: log-str})

Return type

error_code

getPositionerSN(*axis: int*) → str

Gets the serial number of the positioner connected to a given axis. For debugging only.

Parameters

axis –

Returns

Error code sn: Serial number of positioner

Return type

error_code

getReport(*axis: int, name: str*) → str

Get test report of last test run of specific test name == “all”: the test reports of all tests from last test run will be returned For debugging only.

Parameters

- **axis** –
- **name** – Name of the test or “all”

Returns

Error code report: Test report json-serialized

Return type

error_code

getStatus(*axis: int*)

Get the current execution status of the test sequencer For debugging only. Status: IDLE = 0 RUNNING = 1 FINISHED = 2 FINISHED_CYCLE = 3

Parameters

axis –

Returns

Error code status: Status test: Name of test which ran last

Return type

error_code

getTestParameters(*axis: int*) → str

Get test parameters the current test on the given axis is executed with For debugging only.

Parameters:

axis:

Returns:

error_code: Error code parameters: Parameters object as stringified JSON object where “default” is the applied parameter

Example “{“axis”: {“friendlyName”: “Axis”, “default”: 0}, “min_amplitude”: {“friendlyName”: “Min.
amplitude (V)”, “default”: “45”}, “max_amplitude”: {“friendlyName”: “Max. amplitude
(V)”, “default”: “60”}, “start_position”: {“friendlyName”: “Start position”, “default”: 0} }”

getTestplatz() → int

Gets the number of the Testplatz where the AMC belongs to For debugging only. :returns: Error code

testplatz: Number of Testplatz

Return type

error_code

setPositionerSN(axis: int, sn: str)

Sets the serial number of the positioner connected to a given axis. For debugging only.

Parameters

- **axis** – Axis the positioner is connected to
- **sn** – Serial number of the positioner

setTestplatz(testplatz: int)

Sets the number of the Testplatz where the AMC belongs to For debugging only.

Parameters

testplatz – Number of Testplatz

stopCurrentTest(axis: int)

Stops the current test if it is stoppable For debugging only.

Parameters

axis –

atto_device.AMC.update module

class atto_device.AMC.update.**Update(device)**

Bases: object

getLicenseUpdateProgress() → int

Get the progress of running license update :returns: errNo errorCode

value_int: int: progress in percent

Return type

value_errNo

getSwUpdateProgress() → int

Get the progress of running update :returns: errNo errorCode

value_int: int: progress in percent

Return type

value_errNo

licenseUpdateBase64()

Execute the license update with base64 file uploaded.

softwareUpdateBase64()

Execute the update with base64 file uploaded.

uploadLicenseBase64(*offset: int, b64Data: str*)

Upload new license file in format base 64

Parameters

- **offset** – int: offset of the data
- **b64Data** – string: base64 data

uploadSoftwareImageBase64(*offset: int, b64Data: str*)

Upload new firmware image in format base 64

Parameters

- **offset** – int: offset of the data
- **b64Data** – string: base64 data

Module contents**atto_device.CRYO2100 package****Submodules****atto_device.CRYO2100.ACS module**

exception atto_device.CRYO2100.ACS.**AttoException**(*errorText=None, errorNumber=0*)

Bases: Exception

class atto_device.CRYO2100.ACS.**AttoResult**(*resultDict*)

Bases: object

class atto_device.CRYO2100.ACS.**Device**(*address*)

Bases: object

TCP_PORT = 9090

close()

Closes the connection to the device.

connect()

Initializes and connects the selected AMC device.

static discover(*cls*)

getResponse(*request_id*)

handleError(*response, ignoreFunctionError=False*)

is_open = False

printError(*errorNumber*)

Converts the errorNumber into an error string and prints it to the console. :param errorNumber: :type errorNumber: int


```

request(method, params=False)
    Synchronous request.

request_id = 250464

request_id_lock = <unlocked _thread.lock object>

response_buffer = {}

sendRequest(method, params=False)

```

atto_device.CRYO2100.about module

```

class atto_device.CRYO2100.about.About(device)
    Bases: object

    getInstalledPackages() → str
        Get list of packages installed on the device :returns: errNo errorCode
        value_string: string: Comma separated list of packages

        Return type
        value_errNo

    getPackageLicense(pckg: str) → str
        Get the license for a specific package

        Parameters
        pckg – string: Package name

        Returns
        errNo errorCode value_string: string: License for this package

        Return type
        value_errNo

```

atto_device.CRYO2100.access module

```

class atto_device.CRYO2100.access.Access(device)
    Bases: object

    getLockStatus()
        This function returns if the device is locked and if the current client is authorized to use the device. :returns:
        errNo errorCode

        value_locked: locked Is the device locked? value_authorized: authorized Is the client autho-
        rized?

        Return type
        value_errNo

```

grantAccess(password: str)

Grants access to a locked device for the requesting IP by checking against the password

Parameters

password – string the current password

lock(password: str)

This function locks the device with a password, so the calling of functions is only possible with this password. The locking IP is automatically added to the devices which can access functions

Parameters

password – string the password to be set

unlock(password: str)

This function unlocks the device, so it will not be necessary to execute the grantAccess function to run any function

Parameters

password – string the current password

atto_device.CRYO2100.action module

class atto_device.CRYO2100.action.Action(device)

Bases: object

autotunePID(channel: int, tuningType: int)

Autotune heater PID

Parameters

- **channel** – Channel number
- **tuningType** – 0 tune P, 1 tune PI, 2 tune PID

cancelCurrentCommand()

Cancels the current command

getCurrentCommand() → str

Get current command name :returns: No error = 0

current_command:

Return type

errorNumber

getCurrentCommandStatus() → str

Get current command status :returns: No error = 0

current_command:

Return type

errorNumber

getGoToBaseRampRateSetting() → float

Get sample ramp rate :returns: Rate: Ramp rate in Kelvin / minute. 0.1 - 100. 0.0 means ramp limit is off.
:rtype: errorcode

getSampleExchangeRampRateSetting() → float

Get sample ramp rate :returns: Rate: Ramp rate in Kelvin / minute. 0.1 - 100. 0.0 means ramp limit is off.
:rtype: errorcode

getWaitForEvent() → str

Gets the event the cryostat waits for currently :returns: No error = 0

event: event the cryostat waits for

Return type

errorNumber

goToBase()

Cool down to base temperature

goToBaseSoft()

Cool down to base temperature with fixed ramp rate

postEvent(eventName: str)

Posts the event. See: getWaitForEvent()

Parameters

eventName –

sampleExchange()

Start sample exchange

sampleExchangeSoft()

Start sample exchange with fixed ramp rate

setGoToBaseRampRateSetting(rampRate: float)

Set sample ramp rate

Parameters

rampRate – Ramp rate in Kelvin / minute. 0.1 - 100. 0.0 means ramp limit is off.

setSampleExchangeRampRateSetting(rampRate: float)

Set sample ramp rate

Parameters

rampRate – Ramp rate in Kelvin / minute. 0.1 - 100. 0.0 means ramp limit is off.

shutdown()

Start shutdown

switchMagnetsOff()

Switches magnets off

atto_device.CRYO2100.attoDry2100 module

```
class atto_device.CRYO2100.attoDry2100.Device(address)
```

Bases: *Device*

```
atto_device.CRYO2100.attoDry2100.discover()
```

atto_device.CRYO2100.condenser module

```
class atto_device.CRYO2100.condenser.Condenser(device)
```

Bases: object

```
downloadCalibrationCurve() → str
```

Gets the condenser sensor .340 calibration curve :returns: No error = 0

calibration_data:

Return type

errorNumber

```
getHeaterAllZoneRampRates() → float
```

Get condenser heater all zones ramp rate :returns: Error code

rampRate: Ramp rate for this zone 0.1 100 K/min

Return type

errorcode

```
getHeaterDefaultZoneSettings(zone: int)
```

Get condenser heater default zone settings

Parameters

zone – Zone number

Returns

Error code upperbound: Upper set point boundary of this zone in kelvin P: I: D: manualOutput: Manual output for this zone 0 to 100% heatingRange: Heating range see class HeaterRanges and Lake Shore 336 manual for ZONE command rampRate: Ramp rate for this zone 0.1 100 K/min

Return type

errorcode

```
getHeaterHeatingMode() → int
```

Gets the condenser heater heating mode :returns: No error = 0

mode:

Return type

errorNumber

```
getHeaterNumberOfDefaultZones() → int
```

Get the number of the condenser heater default zones :returns: Error code

number_of_zones:

Return type
errorcode

getHeaterNumberOfZones() → int

Get the number of the condenser heater zones :returns: Error code

number_of_zones:

Return type
errorcode

getHeaterPower() → float

Gets the condenser heater power :returns: No error = 0

power:

Return type
errorNumber

getHeaterRampData()

Get condenser heater ramp data :returns: OnOff: Ramp is on or off

Rate: Ramp rate in Kelvin / minute. 0.1 - 100. 0.0 means ramp limit is off.

Return type
errorcode

getHeaterRes() → float

Gets the condenser heater resistance :returns: No error = 0

resistance:

Return type
errorNumber

getHeaterStatus()

Gets the condenser heater status

getHeaterWireRes() → float

Gets the condenser heater wire resistance :returns: No error = 0

wire_resistance:

Return type
errorNumber

getHeaterZoneSettings(zone: int)

Get condenser heater zone settings

Parameters
zone – Zone number

Returns

Error code upperbound: Upper set point boundary of this zone in kelvin P: I: D: manualOutput: Manual output for this zone 0 to 100% heatingRange: Heating range see class HeaterRanges and Lake Shore 336 manual for ZONE command rampRate: Ramp rate for this zone 0.1 100 K/min

Return type
errorCode

getInputFilterSettings()

Gets the condenser input filter settings :returns: No error = 0

off_or_on: point: window:

Return type
errorNumber

getMaxPower() → float

Gets the condenser heater maximum power :returns: No error = 0

maxPower:

Return type
errorNumber

getPID()

Gets the condenser heater PID values :returns: No error = 0

proportional: integral: derivative:

Return type
errorNumber

getRampControlStatus() → bool

Get condenser heater ramp control status :returns: OnOff: Ramp is on or off :rtype: errorCode

getRampRate() → float

Get condenser ramp rate :returns: Rate: Ramp rate in Kelvin / minute. 0.1 - 100. 0.0 means ramp limit is off. :rtype: errorCode

getResistance() → float

Gets the condenser temperature resistance :returns: No error = 0

condenser_resistance:

Return type
errorNumber

getSetPoint() → float

Gets the condenser set point :returns: No error = 0

set_point:

Return type
errorNumber

getTempControlStatus() → bool

Gets the condenser heater status :returns: No error = 0

Status:

Return type
errorNumber

getTemperature() → float

Gets the condenser temperature :returns: No error = 0

condenser_temperature:

Return type

errorNumber

heaterRampOff()

Set condenser heater ramp off

heaterRampOn(rate: float)

Set condenser heater ramp on

Parameters

rate – Ramp rate in Kelvin / minute 0.1 - 100

setD(value: float)

Sets the condenser heater D value

Parameters

value –

setHeaterAllZoneRampRates(rampRate: float)

Set condenser heater all zones ramp rate

Parameters

rampRate – Ramp rate for this zone 0.1 100 K/min

setHeaterPower(value: float)

Sets the condenser heater power

Parameters

value –

setHeaterRes(value: float)

Sets the condenser heater resistance

Parameters

value –

setHeaterWireRes(value: float)

Sets the condenser heater wire resistance

Parameters

value –

setHeaterZoneSettings(zone: int, upperbound: float, p_input: float, i_input: float, d_input: float, manualOutput: float, heatingRange: int)

Set condenser heater zone settings

Parameters

- **zone** – Zone number
- **upperbound** – Upper set point boundary of this zone in kelvin
- **p_input** –
- **i_input** –
- **d_input** –

- **manualOutput** – Manual output for this zone 0 to 100%
- **heatingRange** – Heating range see class HeaterRanges and Lake Shore 336 manual for ZONE command

setI(*value: float*)

Sets the condenser heater I value

Parameters

value –

setInputFilterSettings(*filterOn: bool, numberOfPoints: int, windowSize: int*)

Sets the condenser input filter settings

Parameters

- **filterOn** –
- **numberOfPoints** –
- **windowSize** –

setMaxPower(*value: float*)

Sets the condenser heater maximum power

Parameters

value –

setP(*value: float*)

Sets the condenser heater P value

Parameters

value –

setPID(*proportional: float, integral: float, derivative: float*)

Sets the condenser heater PID values

Parameters

- **proportional** –
- **integral** –
- **derivative** –

setRampRate(*rate: float*)

Set condenser ramp rate

Parameters

rate – Ramp rate in Kelvin / minute. 0.1 - 100. 0.0 means ramp limit is off.

setSetPoint(*setPoint: float*)

Sets the condenser set point

Parameters

setPoint –

startHeaterOpenLoopPower()

Starts the condenser heater in open loop mode with the previously set power

startHeaterZoneMode()

Starts the condenser heater in zone mode

startRampControl()

Set condenser heater ramp on

startTempControl()

Starts the condenser heater control

stopRampControl()

Set condenser heater ramp off (same as heaterRampOff())

stopTempControl()

Stops the condenser heater control

uploadCalibrationCurve(*calibrationData: str*)

Sets the condenser sensor .340 calibration curve may time out, but the upload will still work

Parameters

calibrationData –

atto_device.CRYO2100.cryoInValve module

class atto_device.CRYO2100.cryoInValve.**CryoInvalve**(*device*)

Bases: object

close()

Closes the cryostat in valve

getStatus() → bool

Gets the cryostat in valve status :returns: No error = 0

cryostat_in_valve_status:

Return type

errorNumber

open()

Opens the cryostat in valve

atto_device.CRYO2100.cryoOutValve module

class atto_device.CRYO2100.cryoOutValve.**CryoOutvalve**(*device*)

Bases: object

close()

Closes the cryostat out valve

getStatus() → bool

Gets the cryostat out valve status :returns: No error = 0

pump_valve_status:

Return type

errorNumber

open()

Opens the cryostat out valve

atto_device.CRYO2100.dumpInValve module

```
class atto_device.CRYO2100.dumpInValve.Dumpinvalve(device)
    Bases: object

    close()
        Closes the dump in valve

    getStatus() → bool
        Gets the dump in valve status :returns: No error = 0
        dump_in_valve_status:

        Return type
            errorNumber

    open()
        Opens the dump in valve
```

atto_device.CRYO2100.dumpOutValve module

```
class atto_device.CRYO2100.dumpOutValve.Dumpoutvalve(device)
    Bases: object

    close()
        Closes the dump out valve

    getStatus() → bool
        Gets the dump out valve status :returns: No error = 0
        dump_out_valve_status:

        Return type
            errorNumber

    open()
        opens the dump out valve
```

atto_device.CRYO2100.exchange module

```
class atto_device.CRYO2100.exchange.Exchange(device)
    Bases: object

    downloadCalibrationCurve() → str
        Gets the exchange sensor calibration curve :returns: No error = 0
        calibration_data:

        Return type
            errorNumber
```

downloadCalibrationCurve340() → str

Gets the exchange sensor .340 calibration curve :returns: No error = 0

calibration_data:

Return type

errorNumber

getHeaterAllZoneRampRates() → float

Get exchange heater all zones ramp rate :returns: Error code

rampRate: Ramp rate for this zone 0.1 100 K/min

Return type

errorcode

getHeaterDefaultZoneSettings(zone: int)

Get VTI heater zone settings

Parameters

zone – Zone number

Returns

Error code upperbound: Upper Setpoint boundary of this zone in kelvin P: P I: I D: D manualOutput: Manual output for this zone 0 to 100% heatingRange: Heating range see class HeaterRanges and Lake Shore 336 manual for ZONE command rampRate: Ramp rate for this zone 0.1 100 K/min

Return type

errorcode

getHeaterHeatingMode() → int

Gets the exchange heater heating mode :returns: No error = 0

mode:

Return type

errorNumber

getHeaterNumberOfDefaultZones() → int

Get the number of the VTI heater default zones :returns: Error code

number_of_zones:

Return type

errorcode

getHeaterNumberOfZones() → int

Get the number of the VTI heater zones :returns: Error code

number_of_zones:

Return type

errorcode

getHeaterPower() → float

Gets the exchange heater power :returns: No error = 0

power:

Return type

errorNumber

getHeaterRampData()

Get exchange heater ramp data :returns: OnOff: Ramp is on or off

Rate: Ramp rate in Kelvin / minute. 0.1 - 100. 0.0 means ramp limit is off.

Return type

errorCode

getHeaterRes() → float

Gets the exchange heater resistance :returns: No error = 0

resistance:

Return type

errorNumber

getHeaterStatus()

Gets the exchange heater status

getHeaterWireRes() → float

Gets the exchange heater wire resistance :returns: No error = 0

wire_resistance:

Return type

errorNumber

getHeaterZoneSettings(zone: int)

Get exchange heater zone settings

Parameters

zone – Zone number

Returns

Error code upperbound: Upper Setpoint boundary of this zone in kelvin P: P I: I D: D manualOutput: Manual output for this zone 0 to 100% heatingRange: Heating range see class HeaterRanges and Lake Shore 336 manual for ZONE command rampRate: Ramp rate for this zone 0.1 100 K/min

Return type

errorCode

getInputFilterSettings()

Gets the vti input filter settings :returns: No error = 0

off_or_on: point: window:

Return type

errorNumber

getMaxPower() → float

Gets the exchange heater maximum power :returns: No error = 0

maxPower:

Return type

errorNumber

getPID()

Gets the exchange heater PID values :returns: No error = 0

proportional: integral: derivative:

Return type

errorNumber

getRampControlStatus() → bool

Set exchange heater ramp on :returns: OnOff: Ramp is on or off :rtype: errorcode

getRampRate() → float

Get exchange ramp rate Returns:

errorcode: Rate: Ramp rate in Kelvin / minute. 0.1 - 100. 0.0 means ramp limit

is off.

getResistance() → float

Gets the exchange temperature resistance :returns: No error = 0

exchange_resistance:

Return type

errorNumber

getSetPoint() → float

Gets the exchange set point :returns: No error = 0

set_point:

Return type

errorNumber

getTempControlStatus() → bool

Gets the exchange heater status :returns: No error = 0

Status:

Return type

errorNumber

getTemperature() → float

Gets the exchange temperature :returns: No error = 0

exchange_temperature:

Return type

errorNumber

heaterRampOff()

Set exchange heater ramp off

heaterRampOn(*rate: float*)

Set exchange heater ramp on

Parameters

rate – Ramp rate in Kelvin / minute 0.1 - 100

setD(*value: float*)

Sets the exchange heater D value

Parameters

value –

setHeaterAllZoneRampRates(*rampRate: float*)

Set exchange heater all zones ramp rate

Parameters

rampRate – Ramp rate for this zone 0.1 100 K/min

setHeaterPower(*value: float*)

Sets the exchange heater power

Parameters

value –

setHeaterRes(*value: float*)

Sets the exchange heater resistance

Parameters

value –

setHeaterWireRes(*value: float*)

Sets the exchange heater wire resistance

Parameters

value –

setHeaterZoneSettings(*zone: int, upperbound: float, P: float, I: float, D: float, manualOutput: float, heatingRange: int*)

Set exchange heater zone settings

Parameters

- **zone** – Zone number
- **upperbound** – Upper Setpoint boundary of this zone in kelvin
- **P** – P
- **I** – I
- **D** – D
- **manualOutput** – Manual output for this zone 0 to 100%
- **heatingRange** – Heating range see class HeaterRanges and Lake Shore 336 manual for ZONE command

setI(*value: float*)

Sets the exchange heater I value

Parameters

value –

setInputFilterSettings(*filterOn: bool, numberOfPoints: int, windowSize: int*)

Sets the vti input filter settings

Parameters

- **filterOn** –
- **numberOfPoints** –
- **windowSize** –

setMaxPower(*value: float*)

Sets the exchange heater maximum power

Parameters

value –

setP(*value: float*)

Sets the exchange heater P value

Parameters

value –

setPID(*proportional: float, integral: float, derivative: float*)

Sets the exchange heater PID values

Parameters

- **proportional** –
- **integral** –
- **derivative** –

setRampRate(*rate: float*)

Set exchange ramp rate

Parameters

rate – Ramp rate in Kelvin / minute. 0.1 - 100. 0.0 means ramp limit is off.

setSetPoint(*temperature: float*)

Sets the exchange set point

Parameters

temperature –

startHeaterOpenLoopPower()

Starts the exchange heater in open loop mode with the previously set power

startHeaterZoneMode()

Starts the exchange heater in zone mode

startRampControl()

Set exchange heater ramp on

startTempControl()

Starts the exchange heater

stopRampControl()

Set exchange heater ramp off Same as heaterRampOff()

stopTempControl()

Stops the exchange heater

uploadCalibrationCurve(*calibrationData: str*)

Sets the exchange sensor calibration curve May time out, but the upload will still work

Parameters

calibrationData –

uploadCalibrationCurve340(*calibrationData: str*)

Sets the exchange sensor .340 calibration curve May time out, but the upload will still work

Parameters

calibrationData –

atto_device.CRYO2100.functions module

class atto_device.CRYO2100.functions.**Functions**(*device*)

Bases: object

checkAMCinRack()

If AMC is on Rack position 0, use it as DHCP server, else use it as DHCP client

atto_device.CRYO2100.magnet module

class atto_device.CRYO2100.magnet.**Magnet**(*device*)

Bases: object

downloadCalibrationCurve() → str

Gets the magnet sensor .340 calibration curve :returns: No error = 0

calibration_data:

Return type

errorNumber

getDefaultRampRate(*index: int, channel: int*)

Get the factory defaults for the Ramp Rates

Parameters

- **index** –
- **channel** –

Returns

No error = 0 range: rate:

Return type

errorNumber

getDrivenMode(*channel: int*) → bool

Checks if Magnet is in driven Mode (equivalent to NOT getPersistentMode)

Parameters

channel –

Returns

No error = 0 drivenMode_on_or_off:

Return type

errorNumber

getFieldControl(*channel: int*) → bool

Get whether the magnetic field is being controlled

Parameters

channel –

Returns

No error = 0 field_control_status:

Return type

errorNumber

getFieldsInLeads(*channel: int*) → float

Gets the current in the Leads but in Field units

Parameters

channel –

Returns

No error = 0 field:

Return type

errorNumber

getH(*channel: int*) → float

Gets the magnetic field

Parameters

channel –

Returns

No error = 0 field:

Return type

errorNumber

getHSetPoint(*channel: int*) → float

Gets the magnetic field set point

Parameters

channel –

Returns

No error = 0 setPoint:

Return type

errorNumber

getHSetPoint3D()

Gets the magnetic field set points :returns: No error = 0

setPointZ: setPointY: setPointX:

Return type

errorNumber

getHState(*channel: int*) → str

Gets the state of the magnet state machine for a channel

Parameters

channel –

Returns

No error = 0 state:

Return type

errorNumber

getInputFilterSettings()

Gets the magnet input filter settings :returns: No error = 0

off_or_on: point: window:

Return type

errorNumber

getIsInQuenchState() → bool

Whether the cryo is currently in quench state :returns: No error = 0

state:

Return type

errorNumber

getLeadsHot() → bool

Checks if current is running in the PS Leads :returns: No error = 0

onOrOff:

Return type

errorNumber

getMagnetChannelName(*channel: int*) → str

Gets the Name of magnets

Parameters

channel –

Returns

No error = 0 name:

Return type

errorNumber

getMagnetStateTimer(*channel: int*)

Gets the timer progress of the current magnet SM state for a channel

Parameters

channel –

Returns

No error = 0 time_to_wait: if not in waiting state = -1.0 time_elapsed: if not in waiting state = -1.0

Return type

errorNumber

getNumDefaultRampRates(*channel: int*) → int

Gets the number of default RampRates

Parameters

channel –

Returns

NumberOfRates:

Return type

errorNumber

getNumRampRates(*channel: int*) → int

Return the number of RampRates

Parameters

channel –

Returns

num:

Return type

errorNumber

getNumberOfMagnetChannels() → int

Gets the number of magnets :returns: No error = 0

channels:

Return type

errorNumber

getPause3D() → bool

Checks if at least one of the power supply channels is paused :returns: No error = 0

paused:

Return type

errorNumber

getPersistentMode(*channel: int*) → bool

Checks if Magnet is in persistent mode (equivalent to NOT getDrivenMode)

Parameters

channel –

Returns

No error = 0 drivenMode_on_or_off:

Return type

errorNumber

getPersistentMode3D() → bool

Gets combined persistent mode :returns: No error = 0

present:

Return type

errorNumber

getPersistentSwitchHeaterStatus(channel: int) → bool

Gets the state of the Persistent Heater

Parameters

channel –

Returns

No error = 0 state:

Return type

errorNumber

getPersistentSwitchPresent(channel: int) → bool

Gets whether the persistent switch option is present for a channel

Parameters

channel –

Returns

No error = 0 present:

Return type

errorNumber

getRampRate(channel: int, index: int)

Get ramp rate at index for channel

Parameters

• **channel** –

• **index** –

Returns

No error = 0 range: rate:

Return type

errorNumber

getResistance() → float

Gets the magnet temperature resistance. There is only one for all magnets :returns: No error = 0

resistance:

Return type

errorNumber

getTemperature() → float

Gets the magnet temperature. There is only one for all magnets :returns: No error = 0

temperature:

Return type

errorNumber

getVolt(*channel: int*) → float

Gets the magnetic field voltage

Parameters**channel** –**Returns**

No error = 0 field:

Return type

errorNumber

resetQuenchState()

Let's reset the quench event

setDrivenMode(*channel: int, onOrOff: bool*)

Set driven mode for a specific channel

Parameters

- **channel** –
- **onOrOff** – drivenMode on or off

setHSetPoint(*channel: int, setPoint: float*)

Sets the magnetic field set point

Parameters

- **channel** –
- **setPoint** –

setHSetPoint3D(*setPointZ: float, setPointY: float, setPointX: float*)

Sets the magnetic field set points

Parameters

- **setPointZ** –
- **setPointY** –
- **setPointX** –

setInputFilterSettings(*filterOn: bool, numberOfPoints: int, windowSize: int*)

Sets the magnet input filter settings

Parameters

- **filterOn** –
- **numberOfPoints** –
- **windowSize** –

setPause3D(*onOff: bool*)

Sets the pause state of the power supply channels

Parameters**onOff** –

setPersistentMode3D(*onOff: bool*)

Sets the combined persistent mode

Parameters

onOff –

setRampRate(*channel: int, index: int, range_this: float, rate: float*)

Change the ramp rates

Parameters

- **channel** –
- **index** –
- **range_this** –
- **rate** –

startFieldControl(*channel: int*)

Starts the magnetic field control

Parameters

channel –

stopFieldControl(*channel: int*)

Stops the magnetic field control

Parameters

channel –

stopMagnet3D()

kills the magnetPowerSupplyObjectJT.py should be restarted by main.py

uploadCalibrationCurve(*calibrationData: str*)

Sets the magnet sensor .340 calibration curve may time out, but the upload will still work

Parameters

calibrationData –

atto_device.CRYO2100.main module

class atto_device.CRYO2100.main.**Main**(*device*)

Bases: object

changeCryostatType(*newType: str*)

Change the cryostat type

Parameters

newType –

getAvailableCryostatTypes() → str

Get all available cryostat types :returns: No error = 0

types: CSV string of types

Return type

errorNumber

atto_device.CRYO2100.network module

class atto_device.CRYO2100.network.**Network**(*device*)

Bases: object

apply()

Apply temporary IP configuration and load it

configureWifi(*mode: int, ssid: str, psk: str*)

Change the wifi configuration and applies it

Parameters

- **mode** – 0: Access point, 1: Wifi client
- **ssid** –
- **psk** – Pre-shared key

discard()

Discard temporary IP configuration

getDefaultGateway() → str

Get the default gateway of the device :returns: errNo errorCode

value_Default: Default gateway

Return type

value_errNo

getDnsResolver(*priority: int*) → str

Get the DNS resolver

Parameters

priority – of DNS resolver (Usually: 0 = Default, 1 = Backup)

Returns

errNo errorCode value_IP: IP address of DNS resolver

Return type

value_errNo

getEnableDhcpClient() → bool

Get the state of DHCP client :returns: errNo errorCode

value_boolean: boolean: true = DHCP client enable, false = DHCP client disable

Return type

value_errNo

getEnableDhcpServer() → bool

Get the state of DHCP server :returns: errNo errorCode

value_boolean: boolean: true = DHCP server enable, false = DHCP server disable

Return type

value_errNo

getIpAddress() → str

Get the IP address of the device :returns: errNo errorCode

value_IP: IP address as string

Return type

value_errNo

getProxyServer() → str

Get the proxy settings of the device :returns: errNo errorCode

value_Proxy: Proxy Server String, empty for no proxy

Return type

value_errNo

getRealIpAddress() → str

Get the real IP address of the device set to the network interface (br0, eth1 or eth0) :returns: errNo errorCode

value_IP: IP address as string

Return type

value_errNo

getSubnetMask() → str

Get the subnet mask of the device :returns: errNo errorCode

value_Subnet: Subnet mask as string

Return type

value_errNo

getWifiMode() → int

Get the operation mode of the wifi adapter :returns: errNo errorCode

value_mode: mode 0: Access point, 1: Wifi client

Return type

value_errNo

getWifiPassphrase() → str

Get the the passphrase of the network hosted (mode: Access point) or connected to (mode: client) :returns: errNo errorCode

value_psk: psk Pre-shared key

Return type

value_errNo

getWifiPresent() → bool

Returns is a Wifi interface is present :returns: errNo errorCode

value_True: True, if interface is present

Return type

value_errNo

getWifiSSID() → str

Get the the SSID of the network hosted (mode: Access point) or connected to (mode: client) :returns:
errNo errorCode

SSID: SSID

Return type

value_errNo

setDefaultGateway(gateway: str)

Set the default gateway of the device

Parameters

gateway – Default gateway as string

setDnsResolver(priority: int, resolver: str)

Set the DNS resolver

Parameters

- **priority** – of DNS resolver (Usually: 0 = Default, 1 = Backup)
- **resolver** – The resolver's IP address as string

setEnabledDhcpClient(enable: bool)

Enable or disable DHCP client

Parameters

enable – boolean: true = enable DHCP client, false = disable DHCP client

setEnabledDhcpServer(enable: bool)

Enable or disable DHCP server

Parameters

enable – boolean: true = enable DHCP server, false = disable DHCP server

setIpAddress(address: str)

Set the IP address of the device

Parameters

address – IP address as string

setProxyServer(proxyServer: str)

Set the proxy server of the device

Parameters

proxyServer – Proxy Server Setting as string

setSubnetMask(netmask: str)

Set the subnet mask of the device

Parameters

netmask – Subnet mask as string

setWifiMode(mode: int)

Change the operation mode of the wifi adapter

Parameters

mode – 0: Access point, 1: Wifi client

setWifiPassphrase(*psk: str*)

Change the passphrase of the network hosted (mode: Access point) or connected to (mode: client)

Parameters

psk – Pre-shared key

setWifiSSID(*ssid: str*)

Change the SSID of the network hosted (mode: Access point) or connected to (mode: client)

Parameters

ssid –

verify()

Verify that temporary IP configuration is correct

atto_device.CRYO2100.pressures module

class atto_device.CRYO2100.pressures.**Pressures**(*device*)

Bases: object

getCryoInPressure() → float

Gets the cryostat in pressure :returns: No error = 0

cryostat_in_pressure:

Return type

errorNumber

getCryoOutPressure() → float

Gets the cryostat out pressure :returns: No error = 0

cryostat_out_pressure:

Return type

errorNumber

getDumpPressure() → float

Gets the dump pressure :returns: No error = 0

dump_pressure:

Return type

errorNumber

atto_device.CRYO2100.sample module

class atto_device.CRYO2100.sample.**Sample**(*device*)

Bases: object

downloadCalibrationCurve() → str

Gets the sample sensor .340 calibration curve :returns: No error = 0

calibration_data:

Return type

errorNumber

getHeaterAllZoneRampRates() → float

Get sample heater all zones ramp rate :returns: Error code

rampRate: Ramp rate for this zone 0.1 100 K/min

Return type

errorCode

getHeaterDefaultZoneSettings(*zone: int*)

Get sample heater zone settings

Parameters

zone – Zone number

Returns

Error code upperbound: Upper set point boundary of this zone in kelvin P: P I: I D: D manualOutput: Manual output for this zone 0 to 100% heatingRange: Heating range see class HeaterRanges and Lake Shore 336 manual for ZONE command rampRate: Ramp rate for this zone 0.1 100 K/min

Return type

errorCode

getHeaterHeatingMode() → int

Gets the sample heater heating mode :returns: No error = 0

mode:

Return type

errorNumber

getHeaterNumberOfDefaultZones() → int

Get the number of sample heater default zones :returns: Error code

number_of_zones:

Return type

errorCode

getHeaterNumberOfZones() → int

Get the number of the sample heater zones :returns: Error code

number_of_zones:

Return type
errorCode

getHeaterPower() → float

Gets the sample heater power :returns: No error = 0

power:

Return type
errorNumber

getHeaterRampData()

Get sample heater ramp data :returns: OnOff: Ramp is on or off

Rate: Ramp rate in Kelvin / minute. 0.1 - 100. 0.0 means ramp limit is off.

Return type
errorCode

getHeaterRes() → float

Gets the sample heater resistance :returns: No error = 0

resistance:

Return type
errorNumber

getHeaterStatus()

Gets the sample heater status

getHeaterWireRes() → float

Gets the sample heater wire resistance :returns: No error = 0

wire_resistance:

Return type
errorNumber

getHeaterZoneSettings(zone: int)

Get sample heater zone settings

Parameters
zone – Zone number

Returns

Error code upperbound: Upper set point boundary of this zone in kelvin P: P I: I D: D manualOutput: Manual output for this zone 0 to 100% heatingRange: Heating range see class HeaterRanges and Lake Shore 336 manual for ZONE command rampRate: Ramp rate for this zone 0.1 100 K/min

Return type
errorCode

getInputFilterSettings()

Gets the sample input filter settings :returns: No error = 0

off_or_on: point: window:

Return type
errorNumber

getMaxPower() → float

Gets the sample heater maximum power :returns: No error = 0

maxPower:

Return type
errorNumber

getPID()

Gets the sample heater PID values :returns: No error = 0

proportional: integral: derivative:

Return type
errorNumber

getRampControlStatus() → bool

Get the ramp control status :returns: OnOff: Ramp is on or off :rtype: errorcode

getRampRate() → float

Get sample ramp rate :returns: Rate: Ramp rate in Kelvin / minute. 0.1 - 100. 0.0 means ramp limit is off.
:rtype: errorcode

getResistance() → float

Gets the sample resistance :returns: No error = 0

sample_resistance:

Return type
errorNumber

getSetPoint() → float

Gets the sample set point :returns: No error = 0

set_point:

Return type
errorNumber

getTempControlStatus() → bool

Gets the sample heater status :returns: No error = 0

status:

Return type
errorNumber

getTemperature() → float

Gets the sample temperature :returns: No error = 0

sample_temperature:

Return type
errorNumber

heaterRampOff()

Set sample heater ramp off

heaterRampOn(*rate: float*)

Set sample heater ramp on

Parameters

rate – Ramp rate in Kelvin / minute 0.1 - 100

setD(*value: float*)

Sets the sample heater D value

Parameters

value –

setHeaterAllZoneRampRates(*rampRate: float*)

Set sample heater all zones ramp rate

Parameters

rampRate – Ramp rate for this zone 0.1 100 K/min

setHeaterPower(*value: float*)

Sets the sample heater power

Parameters

value –

setHeaterRes(*value: float*)

Sets the sample heater resistance

Parameters

value –

setHeaterWireRes(*value: float*)

Sets the sample heater wire resistance

Parameters

value –

setHeaterZoneSettings(*zone: int, upperbound: float, p_input: float, i_input: float, d_input: float, manualOutput: float, heatingRange: int*)

Set sample heater zone settings

Parameters

- **zone** – Zone number
- **upperbound** – Upper set point boundary of this zone in kelvin
- **p_input** – P
- **i_input** – I
- **d_input** – D
- **manualOutput** – Manual output for this zone 0 to 100%
- **heatingRange** – Heating range see class HeaterRanges and Lake Shore 336 manual for ZONE command

setI(*value: float*)

Sets the sample heater I value

Parameters

value –

setInputFilterSettings(*filterOn: bool, numberOfPoints: int, windowSize: int*)

Sets the sample input filter settings

Parameters

- **filterOn** –
- **numberOfPoints** –
- **windowSize** –

setMaxPower(*value: float*)

Sets the sample heater maximum power

Parameters

value –

setP(*value: float*)

Sets the sample heater P value

Parameters

value –

setPID(*proportional: float, integral: float, derivative: float*)

Sets the sample heater PID values

Parameters

- **proportional** –
- **integral** –
- **derivative** –

setRampRate(*rate: float*)

Set sample ramp rate

Parameters

rate – Ramp rate in Kelvin / minute. 0.1 - 100. 0.0 means ramp limit is off.

setSetPoint(*setPoint: float*)

Sets the sample set point

Parameters

setPoint –

startHeaterOpenLoopPower()

Starts the sample heater in open loop mode with the previously set power

startHeaterZoneMode()

Starts the sample heater in zone mode

startRampControl()

Start controlling the ramping

startTempControl()

Starts the sample heater

stopRampControl()

Stop controlling the ramping

stopTempControl()

Stops the sample heater

uploadCalibrationCurve(*calibrationData: str*)

Sets the sample sensor .340 calibration curve may time out, but the upload will still work

Parameters

calibrationData –

atto_device.CRYO2100.scrollPump module

class atto_device.CRYO2100.scrollPump.**Scrollpump**(*device*)

Bases: object

getFrequency() → float

Gets the scroll pump frequency :returns: No error = 0

scroll_pump_frequency:

Return type

errorNumber

getStatus() → bool

Gets the scroll pump status :returns: No error = 0

scroll_pump_status:

Return type

errorNumber

getStatusString() → str

Gets the scroll pump status as string :returns: No error = 0

scroll_pump_status_string:

Return type

errorNumber

startFullSpeed()

Starts the scroll pump at full speed

startStandbySpeed()

Starts the scroll pump at standby speed

stop()

Stops the scroll pump

atto_device.CRYO2100.stage40k module**class** atto_device.CRYO2100.stage40k.**Stage40k**(device)

Bases: object

downloadCalibrationCurve() → str

Gets the Stage40K sensor .340 calibration curve :returns: No error = 0

calibration_data:

Return type

errorNumber

getHeaterAllZoneRampRates() → float

Get Stage40K heater all zones ramp rate :returns: Error code

rampRate: Ramp rate for this zone 0.1 100 K/min

Return type

errorcode

getHeaterDefaultZoneSettings(zone: int)

Get Stage40K heater zone settings

Parameters**zone** – Zone number**Returns**

Error code upperbound: Upper set point boundary of this zone in kelvin P: I: D: manualOutput: Manual output for this zone 0 to 100% heatingRange: Heating range see class HeaterRanges and Lake Shore 336 manual for ZONE command rampRate: Ramp rate for this zone 0.1 100 K/min

Return type

errorcode

getHeaterHeatingMode() → int

Gets the Stage40K heater heating mode :returns: No error = 0

mode:

Return type

errorNumber

getHeaterNumberOfDefaultZones() → int

Get the number of the Stage40K heater default zones :returns: Error code

number_of_zones:

Return type

errorcode

getHeaterNumberOfZones() → int

Get the number of the Stage40K heater zones :returns: Error code

number_of_zones:

Return type
errorCode

getHeaterPower() → float

Gets the Stage40K heater power :returns: No error = 0
power:

Return type
errorNumber

getHeaterRampData()

Get Stage40K heater ramp data :returns: OnOff: Ramp is on or off
Rate: Ramp rate in Kelvin / minute. 0.1 - 100. 0.0 means ramp limit is off.

Return type
errorCode

getHeaterRes() → float

Gets the Stage40K heater resistance :returns: No error = 0
resistance:

Return type
errorNumber

getHeaterStatus()

Gets the Stage40K heater status

getHeaterWireRes() → float

Gets the Stage40K heater wire resistance :returns: No error = 0
wire_resistance:

Return type
errorNumber

getHeaterZoneSettings(zone: int)

Get Stage40K heater zone settings

Parameters
zone – Zone number

Returns

Error code upperbound: Upper set point boundary of this zone in kelvin P: P I: I D: D
manualOutput: Manual output for this zone 0 to 100% heatingRange: Heating range see
class HeaterRanges and Lake Shore 336 manual for ZONE command rampRate: Ramp
rate for this zone 0.1 100 K/min

Return type
errorCode

getInputFilterSettings()

Gets the Stage40K input filter settings :returns: No error = 0
off_or_on: point: window:

Return type
errorNumber

getMaxPower() → float

Gets the Stage40K heater maximum power :returns: No error = 0

maxPower:

Return type
errorNumber

getPID()

Gets the Stage40K heater PID values :returns: No error = 0

proportional: integral: derivative:

Return type
errorNumber

getRampControlStatus() → bool

Get Stage40K heater ramp control status :returns: OnOff: Ramp is on or off :rtype: errorcode

getRampRate() → float

Get Stage40K ramp rate :returns: Rate: Ramp rate in Kelvin / minute. 0.1 - 100. 0.0 means ramp limit is off. :rtype: errorcode

getResistance() → float

Gets the Stage40K resistance :returns: No error = 0

Stage40K_resistance:

Return type
errorNumber

getSetPoint() → float

Gets the Stage40K set point :returns: No error = 0

set_point:

Return type
errorNumber

getTempControlStatus() → bool

Gets the Stage40K heater status :returns: No error = 0

Status:

Return type
errorNumber

getTemperature() → float

Gets the Stage40K temperature :returns: No error = 0

Stage40K_temperature:

Return type
errorNumber

heaterRampOff()

Set Stage40K heater ramp off

heaterRampOn(*rate: float*)

Set Stage40K heater ramp on

Parameters

rate – Ramp rate in Kelvin / minute 0.1 - 100

setD(*value: float*)

Sets the Stage40K heater D value

Parameters

value –

setHeaterAllZoneRampRates(*rampRate: float*)

Set Stage40K heater all zones ramp rate

Parameters

rampRate – Ramp rate for this zone 0.1 100 K/min

setHeaterPower(*value: float*)

Sets the Stage40K heater power

Parameters

value –

setHeaterRes(*value: float*)

Sets the Stage40K heater resistance

Parameters

value –

setHeaterWireRes(*value: float*)

Sets the Stage40K heater wire resistance

Parameters

value –

setHeaterZoneSettings(*zone: int, upperbound: float, p_input: float, i_input: float, d_input: float, manualOutput: float, heatingRange: int*)

Set Stage40K heater zone settings

Parameters

- **zone** – Zone number
- **upperbound** – Upper set point boundary of this zone in kelvin
- **p_input** –
- **i_input** –
- **d_input** –
- **manualOutput** – Manual output for this zone 0 to 100%
- **heatingRange** – Heating range see class HeaterRanges and Lake Shore 336 manual for ZONE command

setI(*value: float*)

Sets the Stage40K heater I value

Parameters

value –

setInputFilterSettings(*filterOn: bool, numberOfPoints: int, windowSize: int*)

Sets the Stage40K input filter settings

Parameters

- **filterOn** –
- **numberOfPoints** –
- **windowSize** –

setMaxPower(*value: float*)

Sets the Stage40K heater maximum power

Parameters

value –

setP(*value: float*)

Sets the Stage40K heater P value

Parameters

value –

setPID(*proportional: float, integral: float, derivative: float*)

Sets the Stage40K heater PID values

Parameters

- **proportional** –
- **integral** –
- **derivative** –

setRampRate(*rate: float*)

Set Stage40K ramp rate

Parameters

rate – Ramp rate in Kelvin / minute. 0.1 - 100. 0.0 means ramp limit is off.

setSetPoint(*setPoint: float*)

Sets the Stage40K set point

Parameters

setPoint –

startHeaterOpenLoopPower()

Starts the Stage40K heater in open loop mode with the previously set power

startHeaterZoneMode()

Starts the Stage40K heater in zone mode

startRampControl()

Set Stage40K heater ramp control on

startTempControl()

Starts the Stage40K heater

stopRampControl()

Set Stage40K heater ramp control off (same as heaterRampOff())

stopTempControl()

Stops the Stage40K heater

uploadCalibrationCurve(*calibrationData: str*)

Sets the Stage40K sensor .340 calibration curve may time out, but the upload will still work

Parameters

calibrationData –

atto_device.CRYO2100.system module

class atto_device.CRYO2100.system.**System**(*device*)

Bases: object

emptyExternalDatabaseToMaximumBytes()

Empty database! This solved issues when the UI is hanging! Don't use it otherwise

errorNumberToString(*errorNumber: int*)

“Translate” the error code into an error text Currently we only support the system language

Parameters

errorNumber – error code to translate

getAutoprobeInformationJSONd() → str

Get the hardware autoprobe information :returns: No error = 0

infoJson: JSONd autoprobe information

Return type

errorNumber

getAvailableConfigurationSettings() → str

Gets all available configuration settings :returns: No error = 0

Configuration_settings: CSV of configuration settings

Return type

errorNumber

getConfigOverrideFile() → str

Get the EEPROM JSON file containing the config overrides. To know what entries exist have a look at `getAvailableConfigurationSettings()` and `getConfigSettingInformation()` :returns: No error = 0

overrideFile: String containing the override file

Return type

errorNumber

getConfigurationConfiguration(*settingName: str*) → float

Gets the configuration

Parameters

settingName –

Returns

No error = 0 Configuration_value:

Return type

errorNumber

getConfigurationSetting(*settingName: str*) → float

Get the value of the specified configuration setting

Parameters

settingName –

Returns

No error = 0 Configuration_setting_value:

Return type

errorNumber

getConfigurationSettingInformation(*settingName: str*) → str

Gets the configuration setting info string

Parameters

settingName –

Returns

No error = 0 Configuration_settings_information:

Return type

errorNumber

getCurrentStatusJSONd() → str

Get the current cryostat status :returns: No error = 0

status: Cryostat status

Return type

errorNumber

getDatabaseAsyncQueryResult(*handle: int*) → str

Get the query result

Parameters

handle –

Returns

No error = 0 result:

Return type

errorNumber

getDatabaseColumnsMinMaxIntervalRows(*minRow: int, maxRow: int, columnNames: str*) → str

Get the cryostat database columns

Parameters

- **minRow** –

- **maxRow** –
- **columnNames** – Empty string means all available columns

Returns

No error = 0 result_rows:

Return type

errorNumber

getDatabaseMaximumLogSeconds() → float

Get the cryostat long term status of a time interval :returns: No error = 0

maximumSecondsInLog:

Return type

errorNumber

getDatabaseMinMaxResult(*handle: int*)

Get the min and max rows.

Parameters

handle –

Returns

No error = 0 min_row: max_row:

Return type

errorNumber

getDatabaseMinimumLogSeconds() → float

Get the cryostat long term status of a time interval :returns: No error = 0

minimumSecondsInLog:

Return type

errorNumber

getDeviceType() → str

Gets the device type. :returns: No error = 0

type: type of the device

Return type

errorNumber

getFeatures()

Get the features of the cryostat. :returns: No error = 0

Feature_names: Feature_descriptions: Feature_short_descriptions: Feature_display_names:

Feature_picture:

Return type

errorNumber

getFeaturesActivated() → str

Get the active features of the cryostat. :returns: No error = 0

Feature_names:

Return type

localErrorNumber

getFeaturesJSON() → str

Get the features of the cryostat as a JSON string. :returns: No error = 0

JSON:

Return type

localErrorNumber

getIntervalLongTermEntriesJSONd(*secondsOld: float, secondsNew: float*) → str

Get the cryostat long term status of a time interval

Parameters

- **secondsOld** –
- **secondsNew** –

Returns

No error = 0 intervalJson: JSONd description of cryo status in time interval

Return type

errorNumber

getIntervalShortTermEntriesJSONd(*secondsOld: float, secondsNew: float*) → str

Get the cryostat status of a time interval

Parameters

- **secondsOld** –
- **secondsNew** –

Returns

No error = 0 Short_term_log_entries:

Return type

errorNumber

getLastError()

Gets all available information on the last error :returns: Local errorcode of this call

lastErrorNumber: Last error number recommendation: Recommendation component: Component that raised the error command: Command age: Age

Return type

errorNumber

getLastErrorAge() → int

Gets the age of the last error that happened :returns: No error = 0

age: time in seconds since the last error happened

Return type

errorNumber

getLastErrorCommand() → str

Gets the command of the last error that happened :returns: No error = 0

command: command running when the last error happened

Return type

errorNumber

getLastErrorComponent() → str

Gets the component of the last error that happened :returns: No error = 0

component: cryostat component of the error

Return type

errorNumber

getLastErrorNumber() → int

Gets the error number of the last error that happened :returns: No error = 0

errorNumber: No error = 0

Return type

localErrorNumber

getLastErrorRecommendation() → str

Gets the recommendation of the last error that happened :returns: No error = 0

recommendation: error recommendation

Return type

errorNumber

getLastErrorsJSONd() → str

Gets the last cryostat errors as a JSON string. :returns: No error = 0

JSON:

Return type

localErrorNumber

getLastErrorsPickled(*pythonVersion: int*) → str

Gets the last cryostat errors from the snapshot. Use `getLastErrorsAsJSON()` and `unJSON` the data if this call doesn't work for you (the cryostat data can't be unpickled)

Parameters

pythonVersion –

Returns

No error = 0 Pickled_data:

Return type

localErrorNumber

getLastLongTermEntriesJSONd(*seconds: float*) → str

Get the cryostat long term status of the last seconds

Parameters**seconds** –**Returns**

No error = 0 version: firmware version number

Return type

errorNumber

getLastShortTermEntriesJSONd(*seconds: float*) → str

Get the cryostat status of the last seconds

Parameters**seconds** –**Returns**

No error = 0 version: firmware version number

Return type

errorNumber

getLogLines(*maxBytes: int*) → str

Get the last lines of the system logfile.

Parameters**maxBytes** – maximum bytes to read**Returns**

No error = 0 logLines: String containing the logfile lines

Return type

errorNumber

getSecondsFromStartCryostatInstallation() → float

Gets the seconds from start of cryostat installation :returns: No error = 0

seconds: Seconds from start of cryostat installation

Return type

errorNumber

getUserConfigOverrideFile() → strGet the user JSON file containing the config overrides. To know what entries exist have a look at `getAvailableConfigurationSettings()` and `getConfigurationSettingInformation()` :returns: No error = 0

overrideFile: String containing the override file

Return type

errorNumber

lowerError()

Resets the last error

prepareDatabaseColumnsIntervalAsync(*minRow: int, maxRow: int, columnNames: str, handle: int*)

Prepare the cryostat long term status of a time interval query

Parameters

- **minRow** –
- **maxRow** –

- **columnNames** – Empty string means all available columns
- **handle** –

prepareDatabaseMinMaxInterval(*minTime: float, maxTime: float*) → int

Prepare the cryostat long term status of a time interval query Get the row numbers with `getDatabaseMinMaxResult()` Get the data with `getDatabaseColumnsMinMaxIntervalRows()` `minTime` and `maxTime` can be taken from `getDatabaseMinimumLogSeconds()` and `getDatabaseMaximumLogSeconds()`

Parameters

- **minTime** –
- **maxTime** –

Returns

No error = 0 handle:

Return type

errorNumber

resetUserConfigOverrideFile()

Copy the EEPROM config factory defaults to the user config

setConfigurationSetting(*settingName: str, settingValue: float*)

Sets the configuration setting

Parameters

- **settingName** –
- **settingValue** –

setUserConfigOverrideFile(*fileContent: str*) → str

Set the user JSON file containing the config overrides. To know what entries exist have a look at `getAvailableConfigurationSettings()` and `getConfigurationSettingInformation()`

Parameters

fileContent – Contents of the config override file

Returns

No error = 0 hint_string: Free text string to give the caller some hints if things don't work

Return type

errorNumber

atto_device.CRYO2100.system_service module

class atto_device.CRYO2100.system_service.**System_service**(*device*)

Bases: object

apply(*key: int*)

Apply temporary system configuration

Parameters

key –

errorNumberToRecommendation(*language: int, errNbr: int*) → str

Get a recommendation for the error code

Parameters

- **language** – integer: Language code
- **errNbr** – integer: Error code to translate

Returns

errNo errorCode value_string: string: Error recommendation (currently returning an int = 0 until we have recommendations)

Return type

value_errNo

errorNumberToString(*language: int, errNbr: int*) → str

Get a description of an error code

Parameters

- **language** – integer: Language code 0 for the error name, 1 for a more user friendly error message
- **errNbr** – integer: Error code to translate

Returns

errNo errorCode value_string: string: Error description

Return type

value_errNo

factoryReset()

Turns on the factory reset flag.

getDeviceName() → str

Get the actual device name :returns: errNo errorCode

value_string: string: actual device name

Return type

value_errNo

getFirmwareVersion() → str

Get the firmware version of the system :returns: errNo errorCode

value_string: string: The firmware version

Return type

value_errNo

getFluxCode() → str

Get the flux code of the system :returns: errNo errorCode

value_string: string: flux code

Return type

value_errNo

getHostname() → str

Return device hostname :returns: errNo errorCode

available: available

Return type

value_errNo

getMacAddress() → str

Get the mac address of the system :returns: errNo errorCode

value_string: string: Mac address of the system

Return type

value_errNo

getSerialNumber() → str

Get the serial number of the system :returns: errNo errorCode

value_string: string: Serial number

Return type

value_errNo

rebootSystem()

Reboot the system

setDeviceName(*name: str*)

Set custom name for the device

Parameters

name – string: device name

setTime(*day: int, month: int, year: int, hour: int, minute: int, second: int*)

Set system time manually

Parameters

- **day** – integer: Day (1-31)
- **month** – integer: Day (1-12)
- **year** – integer: Day (eg. 2021)
- **hour** – integer: Day (0-23)
- **minute** – integer: Day (0-59)
- **second** – integer: Day (0-59)

softReset()

Performs a soft reset (Reset without deleting the network settings).

updateTimeFromInternet()

Update system time by querying attocube.com

atto_device.CRYO2100.tboard module

class atto_device.CRYO2100.tboard.**Tboard**(*device*)

Bases: object

autotunePIDError() → int

Gets last autotune error :returns: No error = 0

Autotune_errorcode:

Return type

errorNumber

autotunePIDRunning() → bool

Check whether PID autotune is currently running :returns: No error = 0

running:

Return type

errorNumber

autotunePIDStage() → int

Gets last autotune stage :returns: No error = 0

Autotune_errorcode:

Return type

errorNumber

downloadCalibrationCurve(*channel: int*) → str

Gets the sensor .340 calibration curve

Parameters

channel –

Returns

No error = 0 calibration_data:

Return type

errorNumber

getHeaterPIDChannel(*channelNumber: int*)

Get PID values from channel

Parameters

channelNumber –

Returns

No error = 0 p_output: i_output: d_output:

Return type

errorNumber

getHeaterPowerChannel(*channelNumber: int*) → float

Get heater output power

Parameters

channelNumber –

Returns

No error = 0 power:

Return type

errorNumber

getHeaterSetpointChannel(*channelNumber: int*) → float

Get heater setpoint

Parameters

channelNumber –

Returns

No error = 0 set_point:

Return type

errorNumber

getHeaterZoneSettingsChannel(*channelNumber: int*)

Get heater zone settings channel

Parameters

channelNumber –

Returns

No error = 0 upperbound: Upper temperature bound of zone p_output: i_output: d_output: manualOutput: heatingRange: 0 -> off, 1 -> attocube OEM on, 1 -> 335, 336 Low, , 2 -> 335, 336 Medium, , 3 -> 335, 336 High rampRate:

Return type

errorNumber

getRawSensorInput(*channelNumber: int*) → float

Get raw sensor input.

Parameters

channelNumber –

Returns

No error = 0 Raw_sensor_input:

Return type

errorNumber

getTemperature(*channelNumber: int*) → float

Get temperature.

Parameters

channelNumber –

Returns

No error = 0 Temperature:

Return type

errorNumber

isHeatingChannel(*channelNumber: int*) → bool

Check if channel is heating

Parameters

channelNumber –

Returns

No error = 0 isHeating:

Return type

errorNumber

setHeaterPIDChannel(*channelNumber: int, p_input: float, i_input: float, d_input: float*)

Set PID values for channel

Parameters

- **channelNumber** –
- **p_input** –
- **i_input** –
- **d_input** –

setHeaterSetpointChannel(*channelNumber: int, setpoint: float*)

Start heating in zone mode

Parameters

- **channelNumber** –
- **setpoint** – Temperature set point

setHeaterZoneSettingsChannel(*channelNumber: int, zone: int, upperbound: float, p_input: float, i_input: float, d_input: float, manualOutput: float, heatingRange: int, rampRate: float*)

Set zone parameters for heater channel

Parameters

- **channelNumber** –
- **zone** –
- **upperbound** – Upper temperature bound of zone
- **p_input** –
- **i_input** –
- **d_input** –
- **manualOutput** –
- **heatingRange** – 0 -> off, 1 -> attocube OEM on, 1 -> 335, 336 Low, , 2 -> 335, 336 Medium, , 3 -> 335, 336 High
- **rampRate** –

startHeatingChannel(*channelNumber: int*)

Start heating

Parameters

- channelNumber** –

startHeatingOpenLoopChannel(*channelNumber: int, power: float*)

Start heating in open loop mode

Parameters

- **channelNumber** –

- **power** –

startHeatingZoneModeChannel(*channelNumber: int*)

Start heating in zone mode

Parameters

channelNumber –

stopHeatingChannel(*channelNumber: int*)

Stop heating

Parameters

channelNumber –

uploadCalibrationCurve(*channel: int, curveData: str*)

Sets the sensor .340 calibration curve may time out, but the upload will still work

Parameters

- **channel** –
- **curveData** – calibration data

atto_device.CRYO2100.update module

class atto_device.CRYO2100.update.**Update**(*device*)

Bases: object

getLicenseUpdateProgress() → int

Get the progress of running license update :returns: errNo errorCode

value_int: int: progress in percent

Return type

value_errNo

getSwUpdateProgress() → int

Get the progress of running update :returns: errNo errorCode

value_int: int: progress in percent

Return type

value_errNo

licenseUpdateBase64()

Execute the license update with base64 file uploaded.

softwareUpdateBase64()

Execute the update with base64 file uploaded.

uploadLicenseBase64(*offset: int, b64Data: str*)

Upload new license file in format base 64

Parameters

- **offset** – int: offset of the data
- **b64Data** – string: base64 data

uploadSoftwareImageBase64(*offset: int, b64Data: str*)

Upload new firmware image in format base 64

Parameters

- **offset** – int: offset of the data
- **b64Data** – string: base64 data

atto_device.CRYO2100.vti module

class atto_device.CRYO2100.vti.**Vti**(*device*)

Bases: object

downloadCalibrationCurve() → str

Gets the vti sensor .340 calibration curve :returns: No error = 0

calibration_data:

Return type

errorNumber

getHeaterAllZoneRampRates() → float

Get vti heater all zones ramp rate :returns: Error code

rampRate: Ramp rate for this zone 0.1 100 K/min

Return type

errorcode

getHeaterDefaultZoneSettings(*zone: int*)

Get VTI heater zone settings

Parameters

zone – Zone number

Returns

Error code upperbound: Upper set point boundary of this zone in kelvin P: P I: I D: D manualOutput: Manual output for this zone 0 to 100% heatingRange: Heating range see class HeaterRanges and Lake Shore 336 manual for ZONE command rampRate: Ramp rate for this zone 0.1 100 K/min

Return type

errorcode

getHeaterHeatingMode() → int

Gets the vti heater heating mode :returns: No error = 0

mode:

Return type

errorNumber

getHeaterNumberOfDefaultZones() → int

Get the number of the VTI heater default zones :returns: Error code

number_of_zones:

Return type
errorcode

getHeaterNumberOfZones() → int

Get the number of the VTI heater zones :returns: Error code

number_of_zones:

Return type
errorcode

getHeaterPower() → float

Gets the vti heater power :returns: No error = 0

power:

Return type
errorNumber

getHeaterRampData()

Get VTI heater ramp data :returns: OnOff: Ramp is on or off

Rate: Ramp rate in Kelvin / minute. 0.1 - 100. 0.0 means ramp limit is off.

Return type
errorcode

getHeaterRes() → float

Gets the vti heater resistance :returns: No error = 0

resistance:

Return type
errorNumber

getHeaterStatus()

Gets the vti heater status

getHeaterWireRes() → float

Gets the vti heater wire resistance :returns: No error = 0

wire_resistance:

Return type
errorNumber

getHeaterZoneSettings(zone: int)

Get VTI heater zone settings

Parameters
zone – Zone number

Returns

Error code upperbound: Upper set point boundary of this zone in kelvin P: I: D: manualOutput: Manual output for this zone 0 to 100% heatingRange: Heating range see class HeaterRanges and Lake Shore 336 manual for ZONE command rampRate: Ramp rate for this zone 0.1 100 K/min

Return type
errorCode

getInputFilterSettings()

Gets the vti input filter settings :returns: No error = 0

off_or_on: point: window:

Return type
errorNumber

getMaxPower() → float

Gets the vti heater maximum power :returns: No error = 0

maxPower:

Return type
errorNumber

getPID()

Gets the vti heater PID values :returns: No error = 0

proportional: integral: derivative:

Return type
errorNumber

getRampControlStatus() → bool

Get current ramp control status of VTI heater :returns: OnOff: Ramp is on or off :rtype: errorcode

getRampRate() → float

Get VTI ramp rate :returns: Rate: Ramp rate in Kelvin / minute. 0.1 - 100. 0.0 means ramp limit is off.
:rtype: errorcode

getResistance() → float

Gets the vti temperature resistance :returns: No error = 0

vti_resistance:

Return type
errorNumber

getSetPoint() → float

Gets the vti set point :returns: No error = 0

set_point:

Return type
errorNumber

getTempControlStatus() → bool

Gets the vti heater status :returns: No error = 0

Status:

Return type
errorNumber

getTemperature() → float

Gets the vti temperature :returns: No error = 0

vti_temperature:

Return type

errorNumber

heaterRampOff()

Set VTI heater ramp off

heaterRampOn(rate: float)

Set VTI heater ramp on

Parameters

rate – Ramp rate in Kelvin / minute 0.1 - 100

setD(value: float)

Sets the vti heater D value

Parameters

value –

setHeaterAllZoneRampRates(rampRate: float)

Set vti heater all zones ramp rate

Parameters

rampRate – Ramp rate for this zone 0.1 100 K/min

setHeaterPower(value: float)

Sets the vti heater power

Parameters

value –

setHeaterRes(value: float)

Sets the vti heater resistance

Parameters

value –

setHeaterWireRes(value: float)

Sets the vti heater wire resistance

Parameters

value –

setHeaterZoneSettings(zone: int, upperbound: float, p_input: float, i_input: float, d_input: float, manualOutput: float, heatingRange: int)

Set VTI heater zone settings

Parameters

- **zone** – Zone number
- **upperbound** – Upper set point boundary of this zone in kelvin
- **p_input** –
- **i_input** –
- **d_input** –

- **manualOutput** – Manual output for this zone 0 to 100%
- **heatingRange** – Heating range see class HeaterRanges and Lake Shore 336 manual for ZONE command

setI(*value: float*)

Sets the vti heater I value

Parameters

value –

setInputFilterSettings(*filterOn: bool, numberOfPoints: int, windowSize: int*)

Sets the vti input filter settings

Parameters

- **filterOn** –
- **numberOfPoints** –
- **windowSize** –

setMaxPower(*value: float*)

Sets the vti heater maximum power

Parameters

value –

setP(*value: float*)

Sets the vti heater P value

Parameters

value –

setPID(*proportional: float, integral: float, derivative: float*)

Sets the vti heater PID values

Parameters

- **proportional** –
- **integral** –
- **derivative** –

setRampRate(*rate: float*)

Set VTI ramp rate

Parameters

rate – Ramp rate in Kelvin / minute. 0.1 - 100. 0.0 means ramp limit is off.

setSetPoint(*temperature: float*)

Sets the vti set point

Parameters

temperature –

startHeaterOpenLoopPower()

Starts the vti heater in open loop mode with the previously set power

startHeaterZoneMode()

Starts the vti heater in zone mode

startRampControl()

Set VTI heater ramp control on

startTempControl()

Starts the vti heater control

stopRampControl()

Set VTI heater ramp off (same as heaterRampOff())

stopTempControl()

Stops the vti heater control

uploadCalibrationCurve(*calibrationData: str*)

Sets the vti sensor .340 calibration curve may time out, but the upload will still work

Parameters

calibrationData –

Module contents

atto_device.CRYO800 package

Submodules

atto_device.CRYO800.ACS module

exception atto_device.CRYO800.ACS.**AttoException**(*errorText=None, errorNumber=0*)

Bases: Exception

class atto_device.CRYO800.ACS.**AttoResult**(*resultDict*)

Bases: object

class atto_device.CRYO800.ACS.**Device**(*address*)

Bases: object

TCP_PORT = 9090

close()

Closes the connection to the device.

connect()

Initializes and connects the selected AMC device.

static discover(*cls*)

getResponse(*request_id*)

handleError(*response, ignoreFunctionError=False*)

is_open = False

printError(*errorNumber*)

Converts the errorNumber into an error string and prints it to the console. :param errorNumber: :type errorNumber: int


```

request(method, params=False)
    Synchronous request.

request_id = 3912

request_id_lock = <unlocked _thread.lock object>

response_buffer = {}

sendRequest(method, params=False)

```

atto_device.CRYO800.about module

```

class atto_device.CRYO800.about.About(device)
    Bases: object

    getInstalledPackages() → str
        Get list of packages installed on the device :returns: errNo errorCode
        value_string: string: Comma separated list of packages

        Return type
        value_errNo

    getPackageLicense(pckg: str) → str
        Get the license for a specific package

        Parameters
        pckg – string: Package name

        Returns
        errNo errorCode value_string: string: License for this package

        Return type
        value_errNo

```

atto_device.CRYO800.access module

```

class atto_device.CRYO800.access.Access(device)
    Bases: object

    getLockStatus()
        This function returns if the device is locked and if the current client is authorized to use the device. :returns:
        errNo errorCode

        value_locked: locked Is the device locked? value_authorized: authorized Is the client autho-
        rized?

        Return type
        value_errNo

```

grantAccess(password: str)

Grants access to a locked device for the requesting IP by checking against the password

Parameters

password – string the current password

lock(password: str)

This function locks the device with a password, so the calling of functions is only possible with this password. The locking IP is automatically added to the devices which can access functions

Parameters

password – string the password to be set

unlock(password: str)

This function unlocks the device, so it will not be necessary to execute the grantAccess function to run any function

Parameters

password – string the current password

atto_device.CRYO800.action module

class atto_device.CRYO800.action.**Action**(device)

Bases: object

autotunePID(channel: int, tuningType: int)

Autotune heater PID

Parameters

- **channel** – Channel number
- **tuningType** – 0 tune P, 1 tune PI, 2 tune PID

cancelCurrentCommand()

Cancels the current command

getCurrentCommand() → str

Get current command name :returns: No error = 0

current_command:

Return type

errorNumber

getCurrentCommandStatus() → str

Get current command status :returns: No error = 0

current_command:

Return type

errorNumber

getGoToBaseRampRateSetting() → float

Get sample ramp rate :returns: Rate: Ramp rate in Kelvin / minute. 0.1 - 100. 0.0 means ramp limit is off.
:rtype: errorcode

getSampleExchangePumpOnSetting() → bool

Get sample exchange pump on setting :returns: OnOrOff: :rtype: errorcode

getSampleExchangeRampRateSetting() → float

Get sample ramp rate :returns: Rate: Ramp rate in Kelvin / minute. 0.1 - 100. 0.0 means ramp limit is off.
:rtype: errorcode

getWaitForEvent() → str

Gets the event the cryostat waits for currently :returns: No error = 0

event: event the cryostat waits for

Return type

errorNumber

goToBase()

Cool down to base temperature

goToBaseSoft()

Cool down to base temperature with fixed ramp rate

postEvent(eventName: str)

Posts the event. See: getWaitForEvent()

Parameters

eventName –

sampleExchange()

Start sample exchange

sampleExchangeSoft()

Start sample exchange with fixed ramp rate

setGoToBaseRampRateSetting(rampRate: float)

Set sample ramp rate

Parameters

rampRate – Ramp rate in Kelvin / minute. 0.1 - 100. 0.0 means ramp limit is off.

setSampleExchangePumpOnSetting(onOrOff: bool)

Set sample exchange pump on setting

Parameters

onOrOff –

setSampleExchangeRampRateSetting(rampRate: float)

Set sample ramp rate

Parameters

rampRate – Ramp rate in Kelvin / minute. 0.1 - 100. 0.0 means ramp limit is off.

shutdown()

Start shutdown

switchMagnetsOff()

Switches magnets off

atto_device.CRYO800.attoDry800 module

```
class atto_device.CRYO800.attoDry800.Device(address)
```

Bases: *Device*

```
atto_device.CRYO800.attoDry800.discover()
```

atto_device.CRYO800.breakVacuumValve module

```
class atto_device.CRYO800.breakVacuumValve.Breakvacuumvalve(device)
```

Bases: object

atto_device.CRYO800.compressor module

```
class atto_device.CRYO800.compressor.Compressor(device)
```

Bases: object

atto_device.CRYO800.exchange module

```
class atto_device.CRYO800.exchange.Exchange(device)
```

Bases: object

```
downloadCalibrationCurve() → str
```

Gets the exchange sensor .340 calibration curve :returns: No error = 0

calibration_data:

Return type

errorNumber

```
getHeaterAllZoneRampRates() → float
```

Get exchange heater all zones ramp rate :returns: Error code

rampRate: Ramp rate for this zone 0.1 100 K/min

Return type

errorcode

```
getHeaterDefaultZoneSettings(zone: int)
```

Get exchange heater zone settings

Parameters

zone – Zone number

Returns

Error code upperbound: Upper Setpoint boundary of this zone in kelvin P: P I: I D: D manualOutput: Manual output for this zone 0 to 100% heatingRange: Heating range see class HeaterRanges and Lake Shore 336 manual for ZONE command rampRate: Ramp rate for this zone 0.1 100 K/min

Return type

errorcode

getHeaterHeatingMode() → int

Gets the exchange heater heating mode :returns: No error = 0

mode:

Return type

errorNumber

getHeaterNumberOfDefaultZones() → int

Get the number of the exchange heater default zones :returns: Error code

number_of_zones:

Return type

errorcode

getHeaterNumberOfZones() → int

Get the number of the exchange heater zones :returns: Error code

number_of_zones:

Return type

errorcode

getHeaterPower() → float

Gets the exchange heater power :returns: No error = 0

power:

Return type

errorNumber

getHeaterRes() → float

Gets the exchange heater resistance :returns: No error = 0

resistance:

Return type

errorNumber

getHeaterStatus()

Gets the exchange heater status

getHeaterZoneSettings(zone: int)

Get exchange heater zone settings

Parameters

zone – Zone number

Returns

Error code upperbound: Upper Setpoint boundary of this zone in kelvin P: P I: I D: D manualOutput: Manual output for this zone 0 to 100% heatingRange: Heating range see class HeaterRanges and Lake Shore 336 manual for ZONE command rampRate: Ramp rate for this zone 0.1 100 K/min

Return type

errorcode

getInputFilterSettings()

Gets the exchange input filter settings :returns: No error = 0

off_or_on: point: window:

Return type

errorNumber

getMaxPower() → float

Gets the exchange heater maximum power :returns: No error = 0

maxPower:

Return type

errorNumber

getResistance() → float

Gets the exchange temperature resistance :returns: No error = 0

exchange_resistance:

Return type

errorNumber

getSetPoint() → float

Gets the exchange set point :returns: No error = 0

set_point:

Return type

errorNumber

getTempControlStatus() → bool

Gets the exchange heater status :returns: No error = 0

Status:

Return type

errorNumber

getTemperature() → float

Gets the exchange temperature :returns: No error = 0

exchange_temperature:

Return type

errorNumber

setHeaterAllZoneRampRates(*rampRate: float*)

Set exchange heater all zones ramp rate

Parameters

rampRate – Ramp rate for this zone 0.1 100 K/min

setHeaterPower(*value: float*)

Sets the exchange heater power

Parameters

value –

setHeaterZoneSettings(*zone: int, upperbound: float, P: float, I: float, D: float, manualOutput: float, heatingRange: int*)

Set exchange heater zone settings

Parameters

- **zone** – Zone number
- **upperbound** – Upper Setpoint boundary of this zone in kelvin
- **P** – P
- **I** – I
- **D** – D
- **manualOutput** – Manual output for this zone 0 to 100%
- **heatingRange** – Heating range see class HeaterRanges and Lake Shore 336 manual for ZONE command

setInputFilterSettings(*filterOn: bool, numberOfPoints: int, windowSize: int*)

Sets the exchange input filter settings

Parameters

- **filterOn** –
- **numberOfPoints** –
- **windowSize** –

setSetPoint(*setPoint: float*)

Sets the exchange set point

Parameters

setPoint –

startHeaterOpenLoopPower()

Starts the exchange heater in open loop mode with the previously set power

startHeaterZoneMode()

Starts the exchange heater in zone mode

startTempControl()

Starts the exchange heater

stopTempControl()

Stops the exchange heater

uploadCalibrationCurve(*calibrationData: str*)

Sets the exchange sensor .340 calibration curve May time out, but the upload will still work

Parameters

calibrationData –

atto_device.CRYO800.functions module

class atto_device.CRYO800.functions.**Functions**(*device*)

Bases: object

checkAMCinRack()

If AMC is on Rack position 0, use it as DHCP server, else use it as DHCP client

atto_device.CRYO800.main module

class atto_device.CRYO800.main.**Main**(*device*)

Bases: object

atto_device.CRYO800.membranePump module

class atto_device.CRYO800.membranePump.**Membranepump**(*device*)

Bases: object

atto_device.CRYO800.network module

class atto_device.CRYO800.network.**Network**(*device*)

Bases: object

apply()

Apply temporary IP configuration and load it

configureWifi(*mode: int, ssid: str, psk: str*)

Change the wifi configuration and applies it

Parameters

- **mode** – 0: Access point, 1: Wifi client
- **ssid** –
- **psk** – Pre-shared key

discard()

Discard temporary IP configuration

getDefaultGateway() → str

Get the default gateway of the device :returns: errNo errorCode

value_Default: Default gateway

Return type

value_errNo

getDnsResolver(*priority: int*) → str

Get the DNS resolver

Parameters

priority – of DNS resolver (Usually: 0 = Default, 1 = Backup)

Returns

errNo errorCode value_IP: IP address of DNS resolver

Return type

value_errNo

getEnableDhcpClient() → bool

Get the state of DHCP client :returns: errNo errorCode

value_boolean: boolean: true = DHCP client enable, false = DHCP client disable

Return type

value_errNo

getEnableDhcpServer() → bool

Get the state of DHCP server :returns: errNo errorCode

value_boolean: boolean: true = DHCP server enable, false = DHCP server disable

Return type

value_errNo

getIpAddress() → str

Get the IP address of the device :returns: errNo errorCode

value_IP: IP address as string

Return type

value_errNo

getProxyServer() → str

Get the proxy settings of the device :returns: errNo errorCode

value_Proxy: Proxy Server String, empty for no proxy

Return type

value_errNo

getRealIpAddress() → str

Get the real IP address of the device set to the network interface (br0, eth1 or eth0) :returns: errNo errorCode

value_IP: IP address as string

Return type

value_errNo

getSubnetMask() → str

Get the subnet mask of the device :returns: errNo errorCode

value_Subnet: Subnet mask as string

Return type

value_errNo

getWifiMode() → int

Get the operation mode of the wifi adapter :returns: errNo errorCode

value_mode: mode 0: Access point, 1: Wifi client

Return type

value_errNo

getWifiPassphrase() → str

Get the the passphrase of the network hosted (mode: Access point) or connected to (mode: client) :returns: errNo errorCode

value_psk: psk Pre-shared key

Return type

value_errNo

getWifiPresent() → bool

Returns is a Wifi interface is present :returns: errNo errorCode

value_True: True, if interface is present

Return type

value_errNo

getWifiSSID() → str

Get the the SSID of the network hosted (mode: Access point) or connected to (mode: client) :returns: errNo errorCode

SSID: SSID

Return type

value_errNo

setDefaultGateway(*gateway: str*)

Set the default gateway of the device

Parameters

gateway – Default gateway as string

setDnsResolver(*priority: int, resolver: str*)

Set the DNS resolver

Parameters

- **priority** – of DNS resolver (Usually: 0 = Default, 1 = Backup)
- **resolver** – The resolver's IP address as string

setEnabledDhcpClient(*enable: bool*)

Enable or disable DHCP client

Parameters

enable – boolean: true = enable DHCP client, false = disable DHCP client

setEnabledDhcpServer(*enable: bool*)

Enable or disable DHCP server

Parameters

enable – boolean: true = enable DHCP server, false = disable DHCP server

setIpAddress(*address: str*)

Set the IP address of the device

Parameters

address – IP address as string

setProxyServer(*proxyServer: str*)

Set the proxy server of the device

Parameters

proxyServer – Proxy Server Setting as string

setSubnetMask(*netmask: str*)

Set the subnet mask of the device

Parameters

netmask – Subnet mask as string

setWifiMode(*mode: int*)

Change the operation mode of the wifi adapter

Parameters

mode – 0: Access point, 1: Wifi client

setWifiPassphrase(*psk: str*)

Change the passphrase of the network hosted (mode: Access point) or connected to (mode: client)

Parameters

psk – Pre-shared key

setWifiSSID(*ssid: str*)

Change the SSID of the network hosted (mode: Access point) or connected to (mode: client)

Parameters

ssid –

verify()

Verify that temporary IP configuration is correct

atto_device.CRYO800.pressures module

class atto_device.CRYO800.pressures.**Pressures**(*device*)

Bases: object

getSampleSpacePressure() → float

Gets the sample space pressure :returns: No error = 0

sample_space_pressure:

Return type

errorNumber

atto_device.CRYO800.pumpValve module

```
class atto_device.CRYO800.pumpValve.Pumpvalve(device)
    Bases: object
```

atto_device.CRYO800.sample module

```
class atto_device.CRYO800.sample.Sample(device)
    Bases: object
```

```
downloadCalibrationCurve() → str
```

Gets the sample sensor .340 calibration curve :returns: No error = 0

calibration_data:

Return type

errorNumber

```
getHeaterAllZoneRampRates() → float
```

Get sample heater all zones ramp rate :returns: Error code

rampRate: Ramp rate for this zone 0.1 100 K/min

Return type

errorCode

```
getHeaterDefaultZoneSettings(zone: int)
```

Get sample heater zone settings

Parameters

zone – Zone number

Returns

Error code upperbound: Upper set point boundary of this zone in kelvin P: P I: I D: D manualOutput: Manual output for this zone 0 to 100% heatingRange: Heating range see class HeaterRanges and Lake Shore 336 manual for ZONE command rampRate: Ramp rate for this zone 0.1 100 K/min

Return type

errorCode

```
getHeaterHeatingMode() → int
```

Gets the sample heater heating mode :returns: No error = 0

mode:

Return type

errorNumber

```
getHeaterNumberOfDefaultZones() → int
```

Get the number of sample heater default zones :returns: Error code

number_of_zones:

Return type

errorCode

getHeaterNumberOfZones() → int

Get the number of the sample heater zones :returns: Error code

number_of_zones:

Return type

errorCode

getHeaterPower() → float

Gets the sample heater power :returns: No error = 0

power:

Return type

errorNumber

getHeaterRampData()

Get sample heater ramp data :returns: OnOff: Ramp is on or off

Rate: Ramp rate in Kelvin / minute. 0.1 - 100. 0.0 means ramp limit is off.

Return type

errorCode

getHeaterRes() → float

Gets the sample heater resistance :returns: No error = 0

resistance:

Return type

errorNumber

getHeaterStatus()

Gets the sample heater status

getHeaterWireRes() → float

Gets the sample heater wire resistance :returns: No error = 0

wire_resistance:

Return type

errorNumber

getHeaterZoneSettings(zone: int)

Get sample heater zone settings

Parameters

zone – Zone number

Returns

Error code upperbound: Upper set point boundary of this zone in kelvin P: P I: I D: D manualOutput: Manual output for this zone 0 to 100% heatingRange: Heating range see class HeaterRanges and Lake Shore 336 manual for ZONE command rampRate: Ramp rate for this zone 0.1 100 K/min

Return type

errorCode

getInputFilterSettings()

Gets the sample input filter settings :returns: No error = 0

off_or_on: point: window:

Return type

errorNumber

getMaxPower() → float

Gets the sample heater maximum power :returns: No error = 0

maxPower:

Return type

errorNumber

getPID()

Gets the sample heater PID values :returns: No error = 0

proportional: integral: derivative:

Return type

errorNumber

getRampControlStatus() → bool

Get the ramp control status :returns: OnOff: Ramp is on or off :rtype: errorcode

getRampRate() → float

Get sample ramp rate :returns: Rate: Ramp rate in Kelvin / minute. 0.1 - 100. 0.0 means ramp limit is off.
:rtype: errorcode

getResistance() → float

Gets the sample resistance :returns: No error = 0

sample_resistance:

Return type

errorNumber

getSetPoint() → float

Gets the sample set point :returns: No error = 0

set_point:

Return type

errorNumber

getTempControlStatus() → bool

Gets the sample heater status :returns: No error = 0

status:

Return type

errorNumber

getTemperature() → float

Gets the sample temperature :returns: No error = 0

sample_temperature:

Return type

errorNumber

heaterRampOff()

Set sample heater ramp off

heaterRampOn(rate: float)

Set sample heater ramp on

Parameters

rate – Ramp rate in Kelvin / minute 0.1 - 100

setD(value: float)

Sets the sample heater D value

Parameters

value –

setHeaterAllZoneRampRates(rampRate: float)

Set sample heater all zones ramp rate

Parameters

rampRate – Ramp rate for this zone 0.1 100 K/min

setHeaterPower(value: float)

Sets the sample heater power

Parameters

value –

setHeaterRes(value: float)

Sets the sample heater resistance

Parameters

value –

setHeaterWireRes(value: float)

Sets the sample heater wire resistance

Parameters

value –

setHeaterZoneSettings(zone: int, upperbound: float, p_input: float, i_input: float, d_input: float, manualOutput: float, heatingRange: int)

Set sample heater zone settings

Parameters

- **zone** – Zone number
- **upperbound** – Upper set point boundary of this zone in kelvin
- **p_input** – P
- **i_input** – I
- **d_input** – D

- **manualOutput** – Manual output for this zone 0 to 100%
- **heatingRange** – Heating range see class HeaterRanges and Lake Shore 336 manual for ZONE command

setI(*value: float*)

Sets the sample heater I value

Parameters

value –

setInputFilterSettings(*filterOn: bool, numberOfPoints: int, windowSize: int*)

Sets the sample input filter settings

Parameters

- **filterOn** –
- **numberOfPoints** –
- **windowSize** –

setMaxPower(*value: float*)

Sets the sample heater maximum power

Parameters

value –

setP(*value: float*)

Sets the sample heater P value

Parameters

value –

setPID(*proportional: float, integral: float, derivative: float*)

Sets the sample heater PID values

Parameters

- **proportional** –
- **integral** –
- **derivative** –

setRampRate(*rate: float*)

Set sample ramp rate

Parameters

rate – Ramp rate in Kelvin / minute. 0.1 - 100. 0.0 means ramp limit is off.

setSetPoint(*setPoint: float*)

Sets the sample set point

Parameters

setPoint –

startHeaterOpenLoopPower()

Starts the sample heater in open loop mode with the previously set power

startHeaterZoneMode()

Starts the sample heater in zone mode

startRampControl()

Start controlling the ramping

startTempControl()

Starts the sample heater

stopRampControl()

Stop controlling the ramping

stopTempControl()

Stops the sample heater

uploadCalibrationCurve(*calibrationData: str*)

Sets the sample sensor .340 calibration curve may time out, but the upload will still work

Parameters

calibrationData –

atto_device.CRYO800.sampleValve module

class atto_device.CRYO800.sampleValve.**Samplevalve**(*device*)

Bases: object

atto_device.CRYO800.system module

class atto_device.CRYO800.system.**System**(*device*)

Bases: object

emptyExternalDatabaseToMaximumBytes()

Empty database! This solved issues when the UI is hanging! Don't use it otherwise

errorNumberToString(*errorNumber: int*)

“Translate” the error code into an error text Currently we only support the system language

Parameters

errorNumber – error code to translate

getAutoprobeInformationJSONd() → str

Get the hardware autoprobe information :returns: No error = 0

infoJson: JSONd autoprobe information

Return type

errorNumber

getAvailableConfigurationSettings() → str

Gets all available configuration settings :returns: No error = 0

Configuration_settings: CSV of configuration settings

Return type

errorNumber

getConfigOverrideFile() → str

Get the EEPROM JSON file containing the config overrides. To know what entries exist have a look at `getAvailableConfigurationSettings()` and `getConfigurationSettingInformation()` :returns: No error = 0

overrideFile: String containing the override file

Return type

errorNumber

getConfigurationConfiguration(settingName: str) → float

Gets the configuration

Parameters

settingName –

Returns

No error = 0 Configuration_value:

Return type

errorNumber

getConfigurationSetting(settingName: str) → float

Get the value of the specified configuration setting

Parameters

settingName –

Returns

No error = 0 Configuration_setting_value:

Return type

errorNumber

getConfigurationSettingInformation(settingName: str) → str

Gets the configuration setting info string

Parameters

settingName –

Returns

No error = 0 Configuration_settings_information:

Return type

errorNumber

getCurrentStatusJSONd() → str

Get the current cryostat status :returns: No error = 0

status: Cryostat status

Return type

errorNumber

getDatabaseAsyncQueryResult(handle: int) → str

Get the query result

Parameters

handle –

Returns

No error = 0 result:

Return type

errorNumber

getDatabaseColumnsMinMaxIntervalRows(*minRow: int, maxRow: int, columnNames: str*) → str

Get the cryostat database columns

Parameters

- **minRow** –
- **maxRow** –
- **columnNames** – Empty string means all available columns

Returns

No error = 0 result_rows:

Return type

errorNumber

getDatabaseMaximumLogSeconds() → float

Get the cryostat long term status of a time interval :returns: No error = 0

maximumSecondsInLog:

Return type

errorNumber

getDatabaseMinMaxResult(*handle: int*)

Get the min and max rows.

Parameters**handle** –**Returns**

No error = 0 min_row: max_row:

Return type

errorNumber

getDatabaseMinimumLogSeconds() → float

Get the cryostat long term status of a time interval :returns: No error = 0

minimumSecondsInLog:

Return type

errorNumber

getDeviceType() → str

Gets the device type. :returns: No error = 0

type: type of the device

Return type

errorNumber

getFeatures()

Get the features of the cryostat. :returns: No error = 0

Feature_names: Feature_descriptrions: Feature_short_descriptrions: Feature_display_names:
 Feature_picture:

Return type
errorNumber

getFeaturesActivated() → str

Get the active features of the cryostat. :returns: No error = 0

Feature_names:

Return type
localErrorNumber

getFeaturesJSON() → str

Get the features of the cryostat as a JSON string. :returns: No error = 0

JSON:

Return type
localErrorNumber

getIntervalLongTermEntriesJSONd(*secondsOld: float, secondsNew: float*) → str

Get the cryostat long term status of a time interval

Parameters

- **secondsOld** –
- **secondsNew** –

Returns

No error = 0 intervalJson: JSONd description of cryo status in time interval

Return type
errorNumber

getIntervalShortTermEntriesJSONd(*secondsOld: float, secondsNew: float*) → str

Get the cryostat status of a time interval

Parameters

- **secondsOld** –
- **secondsNew** –

Returns

No error = 0 Short_term_log_entries:

Return type
errorNumber

getLastError()

Gets all available information on the last error :returns: Local errorcode of this call

lastErrorNumber: Last error number recommendation: Recommendation component: Component that raised the error command: Command age: Age

Return type
errorNumber

getLastErrorAge() → int

Gets the age of the last error that happened :returns: No error = 0

age: time in seconds since the last error happened

Return type

errorNumber

getLastErrorCommand() → str

Gets the command of the last error that happened :returns: No error = 0

command: command running when the last error happened

Return type

errorNumber

getLastErrorComponent() → str

Gets the component of the last error that happened :returns: No error = 0

component: cryostat component of the error

Return type

errorNumber

getLastErrorNumber() → int

Gets the error number of the last error that happened :returns: No error = 0

errorNumber: No error = 0

Return type

localErrorNumber

getLastErrorRecommendation() → str

Gets the recommendation of the last error that happened :returns: No error = 0

recommendation: error recommendation

Return type

errorNumber

getLastErrorsJSONd() → str

Gets the last cryostat errors as a JSON string. :returns: No error = 0

JSON:

Return type

localErrorNumber

getLastErrorsPickled(*pythonVersion: int*) → str

Gets the last cryostat errors from the snapshot. Use `getLastErrorsAsJSON()` and `unJSON` the data if this call doesn't work for you (the cryostat data can't be unpickled)

Parameters

pythonVersion –

Returns

No error = 0 Pickled_data:

Return type

localErrorNumber

getLastLongTermEntriesJSONd(*seconds: float*) → str

Get the cryostat long term status of the last seconds

Parameters

seconds –

Returns

No error = 0 version: firmware version number

Return type

errorNumber

getLastShortTermEntriesJSONd(*seconds: float*) → str

Get the cryostat status of the last seconds

Parameters

seconds –

Returns

No error = 0 version: firmware version number

Return type

errorNumber

getLogLines(*maxBytes: int*) → str

Get the last lines of the system logfile.

Parameters

maxBytes – maximum bytes to read

Returns

No error = 0 logLines: String containing the logfile lines

Return type

errorNumber

getSecondsFromStartCryostatInstallation() → float

Gets the seconds from start of cryostat installation :returns: No error = 0

seconds: Seconds from start of cryostat installation

Return type

errorNumber

getUserConfigOverrideFile() → str

Get the user JSON file containing the config overrides. To know what entries exist have a look at `getAvailableConfigurationSettings()` and `getConfiguratioSettingInformation()` :returns: No error = 0

overrideFile: String containing the override file

Return type

errorNumber

lowerError()

Resets the last error

prepareDatabaseColumnsIntervalAsync(*minRow: int, maxRow: int, columnNames: str, handle: int*)

Prepare the cryostat long term status of a time interval query

Parameters

- **minRow** –
- **maxRow** –
- **columnNames** – Empty string means all available columns
- **handle** –

prepareDatabaseMinMaxInterval(*minTime: float, maxTime: float*) → int

Prepare the cryostat long term status of a time interval query Get the row numbers with `getDatabaseMinMaxResult()` Get the data with `getDatabaseColumnsMinMaxIntervalRows()` `minTime` and `maxTime` can be taken from `getDatabaseMinimumLogSeconds()` and `getDatabaseMaximumLogSeconds()`

Parameters

- **minTime** –
- **maxTime** –

Returns

No error = 0 handle:

Return type

errorNumber

resetUserConfigOverrideFile()

Copy the EEPROM config factory defaults to the user config

setConfigurationSetting(*settingName: str, settingValue: float*)

Sets the configuration setting

Parameters

- **settingName** –
- **settingValue** –

setUserConfigOverrideFile(*fileContent: str*) → str

Set the user JSON file containing the config overrides. To know what entries exist have a look at `getAvailableConfigurationSettings()` and `getConfigurationSettingInformation()`

Parameters

fileContent – Contents of the config override file

Returns

No error = 0 hint_string: Free text string to give the caller some hints if things don't work

Return type

errorNumber

atto_device.CRYO800.system_service module**class** atto_device.CRYO800.system_service.**System_service**(*device*)

Bases: object

apply(*key: int*)

Apply temporary system configuration

Parameters**key** –**errorNumberToRecommendation**(*language: int, errNbr: int*) → str

Get a recommendation for the error code

Parameters

- **language** – integer: Language code
- **errNbr** – interger: Error code to translate

Returns

errNo errorCode value_string: string: Error recommendation (currently returning an int = 0 until we have recommendations)

Return type

value_errNo

errorNumberToString(*language: int, errNbr: int*) → str

Get a description of an error code

Parameters

- **language** – integer: Language code 0 for the error name, 1 for a more user friendly error message
- **errNbr** – interger: Error code to translate

Returns

errNo errorCode value_string: string: Error description

Return type

value_errNo

factoryReset()

Turns on the factory reset flag.

getDeviceName() → str

Get the actual device name :returns: errNo errorCode

value_string: string: actual device name

Return type

value_errNo

getFirmwareVersion() → str

Get the firmware version of the system :returns: errNo errorCode

value_string: string: The firmware version

Return type

value_errNo

getFluxCode() → str

Get the flux code of the system :returns: errNo errorCode

value_string: string: flux code

Return type

value_errNo

getHostname() → str

Return device hostname :returns: errNo errorCode

available: available

Return type

value_errNo

getMacAddress() → str

Get the mac address of the system :returns: errNo errorCode

value_string: string: Mac address of the system

Return type

value_errNo

getSerialNumber() → str

Get the serial number of the system :returns: errNo errorCode

value_string: string: Serial number

Return type

value_errNo

rebootSystem()

Reboot the system

setDeviceName(*name: str*)

Set custom name for the device

Parameters

name – string: device name

setTime(*day: int, month: int, year: int, hour: int, minute: int, second: int*)

Set system time manually

Parameters

- **day** – integer: Day (1-31)
- **month** – integer: Day (1-12)
- **year** – integer: Day (eg. 2021)
- **hour** – integer: Day (0-23)
- **minute** – integer: Day (0-59)
- **second** – integer: Day (0-59)

softReset()

Performs a soft reset (Reset without deleting the network settings).

updateTimeFromInternet()

Update system time by querying attocube.com

atto_device.CRYO800.tboard module

class atto_device.CRYO800.tboard.**Tboard**(*device*)

Bases: object

autotunePIDError() → int

Gets last autotune error :returns: No error = 0

Autotune_errorcode:

Return type

errorNumber

autotunePIDRunning() → bool

Check whether PID autotune is currently running :returns: No error = 0

running:

Return type

errorNumber

autotunePIDStage() → int

Gets last autotune stage :returns: No error = 0

Autotune_errorcode:

Return type

errorNumber

downloadCalibrationCurve(*channel: int*) → str

Gets the sensor .340 calibration curve

Parameters

channel –

Returns

No error = 0 calibration_data:

Return type

errorNumber

getHeaterPIDChannel(*channelNumber: int*)

Get PID values from channel

Parameters

channelNumber –

Returns

No error = 0 p_output: i_output: d_output:

Return type
errorNumber

getHeaterPowerChannel(*channelNumber: int*) → float

Get heater output power

Parameters
channelNumber –

Returns
No error = 0 power:

Return type
errorNumber

getHeaterSetpointChannel(*channelNumber: int*) → float

Get heater setpoint

Parameters
channelNumber –

Returns
No error = 0 set_point:

Return type
errorNumber

getHeaterZoneSettingsChannel(*channelNumber: int*)

Get heater zone settings channel

Parameters
channelNumber –

Returns
No error = 0 upperbound: Upper temperature bound of zone p_output: i_output: d_output:
manualOutput: heatingRange: 0 -> off, 1 -> attocube OEM on, 1 -> 335, 336 Low, , 2 ->
335, 336 Medium, , 3 -> 335, 336 High rampRate:

Return type
errorNumber

getRawSensorInput(*channelNumber: int*) → float

Get raw sensor input.

Parameters
channelNumber –

Returns
No error = 0 Raw_sensor_input:

Return type
errorNumber

getTemperature(*channelNumber: int*) → float

Get temperature.

Parameters
channelNumber –

Returns
No error = 0 Temperature:

Return type

errorNumber

isHeatingChannel(*channelNumber: int*) → bool

Check if channel is heating

Parameters**channelNumber** –**Returns**

No error = 0 isHeating:

Return type

errorNumber

setHeaterPIDChannel(*channelNumber: int, p_input: float, i_input: float, d_input: float*)

Set PID values for channel

Parameters

- **channelNumber** –
- **p_input** –
- **i_input** –
- **d_input** –

setHeaterSetpointChannel(*channelNumber: int, setpoint: float*)

Start heating in zone mode

Parameters

- **channelNumber** –
- **setpoint** – Temperature set point

setHeaterZoneSettingsChannel(*channelNumber: int, zone: int, upperbound: float, p_input: float, i_input: float, d_input: float, manualOutput: float, heatingRange: int, rampRate: float*)

Set zone parameters for heater channel

Parameters

- **channelNumber** –
- **zone** –
- **upperbound** – Upper temperature bound of zone
- **p_input** –
- **i_input** –
- **d_input** –
- **manualOutput** –
- **heatingRange** – 0 -> off, 1 -> attocube OEM on, 1 -> 335, 336 Low, , 2 -> 335, 336 Medium, , 3 -> 335, 336 High
- **rampRate** –

startHeatingChannel(*channelNumber: int*)

Start heating

Parameters

channelNumber –

startHeatingOpenLoopChannel(*channelNumber: int, power: float*)

Start heating in open loop mode

Parameters

- **channelNumber** –
- **power** –

startHeatingZoneModeChannel(*channelNumber: int*)

Start heating in zone mode

Parameters

channelNumber –

stopHeatingChannel(*channelNumber: int*)

Stop heating

Parameters

channelNumber –

uploadCalibrationCurve(*channel: int, curveData: str*)

Sets the sensor .340 calibration curve may time out, but the upload will still work

Parameters

- **channel** –
- **curveData** – calibration data

atto_device.CRYO800.turboPump module

class atto_device.CRYO800.turboPump.**Turbopump**(*device*)

Bases: object

atto_device.CRYO800.update module

class atto_device.CRYO800.update.**Update**(*device*)

Bases: object

getLicenseUpdateProgress() → int

Get the progress of running license update :returns: errNo errorCode

value_int: int: progress in percent

Return type

value_errNo

getSwUpdateProgress() → int

Get the progress of running update :returns: errNo errorCode

value_int: int: progress in percent

Return type

value_errNo

licenseUpdateBase64()

Execute the license update with base64 file uploaded.

softwareUpdateBase64()

Execute the update with base64 file uploaded.

uploadLicenseBase64(*offset: int, b64Data: str*)

Upload new license file in format base 64

Parameters

- **offset** – int: offset of the data
- **b64Data** – string: base64 data

uploadSoftwareImageBase64(*offset: int, b64Data: str*)

Upload new firmware image in format base 64

Parameters

- **offset** – int: offset of the data
- **b64Data** – string: base64 data

Module contents

atto_device.CRYOEVU package

Submodules

atto_device.CRYOEVU.ACS module

exception atto_device.CRYOEVU.ACS.**AttoException**(*errorText=None, errorNumber=0*)

Bases: Exception

class atto_device.CRYOEVU.ACS.**AttoResult**(*resultDict*)

Bases: object

class atto_device.CRYOEVU.ACS.**Device**(*address*)

Bases: object

TCP_PORT = 9090

close()

Closes the connection to the device.

connect()

Initializes and connects the selected AMC device.

```

static discover(cls)

getResponse(request_id)

handleError(response, ignoreFunctionError=False)

is_open = False

printError(errorNumber)
    Converts the errorNumber into an error string an prints it to the console. :param errorNumber: :type
    errorNumber: int

request(method, params=False)
    Synchronous request.

request_id = 89337

request_id_lock = <unlocked _thread.lock object>

response_buffer = {}

sendRequest(method, params=False)

```

atto_device.CRYOEUVU.about module

```

class atto_device.CRYOEUVU.about.About(device)
    Bases: object

getInstalledPackages() → str
    Get list of packages installed on the device :returns: errNo errorCode
    value_string: string: Comma separated list of packages

    Return type
        value_errNo

getPackageLicense(pckg: str) → str
    Get the license for a specific package

    Parameters
        pckg – string: Package name

    Returns
        errNo errorCode value_string: string: License for this package

    Return type
        value_errNo

```

atto_device.CRYOEVU.access module

```
class atto_device.CRYOEVU.access.Access(device)
```

Bases: object

```
getLockStatus()
```

This function returns if the device is locked and if the current client is authorized to use the device. :returns: errNo errorCode

value_locked: locked Is the device locked? value_authorized: authorized Is the client authorized?

Return type

value_errNo

```
grantAccess(password: str)
```

Grants access to a locked device for the requesting IP by checking against the password

Parameters

password – string the current password

```
lock(password: str)
```

This function locks the device with a password, so the calling of functions is only possible with this password. The locking IP is automatically added to the devices which can access functions

Parameters

password – string the password to be set

```
unlock(password: str)
```

This function unlocks the device, so it will not be necessary to execute the grantAccess function to run any function

Parameters

password – string the current password

atto_device.CRYOEVU.attoDryEVU module

```
class atto_device.CRYOEVU.attoDryEVU.Device(address)
```

Bases: *Device*

```
atto_device.CRYOEVU.attoDryEVU.discover()
```

atto_device.CRYOEVU.compressor module

```
class atto_device.CRYOEVU.compressor.Compressor(device)
```

Bases: object

```
getStatus() → bool
```

Gets the compressor status :returns: No error = 0

compressor_status:

Return type

errorNumber

start()

starts the compressor

stop()

stops the compressor

atto_device.CRYOEVU.functions module

class atto_device.CRYOEVU.functions.**Functions**(*device*)

Bases: object

checkAMCinRack()

If AMC is on Rack position 0, use it as DHCP server, else use it as DHCP client

atto_device.CRYOEVU.heater module

class atto_device.CRYOEVU.heater.**Heater**(*device*)

Bases: object

getHeaterPower(*channelNumber: int*) → float

Gets the heater power

Parameters

channelNumber –

Returns

No error = 0 heater_power: :param channelNumber:

Return type

errorNumber

initializeHeater(*channelNumber: int*)

Initialized the heater

Parameters

channelNumber –

startHeaterOpenLoopPower(*channelNumber: int, power: float*)

Starts the heater in open loop mode with fixed power

Parameters

• **channelNumber** –

• **power** –

stopHeater(*channelNumber: int*)

Stops the heater

Parameters

channelNumber –

atto_device.CRYOEUVU.main module

```
class atto_device.CRYOEUVU.main.Main(device)
    Bases: object
    changeCryostatType(newType: str)
        Change the cryostat type
        Parameters
            newType –
    getAvailableCryostatTypes() → str
        Get all available cryostat types :returns: No error = 0
        types: CSV string of types
        Return type
            errorNumber
```

atto_device.CRYOEUVU.membranePump module

```
class atto_device.CRYOEUVU.membranePump.Membranepump(device)
    Bases: object
    getStatus() → bool
        Gets the membrane pump status :returns: No error = 0
        membrane_pump_status:
        Return type
            errorNumber
    getStatusString() → str
        Gets the membrane pump status as string :returns: No error = 0
        membrane_pump_status_string:
        Return type
            errorNumber
    start()
        starts the membrane pump
    stop()
        stops the membrane pump
```

atto_device.CRYOEUVU.network module

class atto_device.CRYOEUVU.network.**Network**(*device*)

Bases: object

apply()

Apply temporary IP configuration and load it

configureWifi(*mode: int, ssid: str, psk: str*)

Change the wifi configuration and applies it

Parameters

- **mode** – 0: Access point, 1: Wifi client
- **ssid** –
- **psk** – Pre-shared key

discard()

Discard temporary IP configuration

getDefaultGateway() → str

Get the default gateway of the device :returns: errNo errorCode

value_Default: Default gateway

Return type

value_errNo

getDnsResolver(*priority: int*) → str

Get the DNS resolver

Parameters

priority – of DNS resolver (Usually: 0 = Default, 1 = Backup)

Returns

errNo errorCode value_IP: IP address of DNS resolver

Return type

value_errNo

getEnableDhcpClient() → bool

Get the state of DHCP client :returns: errNo errorCode

value_boolean: boolean: true = DHCP client enable, false = DHCP client disable

Return type

value_errNo

getEnableDhcpServer() → bool

Get the state of DHCP server :returns: errNo errorCode

value_boolean: boolean: true = DHCP server enable, false = DHCP server disable

Return type

value_errNo

getIpAddress() → str

Get the IP address of the device :returns: errNo errorCode

value_IP: IP address as string

Return type

value_errNo

getProxyServer() → str

Get the proxy settings of the device :returns: errNo errorCode

value_Proxy: Proxy Server String, empty for no proxy

Return type

value_errNo

getRealIpAddress() → str

Get the real IP address of the device set to the network interface (br0, eth1 or eth0) :returns: errNo errorCode

value_IP: IP address as string

Return type

value_errNo

getSubnetMask() → str

Get the subnet mask of the device :returns: errNo errorCode

value_Subnet: Subnet mask as string

Return type

value_errNo

getWifiMode() → int

Get the operation mode of the wifi adapter :returns: errNo errorCode

value_mode: mode 0: Access point, 1: Wifi client

Return type

value_errNo

getWifiPassphrase() → str

Get the the passphrase of the network hosted (mode: Access point) or connected to (mode: client) :returns: errNo errorCode

value_psk: psk Pre-shared key

Return type

value_errNo

getWifiPresent() → bool

Returns is a Wifi interface is present :returns: errNo errorCode

value_True: True, if interface is present

Return type

value_errNo

getWifiSSID() → str

Get the the SSID of the network hosted (mode: Access point) or connected to (mode: client) :returns:
errNo errorCode

SSID: SSID

Return type

value_errNo

setDefaultGateway(gateway: str)

Set the default gateway of the device

Parameters

gateway – Default gateway as string

setDnsResolver(priority: int, resolver: str)

Set the DNS resolver

Parameters

- **priority** – of DNS resolver (Usually: 0 = Default, 1 = Backup)
- **resolver** – The resolver's IP address as string

setEnabledDhcpClient(enable: bool)

Enable or disable DHCP client

Parameters

enable – boolean: true = enable DHCP client, false = disable DHCP client

setEnabledDhcpServer(enable: bool)

Enable or disable DHCP server

Parameters

enable – boolean: true = enable DHCP server, false = disable DHCP server

setIpAddress(address: str)

Set the IP address of the device

Parameters

address – IP address as string

setProxyServer(proxyServer: str)

Set the proxy server of the device

Parameters

proxyServer – Proxy Server Setting as string

setSubnetMask(netmask: str)

Set the subnet mask of the device

Parameters

netmask – Subnet mask as string

setWifiMode(mode: int)

Change the operation mode of the wifi adapter

Parameters

mode – 0: Access point, 1: Wifi client

setWifiPassphrase(*psk: str*)

Change the passphrase of the network hosted (mode: Access point) or connected to (mode: client)

Parameters

psk – Pre-shared key

setWifiSSID(*ssid: str*)

Change the SSID of the network hosted (mode: Access point) or connected to (mode: client)

Parameters

ssid –

verify()

Verify that temporary IP configuration is correct

atto_device.CRYOEUV.pressures module

class atto_device.CRYOEUV.pressures.**Pressures**(*device*)

Bases: object

getGaugeType(*gaugeNumber: int*) → str

Gets the gauge type

Parameters

gaugeNumber – 0..5

Returns

No error = 0 type:

Return type

errorNumber

getPressureAndVoltage(*gaugeNumber: int*)

Gets the cryostat in pressure

Parameters

gaugeNumber – 0..5

Returns

No error = 0 pressure: voltage:

Return type

errorNumber

getPressureVoltage(*gaugeNumber: int*) → float

Gets the voltage on that would be a pressure gauge

Parameters

gaugeNumber – 0..5

Returns

No error = 0 pressure_voltage:

Return type

errorNumber

setGaugeType(*gaugeNumber: int, newType: str*)

Sets the gauge type

Parameters

- **gaugeNumber** – 0..5
- **newType** –

atto_device.CRYOEVU.scrollPump module

class atto_device.CRYOEVU.scrollPump.Scrollpump(*device*)

Bases: object

getFrequency() → float

Gets the scroll pump frequency :returns: No error = 0

scroll_pump_frequency:

Return type

errorNumber

getStatus() → bool

Gets the scroll pump status :returns: No error = 0

scroll_pump_status:

Return type

errorNumber

getStatusString() → str

Gets the scroll pump status as string :returns: No error = 0

scroll_pump_status_string:

Return type

errorNumber

startFullSpeed()

Starts the scroll pump at full speed

startStandbySpeed()

Starts the scroll pump at standby speed

stop()

Stops the scroll pump

atto_device.CRYOEUVU.system module

```
class atto_device.CRYOEUVU.system.System(device)
```

Bases: object

```
emptyExternalDatabaseToMaximumBytes()
```

Empty database! This solved issues when the UI is hanging! Don't use it otherwise

```
errorNumberToString(errorNumber: int)
```

“Translate” the error code into an error text Currently we only support the system language

Parameters

errorNumber – error code to translate

```
getAutoprobeInformationJSONd() → str
```

Get the hardware autoprobe information :returns: No error = 0

infoJson: JSONd autoprobe information

Return type

errorNumber

```
getAvailableConfigurationSettings() → str
```

Gets all available configuration settings :returns: No error = 0

Configuration_settings: CSV of configuration settings

Return type

errorNumber

```
getConfigOverrideFile() → str
```

Get the EEPROM JSON file containing the config overrides. To know what entries exist have a look at `getAvailableConfigurationSettings()` and `getConfigSettingInformation()` :returns: No error = 0

overrideFile: String containing the override file

Return type

errorNumber

```
getConfigConfiguration(settingName: str) → float
```

Gets the configuration

Parameters

settingName –

Returns

No error = 0 Configuration_value:

Return type

errorNumber

```
getConfigSetting(settingName: str) → float
```

Get the value of the specified configuration setting

Parameters

settingName –

Returns

No error = 0 Configuration_setting_value:

Return type

errorNumber

getConfiguratioSettingInformation(*settingName: str*) → str

Gets the configuration setting info string

Parameters

settingName –

Returns

No error = 0 Configuration_settings_information:

Return type

errorNumber

getCurrentStatusJSONd() → str

Get the current cryostat status :returns: No error = 0

status: Cryostat status

Return type

errorNumber

getDatabaseAsyncQueryResult(*handle: int*) → str

Get the query result

Parameters

handle –

Returns

No error = 0 result:

Return type

errorNumber

getDatabaseColumnsMinMaxIntervalRows(*minRow: int, maxRow: int, columnNames: str*) → str

Get the cryostat database columns

Parameters

- **minRow** –
- **maxRow** –
- **columnNames** – Empty string means all available columns

Returns

No error = 0 result_rows:

Return type

errorNumber

getDatabaseMaximumLogSeconds() → float

Get the cryostat long term status of a time interval :returns: No error = 0

maximumSecondsInLog:

Return type

errorNumber

getDatabaseMinMaxResult(*handle: int*)

Get the min and max rows.

Parameters

handle –

Returns

No error = 0 min_row: max_row:

Return type

errorNumber

getDatabaseMinimumLogSeconds() → float

Get the cryostat long term status of a time interval :returns: No error = 0

minimumSecondsInLog:

Return type

errorNumber

getDeviceType() → str

Gets the device type. :returns: No error = 0

type: type of the device

Return type

errorNumber

getFeatures()

Get the features of the cryostat. :returns: No error = 0

Feature_names: Feature_descriptions: Feature_short_descriptions: Feature_display_names:
Feature_picture:

Return type

errorNumber

getFeaturesActivated() → str

Get the active features of the cryostat. :returns: No error = 0

Feature_names:

Return type

localErrorNumber

getFeaturesJSON() → str

Get the features of the cryostat as a JSON string. :returns: No error = 0

JSON:

Return type

localErrorNumber

getIntervalLongTermEntriesJSONd(*secondsOld: float, secondsNew: float*) → str

Get the cryostat long term status of a time interval

Parameters

- **secondsOld** –
- **secondsNew** –

Returns

No error = 0 intervalJson: JSONd description of cryo status in time interval

Return type

errorNumber

getIntervalShortTermEntriesJSONd(*secondsOld: float, secondsNew: float*) → str

Get the cryostat status of a time interval

Parameters

- **secondsOld** –
- **secondsNew** –

Returns

No error = 0 Short_term_log_entries:

Return type

errorNumber

getLastError()

Gets all available information on the last error :returns: Local errorcode of this call

lastErrorNumber: Last error number recommendation: Recommendation component: Component that raised the error command: Command age: Age

Return type

errorNumber

getLastErrorAge() → int

Gets the age of the last error that happened :returns: No error = 0

age: time in seconds since the last error happened

Return type

errorNumber

getLastErrorCommand() → str

Gets the command of the last error that happened :returns: No error = 0

command: command running when the last error happened

Return type

errorNumber

getLastErrorComponent() → str

Gets the component of the last error that happened :returns: No error = 0

component: cryostat component of the error

Return type

errorNumber

getLastErrorNumber() → int

Gets the error number of the last error that happened :returns: No error = 0

errorNumber: No error = 0

Return type

localErrorNumber

getLastErrorRecommendation() → str

Gets the recommendation of the last error that happened :returns: No error = 0

recommendation: error recommendation

Return type

errorNumber

getLastErrorsJSONd() → str

Gets the last cryostat errors as a JSON string. :returns: No error = 0

JSON:

Return type

localErrorNumber

getLastErrorsPickled(*pythonVersion: int*) → str

Gets the last cryostat errors from the snapshot. Use getLastErrorsAsJSON() and unJSON the data if this call doesn't work for you (the cryostat data can't be unpickled)

Parameters

pythonVersion –

Returns

No error = 0 Pickled_data:

Return type

localErrorNumber

getLastLongTermEntriesJSONd(*seconds: float*) → str

Get the cryostat long term status of the last seconds

Parameters

seconds –

Returns

No error = 0 version: firmware version number

Return type

errorNumber

getLastShortTermEntriesJSONd(*seconds: float*) → str

Get the cryostat status of the last seconds

Parameters

seconds –

Returns

No error = 0 version: firmware version number

Return type

errorNumber

getLogLines(*maxBytes: int*) → str

Get the last lines of the system logfile.

Parameters

maxBytes – maximum bytes to read

Returns

No error = 0 logLines: String containing the logfile lines

Return type

errorNumber

getSecondsFromStartCryostatInstallation() → float

Gets the seconds from start of cryostat installation :returns: No error = 0

seconds: Seconds from start of cryostat installation

Return type

errorNumber

getUserConfigOverrideFile() → str

Get the user JSON file containing the config overrides. To know what entries exist have a look at `getAvailableConfigurationSettings()` and `getConfigurationSettingInformation()` :returns: No error = 0

overrideFile: String containing the override file

Return type

errorNumber

lowerError()

Resets the last error

prepareDatabaseColumnsIntervalAsync(*minRow: int, maxRow: int, columnNames: str, handle: int*)

Prepare the cryostat long term status of a time interval query

Parameters

- **minRow** –
- **maxRow** –
- **columnNames** – Empty string means all available columns
- **handle** –

prepareDatabaseMinMaxInterval(*minTime: float, maxTime: float*) → int

Prepare the cryostat long term status of a time interval query Get the row numbers with `getDatabaseMinMaxResult()` Get the data with `getDatabaseColumnsMinMaxIntervalRows()` minTime and maxTime can be taken from `getDatabaseMinimumLogSeconds()` and `getDatabaseMaximumLogSeconds()`

Parameters

- **minTime** –
- **maxTime** –

Returns

No error = 0 handle:

Return type

errorNumber

resetUserConfigOverrideFile()

Copy the EEPROM config factory defaults to the user config

setConfigurationSetting(*settingName: str, settingValue: float*)

Sets the configuration setting

Parameters

- **settingName** –
- **settingValue** –

setUserConfigOverrideFile(*fileContent: str*) → str

Set the user JSON file containing the config overrides. To know what entries exist have a look at `getAvailableConfigurationSettings()` and `getConfigurationSettingInformation()`

Parameters

fileContent – Contents of the config override file

Returns

No error = 0 hint_string: Free text string to give the caller some hints if things don't work

Return type

errorNumber

atto_device.CRYOEUVU.system_service module**class** atto_device.CRYOEUVU.system_service.**System_service**(*device*)

Bases: object

apply(*key: int*)

Apply temporary system configuration

Parameters

key –

errorNumberToRecommendation(*language: int, errNbr: int*) → str

Get a recommendation for the error code

Parameters

- **language** – integer: Language code
- **errNbr** – interger: Error code to translate

Returns

errNo errorCode value_string: string: Error recommendation (currently returning an int = 0 until we have recommendations)

Return type

value_errNo

errorNumberToString(*language: int, errNbr: int*) → str

Get a description of an error code

Parameters

- **language** – integer: Language code 0 for the error name, 1 for a more user friendly error message
- **errNbr** – interger: Error code to translate

Returns

errNo errorCode value_string: string: Error description

Return type

value_errNo

factoryReset()

Turns on the factory reset flag.

getDeviceName() → str

Get the actual device name :returns: errNo errorCode

value_string: string: actual device name

Return type

value_errNo

getFirmwareVersion() → str

Get the firmware version of the system :returns: errNo errorCode

value_string: string: The firmware version

Return type

value_errNo

getFluxCode() → str

Get the flux code of the system :returns: errNo errorCode

value_string: string: flux code

Return type

value_errNo

getHostname() → str

Return device hostname :returns: errNo errorCode

available: available

Return type

value_errNo

getMacAddress() → str

Get the mac address of the system :returns: errNo errorCode

value_string: string: Mac address of the system

Return type

value_errNo

getSerialNumber() → str

Get the serial number of the system :returns: errNo errorCode

value_string: string: Serial number

Return type

value_errNo

rebootSystem()

Reboot the system

setDeviceName(name: str)

Set custom name for the device

Parameters

name – string: device name

setTime(day: int, month: int, year: int, hour: int, minute: int, second: int)

Set system time manually

Parameters

- **day** – integer: Day (1-31)
- **month** – integer: Day (1-12)
- **year** – integer: Day (eg. 2021)
- **hour** – integer: Day (0-23)
- **minute** – integer: Day (0-59)
- **second** – integer: Day (0-59)

softReset()

Performs a soft reset (Reset without deleting the network settings).

updateTimeFromInternet()

Update system time by querying attocube.com

atto_device.CRYOEUVU.tboard module**class atto_device.CRYOEUVU.tboard.Tboard(device)**

Bases: object

autotunePIDError() → int

Gets last autotune error :returns: No error = 0

Autotune_errorcode:

Return type

errorNumber

autotunePIDRunning() → bool

Check whether PID autotune is currently running :returns: No error = 0

running:

Return type

errorNumber

autotunePIDStage() → int

Gets last autotune stage :returns: No error = 0

Autotune_errorcode:

Return type
errorNumber

downloadCalibrationCurve(*channel: int*) → str

Gets the sensor .340 calibration curve

Parameters
channel –

Returns
No error = 0 calibration_data:

Return type
errorNumber

getHeaterPIDChannel(*channelNumber: int*)

Get PID values from channel

Parameters
channelNumber –

Returns
No error = 0 p_output: i_output: d_output:

Return type
errorNumber

getHeaterPowerChannel(*channelNumber: int*) → float

Get heater output power

Parameters
channelNumber –

Returns
No error = 0 power:

Return type
errorNumber

getHeaterSetpointChannel(*channelNumber: int*) → float

Get heater setpoint

Parameters
channelNumber –

Returns
No error = 0 set_point:

Return type
errorNumber

getHeaterZoneSettingsChannel(*channelNumber: int*)

Get heater zone settings channel

Parameters
channelNumber –

Returns
No error = 0 upperbound: Upper temperature bound of zone p_output: i_output: d_output:
manualOutput: heatingRange: 0 -> off, 1 -> attocube OEM on, 1 -> 335, 336 Low, , 2 ->
335, 336 Medium, , 3 -> 335, 336 High rampRate:

Return type
errorNumber

getRawSensorInput(*channelNumber: int*) → float

Get raw sensor input.

Parameters
channelNumber –

Returns
No error = 0 Raw_sensor_input:

Return type
errorNumber

getResistance(*channelNumber: int*) → float

Gets the resistance

Parameters
channelNumber –

Returns
No error = 0 resistance: :param channelNumber:

Return type
errorNumber

getTemperature(*channelNumber: int*) → float

Gets the temperature

Parameters
channelNumber –

Returns
No error = 0 temperature: :param channelNumber:

Return type
errorNumber

isHeatingChannel(*channelNumber: int*) → bool

Check if channel is heating

Parameters
channelNumber –

Returns
No error = 0 isHeating:

Return type
errorNumber

setHeaterPIDChannel(*channelNumber: int, p_input: float, i_input: float, d_input: float*)

Set PID values for channel

Parameters

- **channelNumber** –
- **p_input** –
- **i_input** –
- **d_input** –

setHeaterSetpointChannel(*channelNumber: int, setpoint: float*)

Start heating in zone mode

Parameters

- **channelNumber** –
- **setpoint** – Temperature set point

setHeaterZoneSettingsChannel(*channelNumber: int, zone: int, upperbound: float, p_input: float, i_input: float, d_input: float, manualOutput: float, heatingRange: int, rampRate: float*)

Set zone parameters for heater channel

Parameters

- **channelNumber** –
- **zone** –
- **upperbound** – Upper temperature bound of zone
- **p_input** –
- **i_input** –
- **d_input** –
- **manualOutput** –
- **heatingRange** – 0 -> off, 1 -> attocube OEM on, 1 -> 335, 336 Low, , 2 -> 335, 336 Medium, , 3 -> 335, 336 High
- **rampRate** –

startHeatingChannel(*channelNumber: int*)

Start heating

Parameters

- channelNumber** –

startHeatingOpenLoopChannel(*channelNumber: int, power: float*)

Start heating in open loop mode

Parameters

- **channelNumber** –
- **power** –

startHeatingZoneModeChannel(*channelNumber: int*)

Start heating in zone mode

Parameters

- channelNumber** –

stopHeatingChannel(*channelNumber: int*)

Stop heating

Parameters

- channelNumber** –

uploadCalibrationCurve(*channel: int, curveData: str*)

Sets the sensor .340 calibration curve may time out, but the upload will still work

Parameters

- **channel** –
- **curveData** – calibration data

atto_device.CRYOEVU.turboPump module

class atto_device.CRYOEVU.turboPump.**Turbopump**(*device*)

Bases: object

getFrequency() → float

Gets the turbo pump frequency :returns: No error = 0

turbo_pump_frequency:

Return type

errorNumber

getStatus() → bool

Gets the turbo pump status :returns: No error = 0

turbo_pump_status:

Return type

errorNumber

getStatusString() → str

Gets the turbo pump status as string :returns: No error = 0

turbo_pump_status_string:

Return type

errorNumber

start()

Starts the turbo pump

stop()

Stops the turbo pump

atto_device.CRYOEVU.update module

class atto_device.CRYOEVU.update.**Update**(*device*)

Bases: object

getLicenseUpdateProgress() → int

Get the progress of running license update :returns: errNo errorCode

value_int: int: progress in percent

Return type

value_errNo

getSwUpdateProgress() → int

Get the progress of running update :returns: errNo errorCode

value_int: int: progress in percent

Return type

value_errNo

licenseUpdateBase64()

Execute the license update with base64 file uploaded.

softwareUpdateBase64()

Execute the update with base64 file uploaded.

uploadLicenseBase64(*offset: int, b64Data: str*)

Upload new license file in format base 64

Parameters

- **offset** – int: offset of the data
- **b64Data** – string: base64 data

uploadSoftwareImageBase64(*offset: int, b64Data: str*)

Upload new firmware image in format base 64

Parameters

- **offset** – int: offset of the data
- **b64Data** – string: base64 data

atto_device.CRYOEUV.valves module**class** atto_device.CRYOEUV.valves.**Valves**(*device*)

Bases: object

close(*valveNumber: int*)

Closes the valve

Parameters**valveNumber** – 0..12**getStatus**(*valveNumber: int*)

Gets the valve status

Parameters**valveNumber** – 0..11**Returns**

No error = 0 valve_status: open_status_pin_status: closed_status_pin_status:

Return type

errorNumber

open(*valveNumber: int*)

Opens the valve

Parameters

valveNumber – 0..12

Module contents

atto_device.IDS package

Submodules

atto_device.IDS.ACS module

exception atto_device.IDS.ACS.**AttoException**(*errorText=None, errorNumber=0*)

Bases: Exception

class atto_device.IDS.ACS.**AttoResult**(*resultDict*)

Bases: object

class atto_device.IDS.ACS.**Device**(*address*)

Bases: object

TCP_PORT = 9090

close()

Closes the connection to the device.

connect()

Initializes and connects the selected AMC device.

static discover(*cls*)

getResponse(*request_id*)

handleError(*response, ignoreFunctionError=False*)

is_open = False

printError(*errorNumber*)

Converts the errorNumber into an error string and prints it to the console. :param errorNumber: :type errorNumber: int

request(*method, params=False*)

Synchronous request.

request_id = 164164

request_id_lock = <unlocked _thread.lock object>

response_buffer = {}

sendRequest(*method, params=False*)

atto_device.IDS.IDS module

```
class atto_device.IDS.IDS.Device(address)
```

Bases: *Device*

```
atto_device.IDS.IDS.discover()
```

atto_device.IDS.about module

```
class atto_device.IDS.about.About(device)
```

Bases: object

```
getInstalledPackages() → str
```

Get list of packages installed on the device :returns: errNo errorCode

value_string: string: Comma separated list of packages

Return type

value_errNo

```
getPackageLicense(pkg: str) → str
```

Get the license for a specific package

Parameters

pkg – string: Package name

Returns

errNo errorCode value_string: string: License for this package

Return type

value_errNo

atto_device.IDS.access module

```
class atto_device.IDS.access.Access(device)
```

Bases: object

```
getLockStatus()
```

This function returns if the device is locked and if the current client is authorized to use the device. :returns: errNo errorCode

value_locked: locked Is the device locked? value_authorized: authorized Is the client authorized?

Return type

value_errNo

```
grantAccess(password: str)
```

Grants access to a locked device for the requesting IP by checking against the password

Parameters

password – string the current password

lock(*password: str*)

This function locks the device with a password, so the calling of functions is only possible with this password. The locking IP is automatically added to the devices which can access functions

Parameters

password – string the password to be set

unlock(*password: str*)

This function unlocks the device, so it will not be necessary to execute the grantAccess function to run any function

Parameters

password – string the current password

atto_device.IDS.adjustment module

class atto_device.IDS.adjustment.**Adjustment**(*device*)

Bases: object

getAdjustmentEnabled() → bool

This function can be used to see if the adjustment is running :returns: errNo

value_enable: enable true = enabled; false = disabled

Return type

errNo

getContrastInPer mille()

This function can be used to monitor the alignment contrast (peak-to-peak of the /n basic interference signal amplitude) and the baseline (its offset) during alignment /n mode.

Parameters

axis – Axis to get the value from [0..2]

Returns

warningNo Warning code, can be converted into a string using the errorNumberToString function value_contast: contast Contrast of the base band signal in permille value_baseline: baseline Offset of the contrast measurement in permille value_mixcontrast: mixcontrast lower contrast measurment when measuring a mix contrast (indicated by error code)

Return type

value_warningNo

atto_device.IDS.axis module

class atto_device.IDS.axis.**Axis**(*device*)

Bases: object

apply()

Applies new axis settings.

discard()

Discards new axis settings.

getMasterAxis(*tempVal: int*) → int

Returns the master axis (for more information please refer to the IDS User Manual).

Parameters

tempVal –

Returns

errNo value_masteraxis: masteraxis Axis which is operating as masteraxis [0..2]

Return type

errNo

getPassMode(*tempVal: int*) → int

Reads out the current pass mode.

Parameters

tempVal –

Returns

errNo value_mode: mode 0 = single; pass 1 = dual pass

Return type

errNo

setMasterAxis(*axis: int*)

Sets the master axis (for more information please refer to the IDS User Manual).

Parameters

axis – Axis which is operating as masteraxis [0..2]

setPassMode(*mode: int*)

Sets the desired pass mode.

Parameters

mode – 0 = single pass; 1 = dual pass

atto_device.IDS.displacement module

class atto_device.IDS.displacement.**Displacement**(*device*)

Bases: object

getAbsolutePosition()

The absolute position information is estimated at the measurement initialization procedure.

Parameters

axis – [0|1|2]

Returns

warningNo Warning code, can be converted into a string using the errorNumberToString function
value_position: position position of the axis in pm

Return type

value_warningNo

getAbsolutePositions(*ignoreFunctionError=True*)

The absolute position information is estimated at the measurement initialization /nprocedure. :returns: warningNo Warning code, can be converted into a string using the errorNumberToString function

value_position1: position0 position of the axis 0 in pm
value_position2: position1 position of the axis 1 in pm
value_position3: position2 position of the axis 2 in pm

Return type

value_warningNo

getAverageN() → int

Reads-out the averaging (lowpass) parameter N. :returns: errNo

value_averageN: averageN A value from 0 to 24

Return type

errNo

getAxesDisplacement(ignoreFunctionError=True)

Reads out the displacement values of all three measurement axes. :returns: warningNo Warning code, can be converted into a string using the errorNumberToString function

value_displacement1: displacement0 displacement of the axis 0 in pm value_displacement2: displacement1 displacement of the axis 1 in pm value_displacement3: displacement2 displacement of the axis 2 in pm

Return type

value_warningNo

getAxisDisplacement()

Reads out the displacement value of a specific measurement axis.

Parameters**axis** – [0|1|2]**Returns**

warningNo Warning code, can be converted into a string using the errorNumberToString function value_displacement: displacement Displacement of the axis in pm

Return type

value_warningNo

getAxisSignalQuality()

This function can be used to monitor the alignment contrast (peak-to-peak of the basic /ninterference signal amplitude) and the baseline (its offset) during a running /nmeasurement.

Parameters**axis** – [0|1|2]**Returns**

warningNo Warning code, can be converted into a string using the errorNumberToString function value_contrast: contrast Contrast of the base band signal in %% value_baseline: baseline Offset of the contrast measurement in %%

Return type

value_warningNo

getMeasurementEnabled() → bool

This function can be used to see if the measurement is running :returns: errNo

value_enable: enable true = enabled; false = disabled

Return type

errNo

getReferencePosition()

The reference position information is estimated at the measurement initialization procedure or on reset.

Parameters

axis – [0|1|2]

Returns

warningNo Warning code, can be converted into a string using the errorNumberToString function
value_position: position reference position of the axis in pm

Return type

value_warningNo

getReferencePositions(ignoreFunctionError=True)

The reference position information is estimated at the measurement initialization procedure or on reset.
:returns: warningNo Warning code, can be converted into a string using the errorNumberToString function

value_position1: position0 position of the axis 0 in pm value_position2: position1 position of the axis 1 in pm value_position3: position2 position of the axis 2 in pm

Return type

value_warningNo

getSignalQuality()

This function can be used to monitor the alignment contrast (peak-to-peak of the basic /ninterference signal amplitude) and the baseline (its offset) during a running /nmeasurement.

Parameters

axis – [0|1|2]

Returns

warningNo Warning code, can be converted into a string using the errorNumberToString function
value_contrast: contrast Contrast of the base band signal in %% value_baseline: baseline Offset of the contrast measurement in %%

Return type

value_warningNo

linProc(axis: int, fringesnbr: int, samplesperfringe: int, set: int)

Important note: This function is not actively supported anymore.

Parameters

- **axis** – [0|1|2]
- **fringesnbr** – Number of fringes to be acquired
- **samplesperfringe** – Number of samples per fringe
- **set** – 0 = evaluate current nonlinear amplitude /n1 = perform linearization and upload look up table /n2 = Clear look up table /n3 = Perform only calculation of Phi file

Returns

errNo value_lintable: lintable String, which contains all 512 phase related correction values
value_nonlinearp: nonlinearp String which contains the residual positive and negative maximal nonlinear amplitude

Return type

errNo

setAverageN(*value: int*)

Sets the averaging (lowpass) parameter N.

Parameters

value – AverageN value from 0 to 24

atto_device.IDS.ecu module

class atto_device.IDS.ecu.**Ecu**(*device*)

Bases: object

disable()

Disables the ECU interface.

enable()

Enables the ECU interface.

getConnected() → bool

Reads out whether the ECU interface is physically connected or not. Checking if the ECU is connected can only be done on an enabled ECU interface. :returns: Error code, if there was an error, otherwise 0 for ok

connected: true if connected, false if not

Return type

errNo

getEnabled() → bool

Reads out whether the ECU interface is enabled or not. Enabling the ECU interface is crucial for working with the ECU. :returns: Error code, if there was an error, otherwise 0 for ok

enabled: true if enabled, false if not

Return type

errNo

getHumidityInPercent() → float

Reads out the ECU measured air humidity in percent. :returns: Error code, if there was an error, otherwise 0 for ok

humidity: humidity in %

Return type

errNo

getPressureInHPa() → float

Reads out the ECU measured air pressure in hPa. :returns: Error code, if there was an error, otherwise 0 for ok

pressure: pressure in hPa

Return type

errNo

getRefractiveIndex() → float

Reads out the ECU estimated refractive index for the current ECU readings. To get the refractive index for other modes, please see `getRefractiveIndexForCompensation`. :returns: Error code, if there was an error, otherwise 0 for ok

rIndex: refractive index

Return type

errNo

getRefractiveIndexCompensationMode(axis: int) → int

Reads out the compensation mode (see below) which is currently used for the environmental compensation.

Parameters:

axis: Axis to get the mode for.

Parameter has to be -1 for the moment, individual axes will be supported in the next firmware release

Returns:

errNo: Error code, if there was an error, otherwise 0 for ok mode: mode see definition in set function

getRefractiveIndexForCompensation(axis: int) → float

Reads out the refractive index used according to the current environmental compensation mode for this axis.

Parameters:

axis: Axis to get the refractive index for.

Parameter has to be -1 for the moment, individual axes will be supported in the next firmware release

Returns:

errNo: Error code, if there was an error, otherwise 0 for ok rIndex: refractive index

getTemperatureInDegrees() → float

Reads out the ECU measured air temperature in degrees Celsius. :returns: Error code, if there was an error, otherwise 0 for ok

temperature: temperature in degrees C

Return type

errNo

setRefractiveIndexCompensationMode(axis: int, mode: int)

Sets the refractive index mode.

Parameters:

axis: Axis to set the mode for.

Parameter has to be -1 for the moment, individual axes will be supported in the next firmware release

mode: mode can be 0 for direct ECU mode, 1 to take the manual values and calculate the refractive index from this or 2 to directly take the set refractive index

atto_device.IDS.functions module

```
class atto_device.IDS.functions.Functions(device)
```

Bases: object

```
checkAMCinRack()
```

If AMC is on Rack position 0, use it as DHCP server, else use it as DHCP client

atto_device.IDS.manual module

```
class atto_device.IDS.manual.Manual(device)
```

Bases: object

```
getHumidityInPercent(axis: int) → float
```

Reads out the manually configured humidity (compensation mode 1).

Parameters:

axis: Axis to get the humidity for.

Parameter has to be -1 for the moment, individual axes will be supported in the next firmware release

Returns:

errNo: Error code, if there was an error, otherwise 0 for ok humidity: humidity in hPa

```
getPressureInHPa(axis: int) → float
```

Reads out the manually configured Pressure (compensation mode 1).

Parameters:

axis: Axis to get the pressure for.

Parameter has to be -1 for the moment, individual axes will be supported in the next firmware release

Returns:

errNo: Error code, if there was an error, otherwise 0 for ok pressure: pressure in hPa

```
getRefractiveIndex(axis: int) → float
```

Reads out the manually configured value for the refractive index (compensation mode 2).

Parameters:

axis: Axis to get the mode for.

Parameter has to be -1 for the moment, individual axes will be supported in the next firmware release

Returns:

errNo: Error code, if there was an error, otherwise 0 for ok rindex: refractive index

```
getTemperatureInDegrees(axis: int) → float
```

Reads out the manually configured Temperature (compensation mode 1).

Parameters:

axis: Axis to set the temperature for.

Parameter has to be -1 for the moment, individual axes will be supported in the next firmware release

Returns:

errNo: Error code, if there was an error, otherwise 0 for ok temperature: temperature in degree celsius

setHumidityInPercent(*axis: int, humidity: float*)

Sets the manually configured Humidity (compensation mode 1). The input range is defined to 0 to 100 % (valid range for the Ciddor Equation).

Parameters:

axis: Axis to set the humidity for.

Parameter has to be -1 for the moment, individual axes will be supported in the next firmware release

humidity: humidity in Percent

setPressureInHPa(*axis: int, pressure: float*)

Sets the manually configured Pressure (compensation mode 1). The input range is defined to 800 to 1200 hPa (valid range for the Ciddor Equation).

Parameters:

axis: Axis to set the pressure for-

Parameter has to be -1 for the moment, individual axes will be supported in the next firmware release

pressure: pressure in hPa

setRefractiveIndex(*axis: int, rindex: float*)

Sets the refractive index for the direct mode (compensation mode 2). The input range is defined to be greater than 1.

Parameters:

axis: Axis to set the mode for.

Parameter has to be -1 for the moment, individual axes will be supported in the next firmware release

rindex: refractive index

setTemperatureInDegrees(*axis: int, temperature: float*)

Sets the manually configured Temperature (compensation mode 1). The input range is defined to -40 to +100 °C (valid range for the Ciddor Equation).

Parameters:

axis: Axis to set the temperature for.

Parameter has to be -1 for the moment, individual axes will be supported in the next firmware release

temperature: temperature in degree celcius

atto_device.IDS.network module

class atto_device.IDS.network.**Network**(*device*)

Bases: object

apply()

Apply temporary IP configuration and load it

configureWifi(*mode: int, ssid: str, psk: str*)

Change the wifi configuration and applies it

Parameters

- **mode** – 0: Access point, 1: Wifi client

- **ssid** –
- **psk** – Pre-shared key

discard()

Discard temporary IP configuration

getDefaultGateway() → str

Get the default gateway of the device :returns: errNo errorCode

value_Default: Default gateway

Return type

value_errNo

getDnsResolver(priority: int) → str

Get the DNS resolver

Parameters

priority – of DNS resolver (Usually: 0 = Default, 1 = Backup)

Returns

errNo errorCode value_IP: IP address of DNS resolver

Return type

value_errNo

getEnableDhcpClient() → bool

Get the state of DHCP client :returns: errNo errorCode

value_boolean: boolean: true = DHCP client enable, false = DHCP client disable

Return type

value_errNo

getEnableDhcpServer() → bool

Get the state of DHCP server :returns: errNo errorCode

value_boolean: boolean: true = DHCP server enable, false = DHCP server disable

Return type

value_errNo

getIpAddress() → str

Get the IP address of the device :returns: errNo errorCode

value_IP: IP address as string

Return type

value_errNo

getProxyServer() → str

Get the proxy settings of the device :returns: errNo errorCode

value_Proxy: Proxy Server String, empty for no proxy

Return type

value_errNo

getRealIpAddress() → str

Get the real IP address of the device set to the network interface (br0, eth1 or eth0) :returns: errNo errorCode

value_IP: IP address as string

Return type

value_errNo

getSubnetMask() → str

Get the subnet mask of the device :returns: errNo errorCode

value_Subnet: Subnet mask as string

Return type

value_errNo

getWifiMode() → int

Get the operation mode of the wifi adapter :returns: errNo errorCode

value_mode: mode 0: Access point, 1: Wifi client

Return type

value_errNo

getWifiPassphrase() → str

Get the the passphrase of the network hosted (mode: Access point) or connected to (mode: client) :returns: errNo errorCode

value_psk: psk Pre-shared key

Return type

value_errNo

getWifiPresent() → bool

Returns is a Wifi interface is present :returns: errNo errorCode

value_True: True, if interface is present

Return type

value_errNo

getWifiSSID() → str

Get the the SSID of the network hosted (mode: Access point) or connected to (mode: client) :returns: errNo errorCode

SSID: SSID

Return type

value_errNo

setDefaultGateway(gateway: str)

Set the default gateway of the device

Parameters

gateway – Default gateway as string

setDnsResolver(*priority: int, resolver: str*)

Set the DNS resolver

Parameters

- **priority** – of DNS resolver (Usually: 0 = Default, 1 = Backup)
- **resolver** – The resolver's IP address as string

setEnabledDhcpClient(*enable: bool*)

Enable or disable DHCP client

Parameters

enable – boolean: true = enable DHCP client, false = disable DHCP client

setEnabledDhcpServer(*enable: bool*)

Enable or disable DHCP server

Parameters

enable – boolean: true = enable DHCP server, false = disable DHCP server

setIpAddress(*address: str*)

Set the IP address of the device

Parameters

address – IP address as string

setProxyServer(*proxyServer: str*)

Set the proxy server of the device

Parameters

proxyServer – Proxy Server Setting as string

setSubnetMask(*netmask: str*)

Set the subnet mask of the device

Parameters

netmask – Subnet mask as string

setWifiMode(*mode: int*)

Change the operation mode of the wifi adapter

Parameters

mode – 0: Access point, 1: Wifi client

setWifiPassphrase(*psk: str*)

Change the passphrase of the network hosted (mode: Access point) or connected to (mode: client)

Parameters

psk – Pre-shared key

setWifiSSID(*ssid: str*)

Change the SSID of the network hosted (mode: Access point) or connected to (mode: client)

Parameters

ssid –

verify()

Verify that temporary IP configuration is correct

atto_device.IDS.nlc module

class atto_device.IDS.nlc.**Nlc**(*device*)

Bases: object

clearLut(*axis: int*)

Deactivates the LUT and removes it from the device

Parameters

axis – Axis [0|1|2] of the IDS

createLut(*axis: int*)

Creates a new LUT for the given axis.

Parameters

axis – Axis [0|1|2] of the IDS where the LUT should be generated

estimateNonlinearities(*axis: int*)

Estimates the nonlinearity error for the current device settings without changing or updating any settings.

Parameters

axis – Axis [0|1|2] of the IDS of which the nonlinearity error is to be estimated

getDynamicNormalization(*axis: int*) → int

Returns the normalization mode of a specific axis.

Parameters:

axis: Axis [0|1|2] of the IDS of which the normalization mode is queried

Returns:

errNo: Error number if an error occurred while getting normalization mode mode: Normalization Mode

0 Dynamic normalization

1 Normalization frozen

2 Normalization mode determined by target velocity

getHistogram(*axis: int*) → str

Returns a histogram of the measured nonlinearity errors obtained from the last call of createLut or estimateNonlinearities.

Parameters

axis – Axis [0|1|2] of the IDS

Returns

Error number if one occurred during retrieving the histogram histogram: Json dumped histogram array

Return type

errNo

getLut(*axis: int*) → str

Returns the LUT determined by createLut (which can be applied by setLutApplied).

Parameters

axis – Axis [0|1|2] of the IDS

Returns

Error number if one occurred during retrieving the LUT lut: Json dumped LUT array with 512 integers

Return type

errNo

getLutApplied(axis: int) → bool

Returns whether a LUT is applied or not for a given axis.

Parameters

axis – Axis [0|1|2] of the IDS of which the LUT apply rule is queried

Returns

Error number if an error occurred while querying the LUT apply rule apply: True, if LUT is applied on this axis else False

Return type

errNo

getLutStatus(axis: int) → bool

Returns if a LUT is available and if warnings or errors occurred during creation.

Parameters

axis – Axis [0|1|2] of the IDS of which the status of the LUT should be returned

Returns

Error or warning number if one occurred while creating the LUT, 0 in case of no error status: True, if a LUT exists else False

Return type

creationWarning

getNonlinearityEstimation() → str

Returns the LUT created by estimateNonlinearities (read-only mode) to compensate the nonlinearity error of the device for the current device settings. If no estimation was created an array of zeros is returned. :returns: Error number if one occurred loading the LUT

lut: Json dumped LUT array with 512 integers

Return type

errNo

getRawLut(axis: int) → str

Returns the raw lut created by createLut or estimateNonlinearities. For debugging only.

Parameters

axis – Axis [0|1|2] of the IDS

Returns

Error number if one occurred loading the lut raw_lut: Json dumped lut array

Return type

errNo

getVelocityThresholds()

Returns the threshold velocity (in $\mu\text{m/s}$) for mode 2 of setDynamicNormalization. :returns: Error number if an error occurred

velocityOn: Velocity of the target in $\mu\text{m/s}$ when to switch the normalization on (default: 10 $\mu\text{m/s}$) **velocityOff:** Velocity of the target in $\mu\text{m/s}$ when to switch the normalization off (default: 5 $\mu\text{m/s}$)

Return type
errNo

setDynamicNormalization(axis: int, mode: int)

Sets the normalization mode of a specific axis.

Parameters:

axis: Axis [0|1|2] of the IDS of which the normalization mode should be set mode: Normalization Mode

0 Dynamic normalization (default) 1 Normalization frozen (for slow target drifts) 2 Automatic alternation between mode 0 and 1 depending on the target velocity

setLut(axis: int, lut: str)

Uploads a LUT for a specific axis (which can be applied by setLutApplied)

Parameters

- **axis** – Axis [0|1|2] of the IDS
- **lut** – Json dumped LUT array with 512 integers

setLutApplied(axis: int, apply: bool)

Sets the apply rule for the given axis

Parameters

- **axis** – Axis [0|1|2] of the IDS of which the apply rule should be set
- **apply** – True for applying a LUT, False for disabling a LUT

setVelocityThresholds(velocityOn: int, velocityOff: int)

Sets the threshold velocity (in $\mu\text{m/s}$) for mode 2 of setDynamicNormalization. By default, the normalization in mode 2 is frozen for velocities below 5 $\mu\text{m/s}$ and switched to dynamic mode for velocities above 10 $\mu\text{m/s}$.

Parameters

- **velocityOn** – Velocity of the target in $\mu\text{m/s}$ when to switch the normalization on (default: 10 $\mu\text{m/s}$)
- **velocityOff** – Velocity of the target in $\mu\text{m/s}$ when to switch the normalization off (default: 5 $\mu\text{m/s}$)

atto_device.IDS.pilotlaser module

class atto_device.IDS.pilotlaser.Pilotlaser(device)

Bases: object

disable()

Disables the pilot laser.

enable()

Enables the pilot laser.

getEnabled() → bool

Reads out whether the pilot laser is enabled or not. :returns: errNo

value_enable: enable true = enabled; false = disabled

Return type

errNo

atto_device.IDS.realtime module

class atto_device.IDS.realtime.**Realtime**(*device*)

Bases: object

AafIsEnabled() → bool

Checks if the anti-aliasing filter is enabled. :returns: errNo

value_enabled: enabled false: Anti-Aliasing Filter is disabled /n true: Anti-Aliasing Filter is enabled

Return type

errNo

apply()

Applies new real time settings.

disableTestChannel()

Disables the test channel.

discard()

Discards new real time settings.

enableTestChannel(*axis: int*)

Enables the Test Channel, which can be used for estimating the maximum signal range.

Parameters

axis – Test Channel Master Axis

getAafAttenuation() → int

Returns the current attenuation at f_nyquist of the anti-aliasing filter. :returns: errNo

value_attenuation: attenuation [3-30] dB m f_nyquist

Return type

errNo

getAafEnabled() → bool

Checks if the anti-aliasing filter is enabled. :returns: errNo

value_enabled: enabled false - the Anti-Aliasing Filter is disabled /n true - the Anti-Aliasing Filter is enabled

Return type

errNo

getAafWindow() → int

Returns the current filter window of the anti-aliasing filter. :returns: errNo

value_window: window 0 = Rectangular,/n 1 = Cosine,/n 2 = Cosine^2,/n 3 = Hamming,/n 4 = Raised Cosine,/n 5 = Automatic

Return type

errNo

getHighPassCutOffFreq(tempVal: int) → int

Reads out the high pass filter number of Linear/Analog output mode.

Parameters

tempVal –

Returns

errNo value_value: value N, Linear Analog High Pass Cut-Off frequency is $1600/2^N$ kHz, with N /in [1,24]

Return type

errNo

getLinearRange(tempVal: int) → int

Reads out the range number of Linear/Analog output mode.

Parameters

tempVal –

Returns

errNo value_rangenum: rangenum N, Linear Analog Range is $\pm 2^{(N+1)}$ pm, with N /in [0, 34]

Return type

errNo

getPeriodHsslClk(tempVal: int) → int

Reads out the HSSL period clock.

Parameters

tempVal –

Returns

errNo value_period: period Period in the Range of [40ns..10200ns]

Return type

errNo

getPeriodHsslGap(tempVal: int) → int

Reads out the HSSL period gap.

Parameters

tempVal –

Returns

errNo value_gap: gap Number of clocks

Return type

errNo

getPeriodSinCosClk(*tempVal: int*) → int

Reads out the Sine-Cosine and AquadB period clock.

Parameters

tempVal –

Returns

errNo value_period: period 40ns to 10200ns

Return type

errNo

getResolutionBissC(*tempVal: int*) → int

Reads out the BissC resolution.

Parameters

tempVal –

Returns

errNo value_resolution: resolution 1pm to 65535pm

Return type

errNo

getResolutionHsslHigh(*tempVal: int*) → int

Reads out the HSSL resolution high bit.

Parameters

tempVal –

Returns

errNo value_resolution: resolution Resolution in the Range of [0..46]

Return type

errNo

getResolutionHsslLow(*tempVal: int*) → int

Reads out the HSSL resolution low bit.#

Parameters

tempVal –

Returns

errNo value_resolution: resolution Resolution in the range of [0..46]

Return type

errNo

getResolutionSinCos(*tempVal: int*) → int

Reads out the Sine-Cosine and AquadB resolution.

Parameters

tempVal –

Returns

errNo value_resolution: resolution 1pm to 65535pm

Return type

errNo

getRtDistanceMode(*tempVal: int*) → int

Reads out the distance mode.

Parameters**tempVal** –**Returns**

errNo value_linearMode: linearMode 1 = Displacement (Available in HSSL mode and Linear Mode) /n2 = Absolute Distance (Available in HSSL mode only) /n3 = Vibrometry (Available in Linear mode)

Return type

errNo

getRtOutMode(tempVal: int) → int

Reads out the current realtime output mode.

Parameters**tempVal** –**Returns**

errNo value_rtOutMode: rtOutMode 0 = HSSL (TTL), 1 = HSSL (LVDS), 2 = AquadB (TTL), /n3 = AquadB (LVDS), 4 = SinCos (TTL Error Signal), /n5 = SinCos (LVDS Error Signal), 6 = Linear (TTL), 7 = Linear (LVDS), /n8 = BiSS-C, 9 = Deactivated

Return type

errNo

getTestChannelEnabled() → bool

Checks if the test channel is enabled :returns: errNo

value_enabled: enabled true = enabled, false = disabled

Return type

errNo

setAaf(enabled: int, attenuation: int, window: int)

Sets the anti-aliasing filter with assigned filter window.

Parameters

- **enabled** – 0 - disables the Anti-Aliasing Filter /n 1 - enables the Anti-Aliasing Filter
- **attenuation** – [3-30] dB m f_nyquist
- **window** – 0 = Rectangular, /n 1 = Cosine, /n 2 = Cosine^2, /n 3 = Hamming, /n 4 = Raised Cosine, /n 5 = Automatic

setHighPassCutOffFreq(value: int)

Sets the high pass filter number of Linear/Analog output mode.

Parameters**value** – N, Linear Analog High Pass Cut-Off frequency is $1600/2^N$ kHz, with N /in [1,24]**setLinearRange**(rangenumber: int)

Sets the range number of Linear/Analog output mode.

Parameters**rangenumber** – N, Linear Analog Range is $\pm 2^{(N+1)}$ pm, with N /in [0, 34]**setPeriodHsslClk**(period: int)

Set the HSSL period clock.

Parameters**period** – Period in the Range of [40ns..10200ns]

setPeriodHsslGap(*value: int*)

Set the HSSL gap.

Parameters

value – Number of clocks

setPeriodSinCosClk(*value: int*)

Sets the Sine-Cosine and AquadB period clock.

Parameters

value – period 40ns to 10200ns

setResolutionBissC(*value: int*)

Sets the BissC resolution.

Parameters

value – resolution 1pm to 65535pm

setResolutionHsslHigh(*value: int*)

Sets the HSSL resolution high bit.

Parameters

value – Resolution in the Range of [0..46]

setResolutionHsslLow(*value: int*)

Sets the HSSL resolution low bit.

Parameters

value – Resolution in the Range of [0..46]

setResolutionSinCos(*value: int*)

Sets the Sine-Cosine and AquadB resolution.

Parameters

value – resolution 1pm to 65535pm

setRtDistanceMode(*value: int*)

Sets the distance mode.

Parameters

value – 1 = Displacement (HSSL mode and Linear Mode) /n2 = Absolute Distance (HSSL mode only) /n3 = Vibrometry (Linear mode)

setRtOutMode(*value: int*)

Sets the real time output mode.

Parameters

value – rtOutMode 0 = HSSL (TTL), 1 = HSSL (LVDS), 2 = AquadB (TTL), /n3 = AquadB (LVDS), 4 = SinCos (TTL Error Signal), /n5 = SinCos (LVDS Error Signal), 6 = Linear (TTL), 7 = Linear (LVDS), /n8 = BiSS-C, 9 = Deactivated

atto_device.IDS.system module

class atto_device.IDS.system.**System**(*device*)

Bases: object

getCurrentMode() → str

Reads out the current IDS system state. :returns: errNo

value_mode: mode Values: “system idle”, “measurement starting”, “measurement running”, “optics alignment starting”, “optics alignment running”, “test channels enabled”

Return type

errNo

getDeviceType() → str

Reads out the IDS device type. :returns: errNo

value_type: type Type of IDS (e.g. “IDS3010”)

Return type

errNo

getFeaturesName(*featurenumber: int*) → str

Converts the IDS feature number to its corresponding name.

Parameters

featurenumber – Number of feature

Returns

errNo value_names: names The name of the corresponding feature

Return type

errNo

getFpgaVersion() → str

Reads out the IDS FPGA version. :returns: errNo

value_version: version Version in the form X.Y.Z

Return type

errNo

getInitMode() → int

Returns the Initialization mode. :returns: errNo

value_mode: mode 0 = High Accuracy Initialization; 1 = Quick Initialization

Return type

errNo

getNbrFeaturesActivated() → int

Reads out the amount of activated features activated on the IDS. :returns: errNo

value_nbr: nbr Gives the number of activated features.

Return type

errNo

getSystemError()

Checks for a system error.

resetAxes()

Resets the position value of all measurement axes to zero.

resetAxis(*axis: int*)

Resets the position value of a specific measurement axis to zero.

Parameters

axis – [0|1|2]

resetError(*perform: bool*)

Resets a measurement error that can have occurred with the aim to continue the interrupted measurement.

Parameters

perform – renormalization

setInitMode(*mode: int*)

Sets the mode for the initialization procedure that is performed when starting a measurement.

Parameters

mode – 0 = High Accuracy Initialization; 1 = Quick Initialization

startMeasurement()

Starts the displacement measurement system state.

startOpticsAlignment()

Starts the optical alignment system state.

stopMeasurement()

Stops the displacement measurement system state.

stopOpticsAlignment()

Stops the optical alignment system state.

atto_device.IDS.system_service module**class** atto_device.IDS.system_service.**System_service**(*device*)

Bases: object

apply(*key: int*)

Apply temporary system configuration

Parameters

key –

errorNumberToRecommendation(*language: int, errNbr: int*) → str

Get a recommendation for the error code

Parameters

- **language** – integer: Language code
- **errNbr** – integer: Error code to translate

Returns

errNo errorCode value_string: string: Error recommendation (currently returning an int = 0 until we have recommendations)

Return type

value_errNo

errorNumberToString(*language: int, errNbr: int*) → str

Get a description of an error code

Parameters

- **language** – integer: Language code 0 for the error name, 1 for a more user friendly error message
- **errNbr** – integer: Error code to translate

Returns

errNo errorCode value_string: string: Error description

Return type

value_errNo

factoryReset()

Turns on the factory reset flag.

getDeviceName() → str

Get the actual device name :returns: errNo errorCode

value_string: string: actual device name

Return type

value_errNo

getFirmwareVersion() → str

Get the firmware version of the system :returns: errNo errorCode

value_string: string: The firmware version

Return type

value_errNo

getFluxCode() → str

Get the flux code of the system :returns: errNo errorCode

value_string: string: flux code

Return type

value_errNo

getHostname() → str

Return device hostname :returns: errNo errorCode

available: available

Return type

value_errNo

getMacAddress() → str

Get the mac address of the system :returns: errNo errorCode

value_string: string: Mac address of the system

Return type

value_errNo

getSerialNumber() → str

Get the serial number of the system :returns: errNo errorCode

value_string: string: Serial number

Return type

value_errNo

rebootSystem()

Reboot the system

setDeviceName(name: str)

Set custom name for the device

Parameters**name** – string: device name**setTime(day: int, month: int, year: int, hour: int, minute: int, second: int)**

Set system time manually

Parameters

- **day** – integer: Day (1-31)
- **month** – integer: Day (1-12)
- **year** – integer: Day (eg. 2021)
- **hour** – integer: Day (0-23)
- **minute** – integer: Day (0-59)
- **second** – integer: Day (0-59)

softReset()

Performs a soft reset (Reset without deleting the network settings).

updateTimeFromInternet()

Update system time by querying attocube.com

atto_device.IDS.update module**class** atto_device.IDS.update.**Update(device)**

Bases: object

getLicenseUpdateProgress() → int

Get the progress of running license update :returns: errNo errorCode

value_int: int: progress in percent

Return type

value_errNo

getSwUpdateProgress() → int

Get the progress of running update :returns: errNo errorCode

value_int: int: progress in percent

Return type

value_errNo

licenseUpdateBase64()

Execute the license update with base64 file uploaded.

softwareUpdateBase64()

Execute the update with base64 file uploaded.

uploadLicenseBase64(*offset: int, b64Data: str*)

Upload new license file in format base 64

Parameters

- **offset** – int: offset of the data
- **b64Data** – string: base64 data

uploadSoftwareImageBase64(*offset: int, b64Data: str*)

Upload new firmware image in format base 64

Parameters

- **offset** – int: offset of the data
- **b64Data** – string: base64 data

Module contents

3.1.2 Module contents

INDICES AND TABLES

- `genindex`
- `modindex`
- `search`

PYTHON MODULE INDEX

a

atto_device, 179
atto_device.AMC, 44
atto_device.AMC.about, 8
atto_device.AMC.access, 8
atto_device.AMC.ACS, 7
atto_device.AMC.AMC, 8
atto_device.AMC.amcids, 9
atto_device.AMC.control, 11
atto_device.AMC.description, 19
atto_device.AMC.diagnostic, 20
atto_device.AMC.functions, 21
atto_device.AMC.move, 21
atto_device.AMC.network, 24
atto_device.AMC.res, 28
atto_device.AMC.rotcomp, 29
atto_device.AMC.rtin, 30
atto_device.AMC.rtout, 33
atto_device.AMC.status, 36
atto_device.AMC.system_service, 38
atto_device.AMC.test, 40
atto_device.AMC.update, 43
atto_device.CRYO2100, 100
atto_device.CRYO2100.about, 45
atto_device.CRYO2100.access, 45
atto_device.CRYO2100.ACS, 44
atto_device.CRYO2100.action, 46
atto_device.CRYO2100.attoDry2100, 48
atto_device.CRYO2100.condenser, 48
atto_device.CRYO2100.cryoInValve, 53
atto_device.CRYO2100.cryoOutValve, 53
atto_device.CRYO2100.dumpInValve, 54
atto_device.CRYO2100.dumpOutValve, 54
atto_device.CRYO2100.exchange, 54
atto_device.CRYO2100.functions, 60
atto_device.CRYO2100.magnet, 60
atto_device.CRYO2100.main, 66
atto_device.CRYO2100.network, 67
atto_device.CRYO2100.pressures, 70
atto_device.CRYO2100.sample, 71
atto_device.CRYO2100.scrollPump, 76
atto_device.CRYO2100.stage40k, 77
atto_device.CRYO2100.system, 82
atto_device.CRYO2100.system_service, 88
atto_device.CRYO2100.tboard, 91
atto_device.CRYO2100.update, 94
atto_device.CRYO2100.vti, 95
atto_device.CRYO800, 130
atto_device.CRYO800.about, 101
atto_device.CRYO800.access, 101
atto_device.CRYO800.ACS, 100
atto_device.CRYO800.action, 102
atto_device.CRYO800.attoDry800, 104
atto_device.CRYO800.breakVacuumValve, 104
atto_device.CRYO800.compressor, 104
atto_device.CRYO800.exchange, 104
atto_device.CRYO800.functions, 108
atto_device.CRYO800.main, 108
atto_device.CRYO800.membranePump, 108
atto_device.CRYO800.network, 108
atto_device.CRYO800.pressures, 111
atto_device.CRYO800.pumpValve, 112
atto_device.CRYO800.sample, 112
atto_device.CRYO800.sampleValve, 117
atto_device.CRYO800.system, 117
atto_device.CRYO800.system_service, 124
atto_device.CRYO800.tboard, 126
atto_device.CRYO800.turboPump, 129
atto_device.CRYO800.update, 129
atto_device.CRYOEVU, 154
atto_device.CRYOEVU.about, 131
atto_device.CRYOEVU.access, 132
atto_device.CRYOEVU.ACS, 130
atto_device.CRYOEVU.attoDryEVU, 132
atto_device.CRYOEVU.compressor, 132
atto_device.CRYOEVU.functions, 133
atto_device.CRYOEVU.heater, 133
atto_device.CRYOEVU.main, 134
atto_device.CRYOEVU.membranePump, 134
atto_device.CRYOEVU.network, 135
atto_device.CRYOEVU.pressures, 138
atto_device.CRYOEVU.scrollPump, 139
atto_device.CRYOEVU.system, 140
atto_device.CRYOEVU.system_service, 146

atto_device.CRYOEVU.tboard, 148
atto_device.CRYOEVU.turboPump, 152
atto_device.CRYOEVU.update, 152
atto_device.CRYOEVU.valves, 153
atto_device.IDS, 179
atto_device.IDS.about, 155
atto_device.IDS.access, 155
atto_device.IDS.ACS, 154
atto_device.IDS.adjustment, 156
atto_device.IDS.axis, 156
atto_device.IDS.displacement, 157
atto_device.IDS.ecu, 160
atto_device.IDS.functions, 162
atto_device.IDS.IDS, 155
atto_device.IDS.manual, 162
atto_device.IDS.network, 163
atto_device.IDS.nlc, 167
atto_device.IDS.pilotlaser, 169
atto_device.IDS.realtime, 170
atto_device.IDS.system, 175
atto_device.IDS.system_service, 176
atto_device.IDS.update, 178

A

- AafIsEnabled() (*atto_device.IDS.realtime.Realtime method*), 170
- About (*class in atto_device.AMC.about*), 8
- About (*class in atto_device.CRYO2100.about*), 45
- About (*class in atto_device.CRYO800.about*), 101
- About (*class in atto_device.CRYOEVU.about*), 131
- About (*class in atto_device.IDS.about*), 155
- Access (*class in atto_device.AMC.access*), 8
- Access (*class in atto_device.CRYO2100.access*), 45
- Access (*class in atto_device.CRYO800.access*), 101
- Access (*class in atto_device.CRYOEVU.access*), 132
- Access (*class in atto_device.IDS.access*), 155
- Action (*class in atto_device.CRYO2100.action*), 46
- Action (*class in atto_device.CRYO800.action*), 102
- Adjustment (*class in atto_device.IDS.adjustment*), 156
- Amcids (*class in atto_device.AMC.amcids*), 9
- apply() (*atto_device.AMC.network.Network method*), 24
- apply() (*atto_device.AMC.rtin.Rtin method*), 30
- apply() (*atto_device.AMC.rtout.Rtout method*), 33
- apply() (*atto_device.AMC.system_service.System_service method*), 38
- apply() (*atto_device.CRYO2100.network.Network method*), 67
- apply() (*atto_device.CRYO2100.system_service.System_service method*), 88
- apply() (*atto_device.CRYO800.network.Network method*), 108
- apply() (*atto_device.CRYO800.system_service.System_service method*), 124
- apply() (*atto_device.CRYOEVU.network.Network method*), 135
- apply() (*atto_device.CRYOEVU.system_service.System_service method*), 146
- apply() (*atto_device.IDS.axis.Axis method*), 156
- apply() (*atto_device.IDS.network.Network method*), 163
- apply() (*atto_device.IDS.realtime.Realtime method*), 170
- apply() (*atto_device.IDS.system_service.System_service method*), 176
- applyAxis() (*atto_device.AMC.rtout.Rtout method*), 33
- atto_device
 - module, 179
- atto_device.AMC
 - module, 44
- atto_device.AMC.about
 - module, 8
- atto_device.AMC.access
 - module, 8
- atto_device.AMC.ACS
 - module, 7
- atto_device.AMC.AMC
 - module, 8
- atto_device.AMC.amcids
 - module, 9
- atto_device.AMC.control
 - module, 11
- atto_device.AMC.description
 - module, 19
- atto_device.AMC.diagnostic
 - module, 20
- atto_device.AMC.functions
 - module, 21
- atto_device.AMC.move
 - module, 21
- atto_device.AMC.network
 - module, 24
- atto_device.AMC.res
 - module, 28
- atto_device.AMC.rotcomp
 - module, 29
- atto_device.AMC.rtin
 - module, 30
- atto_device.AMC.rtout
 - module, 33
- atto_device.AMC.status
 - module, 36
- atto_device.AMC.system_service
 - module, 38
- atto_device.AMC.test
 - module, 40
- atto_device.AMC.update
 - module, 43

```
atto_device.CRY02100
    module, 100
atto_device.CRY02100.about
    module, 45
atto_device.CRY02100.access
    module, 45
atto_device.CRY02100.ACS
    module, 44
atto_device.CRY02100.action
    module, 46
atto_device.CRY02100.attoDry2100
    module, 48
atto_device.CRY02100.condenser
    module, 48
atto_device.CRY02100.cryoInValve
    module, 53
atto_device.CRY02100.cryoOutValve
    module, 53
atto_device.CRY02100.dumpInValve
    module, 54
atto_device.CRY02100.dumpOutValve
    module, 54
atto_device.CRY02100.exchange
    module, 54
atto_device.CRY02100.functions
    module, 60
atto_device.CRY02100.magnet
    module, 60
atto_device.CRY02100.main
    module, 66
atto_device.CRY02100.network
    module, 67
atto_device.CRY02100.pressures
    module, 70
atto_device.CRY02100.sample
    module, 71
atto_device.CRY02100.scrollPump
    module, 76
atto_device.CRY02100.stage40k
    module, 77
atto_device.CRY02100.system
    module, 82
atto_device.CRY02100.system_service
    module, 88
atto_device.CRY02100.tboard
    module, 91
atto_device.CRY02100.update
    module, 94
atto_device.CRY02100.vti
    module, 95
atto_device.CRY0800
    module, 130
atto_device.CRY0800.about
    module, 101
atto_device.CRY0800.access
    module, 101
atto_device.CRY0800.ACS
    module, 100
atto_device.CRY0800.action
    module, 102
atto_device.CRY0800.attoDry800
    module, 104
atto_device.CRY0800.breakVacuumValve
    module, 104
atto_device.CRY0800.compressor
    module, 104
atto_device.CRY0800.exchange
    module, 104
atto_device.CRY0800.functions
    module, 108
atto_device.CRY0800.main
    module, 108
atto_device.CRY0800.membranePump
    module, 108
atto_device.CRY0800.network
    module, 108
atto_device.CRY0800.pressures
    module, 111
atto_device.CRY0800.pumpValve
    module, 112
atto_device.CRY0800.sample
    module, 112
atto_device.CRY0800.sampleValve
    module, 117
atto_device.CRY0800.system
    module, 117
atto_device.CRY0800.system_service
    module, 124
atto_device.CRY0800.tboard
    module, 126
atto_device.CRY0800.turboPump
    module, 129
atto_device.CRY0800.update
    module, 129
atto_device.CRYOEVU
    module, 154
atto_device.CRYOEVU.about
    module, 131
atto_device.CRYOEVU.access
    module, 132
atto_device.CRYOEVU.ACS
    module, 130
atto_device.CRYOEVU.attoDryEVU
    module, 132
atto_device.CRYOEVU.compressor
    module, 132
atto_device.CRYOEVU.functions
    module, 133
```

atto_device.CRYOEUV.heater
 module, 133
 atto_device.CRYOEUV.main
 module, 134
 atto_device.CRYOEUV.membranePump
 module, 134
 atto_device.CRYOEUV.network
 module, 135
 atto_device.CRYOEUV.pressures
 module, 138
 atto_device.CRYOEUV.scrollPump
 module, 139
 atto_device.CRYOEUV.system
 module, 140
 atto_device.CRYOEUV.system_service
 module, 146
 atto_device.CRYOEUV.tboard
 module, 148
 atto_device.CRYOEUV.turboPump
 module, 152
 atto_device.CRYOEUV.update
 module, 152
 atto_device.CRYOEUV.valves
 module, 153
 atto_device.IDS
 module, 179
 atto_device.IDS.about
 module, 155
 atto_device.IDS.access
 module, 155
 atto_device.IDS.ACS
 module, 154
 atto_device.IDS.adjustment
 module, 156
 atto_device.IDS.axis
 module, 156
 atto_device.IDS.displacement
 module, 157
 atto_device.IDS.ecu
 module, 160
 atto_device.IDS.functions
 module, 162
 atto_device.IDS.IDS
 module, 155
 atto_device.IDS.manual
 module, 162
 atto_device.IDS.network
 module, 163
 atto_device.IDS.nlc
 module, 167
 atto_device.IDS.pilotlaser
 module, 169
 atto_device.IDS.realtime
 module, 170
 atto_device.IDS.system
 module, 175
 atto_device.IDS.system_service
 module, 176
 atto_device.IDS.update
 module, 178
 AttoException, 7, 44, 100, 130, 154
 AttoResult (class in atto_device.AMC.ACS), 7
 AttoResult (class in atto_device.CRYO2100.ACS), 44
 AttoResult (class in atto_device.CRYO800.ACS), 100
 AttoResult (class in atto_device.CRYOEUV.ACS), 130
 AttoResult (class in atto_device.IDS.ACS), 154
 autotunePID() (atto_device.CRYO2100.action.Action
 method), 46
 autotunePID() (atto_device.CRYO800.action.Action
 method), 102
 autotunePIDError() (atto_device.CRYO2100.tboard.Tboard
 method), 91
 autotunePIDError() (atto_device.CRYO800.tboard.Tboard
 method), 126
 autotunePIDError() (atto_device.CRYOEUV.tboard.Tboard
 method), 148
 autotunePIDRunning()
 (atto_device.CRYO2100.tboard.Tboard
 method), 91
 autotunePIDRunning()
 (atto_device.CRYO800.tboard.Tboard method),
 126
 autotunePIDRunning()
 (atto_device.CRYOEUV.tboard.Tboard
 method), 148
 autotunePIDStage() (atto_device.CRYO2100.tboard.Tboard
 method), 91
 autotunePIDStage() (atto_device.CRYO800.tboard.Tboard
 method), 126
 autotunePIDStage() (atto_device.CRYOEUV.tboard.Tboard
 method), 148
 Axis (class in atto_device.IDS.axis), 156

B

Breakvacuumvalve (class in
 atto_device.CRYO800.breakVacuumValve),
 104

C

cancelCurrentCommand()
 (atto_device.CRYO2100.action.Action method),
 46
 cancelCurrentCommand()
 (atto_device.CRYO800.action.Action method),
 102
 changeCryostatType()
 (atto_device.CRYO2100.main.Main method),
 66

changeCryostatType() (atto_device.CRYOEUVU.main.Main method), 134
 checkAMCinRack() (atto_device.AMC.functions.Functions method), 21
 checkAMCinRack() (atto_device.CRYO2100.functions.Functions method), 60
 checkAMCinRack() (atto_device.CRYO800.functions.Functions method), 108
 checkAMCinRack() (atto_device.CRYOEUVU.functions.Functions method), 133
 checkAMCinRack() (atto_device.IDS.functions.Functions method), 162
 checkChassisNbr() (atto_device.AMC.description.Description method), 19
 clearLog() (atto_device.AMC.test.Test method), 40
 clearLut() (atto_device.IDS.nlc.Nlc method), 167
 close() (atto_device.AMC.ACS.Device method), 7
 close() (atto_device.CRYO2100.ACS.Device method), 44
 close() (atto_device.CRYO2100.cryoInValve.Cryoinvalve method), 53
 close() (atto_device.CRYO2100.cryoOutValve.Cryooutvalve method), 53
 close() (atto_device.CRYO2100.dumpInValve.Dumpinvalve method), 54
 close() (atto_device.CRYO2100.dumpOutValve.Dumpoutvalve method), 54
 close() (atto_device.CRYO800.ACS.Device method), 100
 close() (atto_device.CRYOEUVU.ACS.Device method), 130
 close() (atto_device.CRYOEUVU.valves.Valves method), 153
 close() (atto_device.IDS.ACS.Device method), 154
 Compressor (class in atto_device.CRYO800.compressor), 104
 Compressor (class in atto_device.CRYOEUVU.compressor), 132
 Condenser (class in atto_device.CRYO2100.condenser), 48
 configureWifi() (atto_device.AMC.network.Network method), 24
 configureWifi() (atto_device.CRYO2100.network.Network method), 67
 configureWifi() (atto_device.CRYO800.network.Network method), 108
 configureWifi() (atto_device.CRYOEUVU.network.Network method), 135
 configureWifi() (atto_device.IDS.network.Network method), 163
 connect() (atto_device.AMC.ACS.Device method), 7
 connect() (atto_device.CRYO2100.ACS.Device method), 44
 connect() (atto_device.CRYO800.ACS.Device method), 100
 connect() (atto_device.CRYOEUVU.ACS.Device method), 130
 connect() (atto_device.IDS.ACS.Device method), 154
 Control (class in atto_device.AMC.control), 11
 createLut() (atto_device.IDS.nlc.Nlc method), 167
 Cryoinvalve (class in atto_device.CRYO2100.cryoInValve), 53
 Cryooutvalve (class in atto_device.CRYO2100.cryoOutValve), 53

D

Description (class in atto_device.AMC.description), 19
 Device (class in atto_device.AMC.ACS), 7
 Device (class in atto_device.AMC.AMC), 8
 Device (class in atto_device.CRYO2100.ACS), 44
 Device (class in atto_device.CRYO2100.attoDry2100), 48
 Device (class in atto_device.CRYO800.ACS), 100
 Device (class in atto_device.CRYO800.attoDry800), 104
 Device (class in atto_device.CRYOEUVU.ACS), 130
 Device (class in atto_device.CRYOEUVU.attoDryEVU), 132
 Device (class in atto_device.IDS.ACS), 154
 Device (class in atto_device.IDS.IDS), 155
 Diagnostic (class in atto_device.AMC.diagnostic), 20
 disable() (atto_device.IDS.ecu.Ecu method), 160
 disable() (atto_device.IDS.pilotlaser.Pilotlaser method), 169
 disableTestChannel() (atto_device.IDS.realtime.Realtime method), 170
 discard() (atto_device.AMC.network.Network method), 25
 discard() (atto_device.AMC.rtin.Rtin method), 30
 discard() (atto_device.AMC.rtout.Rtout method), 33
 discard() (atto_device.CRYO2100.network.Network method), 67
 discard() (atto_device.CRYO800.network.Network method), 108
 discard() (atto_device.CRYOEUVU.network.Network method), 135
 discard() (atto_device.IDS.axis.Axis method), 156
 discard() (atto_device.IDS.network.Network method), 164
 discard() (atto_device.IDS.realtime.Realtime method), 170
 discardAxis() (atto_device.AMC.rtout.Rtout method), 33
 discardSignalMode() (atto_device.AMC.rtout.Rtout method), 33
 discover() (atto_device.AMC.ACS.Device static method), 7

`discover()` (*atto_device.CRYO2100.ACS.Device static method*), 44
`discover()` (*atto_device.CRYO800.ACS.Device static method*), 100
`discover()` (*atto_device.CRYOEVU.ACS.Device static method*), 130
`discover()` (*atto_device.IDS.ACS.Device static method*), 154
`discover()` (*in module atto_device.AMC.AMC*), 8
`discover()` (*in module atto_device.CRYO2100.attoDry2100*), 48
`discover()` (*in module atto_device.CRYO800.attoDry800*), 104
`discover()` (*in module atto_device.CRYOEVU.attoDryEVU*), 132
`discover()` (*in module atto_device.IDS.IDS*), 155
`Displacement` (*class in atto_device.IDS.displacement*), 157
`downloadCalibrationCurve()` (*atto_device.CRYO2100.condenser.Condenser method*), 48
`downloadCalibrationCurve()` (*atto_device.CRYO2100.exchange.Exchange method*), 54
`downloadCalibrationCurve()` (*atto_device.CRYO2100.magnet.Magnet method*), 60
`downloadCalibrationCurve()` (*atto_device.CRYO2100.sample.Sample method*), 71
`downloadCalibrationCurve()` (*atto_device.CRYO2100.stage40k.Stage40k method*), 77
`downloadCalibrationCurve()` (*atto_device.CRYO2100.tboard.Tboard method*), 91
`downloadCalibrationCurve()` (*atto_device.CRYO2100.vti.Vti method*), 95
`downloadCalibrationCurve()` (*atto_device.CRYO800.exchange.Exchange method*), 104
`downloadCalibrationCurve()` (*atto_device.CRYO800.sample.Sample method*), 112
`downloadCalibrationCurve()` (*atto_device.CRYO800.tboard.Tboard method*), 126
`downloadCalibrationCurve()` (*atto_device.CRYOEVU.tboard.Tboard method*), 149
`downloadCalibrationCurve340()` (*atto_device.CRYO2100.exchange.Exchange method*), 54
`Dumpinvalve` (*class in atto_device.CRYO2100.dumpInValve*), 54
`Dumpoutvalve` (*class in atto_device.CRYO2100.dumpOutValve*), 54

E

`Ecu` (*class in atto_device.IDS.ecu*), 160
`emptyExternalDatabaseToMaximumBytes()` (*atto_device.CRYO2100.system.System method*), 82
`emptyExternalDatabaseToMaximumBytes()` (*atto_device.CRYO800.system.System method*), 117
`emptyExternalDatabaseToMaximumBytes()` (*atto_device.CRYOEVU.system.System method*), 140
`enable()` (*atto_device.IDS.ecu.Ecu method*), 160
`enable()` (*atto_device.IDS.pilotlaser.Pilotlaser method*), 169
`enableTestChannel()` (*atto_device.IDS.realtime.Realtime method*), 170
`errorNumberToRecommendation()` (*atto_device.AMC.system_service.System_service method*), 38
`errorNumberToRecommendation()` (*atto_device.CRYO2100.system_service.System_service method*), 88
`errorNumberToRecommendation()` (*atto_device.CRYO800.system_service.System_service method*), 124
`errorNumberToRecommendation()` (*atto_device.CRYOEVU.system_service.System_service method*), 146
`errorNumberToRecommendation()` (*atto_device.IDS.system_service.System_service method*), 176
`errorNumberToString()` (*atto_device.AMC.system_service.System_service method*), 38
`errorNumberToString()` (*atto_device.CRYO2100.system.System method*), 82
`errorNumberToString()` (*atto_device.CRYO2100.system_service.System_service method*), 89
`errorNumberToString()` (*atto_device.CRYO800.system.System method*), 117
`errorNumberToString()` (*atto_device.CRYO800.system_service.System_service method*), 124
`errorNumberToString()` (*atto_device.CRYOEVU.system.System*

`method`), 140
`errorNumberToString()`
`(atto_device.CRYOEUVU.system_service.System_service`
`method)`, 146
`errorNumberToString()`
`(atto_device.IDS.system_service.System_service`
`method)`, 177
`estimateNonlinearities()` (`atto_device.IDS.nlc.Nlc`
`method`), 167
`Exchange` (class in `atto_device.CRYO2100.exchange`), 54
`Exchange` (class in `atto_device.CRYO800.exchange`), 104
`execute()` (`atto_device.AMC.test.Test` method), 40

F

`factoryReset()` (`atto_device.AMC.system_service.System_service`
`method`), 38
`factoryReset()` (`atto_device.CRYO2100.system_service.System_service`
`method`), 89
`factoryReset()` (`atto_device.CRYO800.system_service.System_service`
`method`), 124
`factoryReset()` (`atto_device.CRYOEUVU.system_service.System_service`
`method`), 147
`factoryReset()` (`atto_device.IDS.system_service.System_service`
`method`), 177
`Functions` (class in `atto_device.AMC.functions`), 21
`Functions` (class in `atto_device.CRYO2100.functions`),
 60
`Functions` (class in `atto_device.CRYO800.functions`),
 108
`Functions` (class in `atto_device.CRYOEUVU.functions`),
 133
`Functions` (class in `atto_device.IDS.functions`), 162

G

`getAafAttenuation()`
`(atto_device.IDS.realtime.Realtime` method),
 170
`getAafEnabled()` (`atto_device.IDS.realtime.Realtime`
`method`), 170
`getAafWindow()` (`atto_device.IDS.realtime.Realtime`
`method`), 170
`getAbsolutePosition()`
`(atto_device.IDS.displacement.Displacement`
`method)`, 157
`getAbsolutePositions()`
`(atto_device.IDS.displacement.Displacement`
`method)`, 157
`getActorName()` (`atto_device.AMC.control.Control`
`method`), 11
`getActorParametersActorName()`
`(atto_device.AMC.control.Control` method), 11
`getActorSensitivity()`
`(atto_device.AMC.control.Control` method), 11
`getActorType()` (`atto_device.AMC.control.Control`
`method`), 12
`getAdjustmentEnabled()`
`(atto_device.IDS.adjustment.Adjustment`
`method)`, 156
`getAllTests()` (`atto_device.AMC.test.Test` method), 41
`getAutoMeasure()` (`atto_device.AMC.control.Control`
`method`), 12
`getAutoprobeInformationJSONd()`
`(atto_device.CRYO2100.system.System`
`method)`, 82
`getAutoprobeInformationJSONd()`
`(atto_device.CRYO800.system.System` method),
 117
`getAutoprobeInformationJSONd()`
`(atto_device.CRYOEUVU.system.System`
`method)`, 140
`getAvailableConfigurationSettings()`
`(atto_device.CRYO2100.system.System`
`method)`, 82
`getAvailableConfigurationSettings()`
`(atto_device.CRYO800.system.System` method),
 117
`getAvailableConfigurationSettings()`
`(atto_device.CRYOEUVU.system.System`
`method)`, 140
`getAvailableCryostatTypes()`
`(atto_device.CRYO2100.main.Main` method),
 66
`getAvailableCryostatTypes()`
`(atto_device.CRYOEUVU.main.Main` method),
 134
`getAverageN()` (`atto_device.IDS.displacement.Displacement`
`method)`, 158
`getAxesDisplacement()`
`(atto_device.IDS.displacement.Displacement`
`method)`, 158
`getAxisDisplacement()`
`(atto_device.IDS.displacement.Displacement`
`method)`, 158
`getAxisSignalQuality()`
`(atto_device.IDS.displacement.Displacement`
`method)`, 158
`getChainGain()` (`atto_device.AMC.res.Res` method), 28
`getConfigOverrideFile()`
`(atto_device.CRYO2100.system.System`
`method)`, 82
`getConfigOverrideFile()`
`(atto_device.CRYO800.system.System` method),
 117
`getConfigOverrideFile()`
`(atto_device.CRYOEUVU.system.System`
`method)`, 140
`getConfigurationConfiguration()`

(*atto_device.CRYO2100.system.System*
method), 82
 getConfigurationConfiguration()
 (*atto_device.CRYO800.system.System* *method*),
 118
 getConfigurationConfiguration()
 (*atto_device.CRYOEVU.system.System*
method), 140
 getConfigurationSetting()
 (*atto_device.CRYO2100.system.System*
method), 83
 getConfigurationSetting()
 (*atto_device.CRYO800.system.System* *method*),
 118
 getConfigurationSetting()
 (*atto_device.CRYOEVU.system.System*
method), 140
 getConfigurationSettingInformation()
 (*atto_device.CRYO2100.system.System*
method), 83
 getConfigurationSettingInformation()
 (*atto_device.CRYO800.system.System* *method*),
 118
 getConfigurationSettingInformation()
 (*atto_device.CRYOEVU.system.System*
method), 141
 getConnected() (*atto_device.IDS.ecu.Ecu* *method*),
 160
 getContrastInPer mille()
 (*atto_device.IDS.adjustment.Adjustment*
method), 156
 getControlAmplitude()
 (*atto_device.AMC.control.Control* *method*), 12
 getControlAQuadBInResolution()
 (*atto_device.AMC.rtin.Rtin* *method*), 30
 getControlAQuadBOut()
 (*atto_device.AMC.rtout.Rtout* *method*), 34
 getControlAQuadBOutClock()
 (*atto_device.AMC.rtout.Rtout* *method*), 34
 getControlAQuadBOutResolution()
 (*atto_device.AMC.rtout.Rtout* *method*), 34
 getControlAutoReset()
 (*atto_device.AMC.control.Control* *method*), 12
 getControlContinuousBkwd()
 (*atto_device.AMC.move.Move* *method*), 21
 getControlContinuousFwd()
 (*atto_device.AMC.move.Move* *method*), 21
 getControlEotOutputDeactive()
 (*atto_device.AMC.move.Move* *method*), 21
 getControlFixOutputVoltage()
 (*atto_device.AMC.control.Control* *method*), 12
 getControlFrequency()
 (*atto_device.AMC.control.Control* *method*), 13
 getControlMove() (*atto_device.AMC.control.Control*
method), 13
 getControlMoveGPIO() (*atto_device.AMC.rtin.Rtin*
method), 31
 getControlOutput() (*atto_device.AMC.control.Control*
method), 13
 getControlReferenceAutoUpdate()
 (*atto_device.AMC.control.Control* *method*), 13
 getControlTargetPosition()
 (*atto_device.AMC.move.Move* *method*), 21
 getControlTargetRange()
 (*atto_device.AMC.control.Control* *method*), 13
 getControlTargetRanges()
 (*atto_device.AMC.rotcomp.Rotcomp* *method*),
 29
 getCrosstalkThreshold()
 (*atto_device.AMC.control.Control* *method*), 13
 getCryoInPressure()
 (*atto_device.CRYO2100.pressures.Pressures*
method), 70
 getCryoOutPressure()
 (*atto_device.CRYO2100.pressures.Pressures*
method), 70
 getCurrentCommand()
 (*atto_device.CRYO2100.action.Action* *method*),
 46
 getCurrentCommand()
 (*atto_device.CRYO800.action.Action* *method*),
 102
 getCurrentCommandStatus()
 (*atto_device.CRYO2100.action.Action* *method*),
 46
 getCurrentCommandStatus()
 (*atto_device.CRYO800.action.Action* *method*),
 102
 getCurrentMode() (*atto_device.IDS.system.System*
method), 175
 getCurrentOutputVoltage()
 (*atto_device.AMC.control.Control* *method*), 14
 getCurrentStatusJSONd()
 (*atto_device.CRYO2100.system.System*
method), 83
 getCurrentStatusJSONd()
 (*atto_device.CRYO800.system.System* *method*),
 118
 getCurrentStatusJSONd()
 (*atto_device.CRYOEVU.system.System*
method), 141
 getDatabaseAsyncQueryResult()
 (*atto_device.CRYO2100.system.System*
method), 83
 getDatabaseAsyncQueryResult()
 (*atto_device.CRYO800.system.System* *method*),
 118
 getDatabaseAsyncQueryResult()

```

        (atto_device.CRYOEVU.system.System
        method), 141
getDatabaseColumnsMinMaxIntervalRows()
    (atto_device.CRYO2100.system.System
    method), 83
getDatabaseColumnsMinMaxIntervalRows()
    (atto_device.CRYO800.system.System method),
    119
getDatabaseColumnsMinMaxIntervalRows()
    (atto_device.CRYOEVU.system.System
    method), 141
getDatabaseMaximumLogSeconds()
    (atto_device.CRYO2100.system.System
    method), 84
getDatabaseMaximumLogSeconds()
    (atto_device.CRYO800.system.System method),
    119
getDatabaseMaximumLogSeconds()
    (atto_device.CRYOEVU.system.System
    method), 141
getDatabaseMinimumLogSeconds()
    (atto_device.CRYO2100.system.System
    method), 84
getDatabaseMinimumLogSeconds()
    (atto_device.CRYO800.system.System method),
    119
getDatabaseMinimumLogSeconds()
    (atto_device.CRYOEVU.system.System
    method), 142
getDatabaseMinMaxResult()
    (atto_device.CRYO2100.system.System
    method), 84
getDatabaseMinMaxResult()
    (atto_device.CRYO800.system.System method),
    119
getDatabaseMinMaxResult()
    (atto_device.CRYOEVU.system.System
    method), 141
getDefaultGateway()
    (atto_device.AMC.network.Network method),
    25
getDefaultGateway()
    (atto_device.CRYO2100.network.Network
    method), 67
getDefaultGateway()
    (atto_device.CRYO800.network.Network
    method), 108
getDefaultGateway()
    (atto_device.CRYOEVU.network.Network
    method), 135
getDefaultGateway()
    (atto_device.IDS.network.Network method),
    164
getDefaultRampRate()

```

```

        (atto_device.CRYO2100.magnet.Magnet
        method), 60
getDeviceName() (atto_device.AMC.system_service.System_service
        method), 38
getDeviceName() (atto_device.CRYO2100.system_service.System_service
        method), 89
getDeviceName() (atto_device.CRYO800.system_service.System_service
        method), 124
getDeviceName() (atto_device.CRYOEVU.system_service.System_service
        method), 147
getDeviceName() (atto_device.IDS.system_service.System_service
        method), 177
getDeviceType() (atto_device.AMC.description.Description
        method), 19
getDeviceType() (atto_device.CRYO2100.system.System
        method), 84
getDeviceType() (atto_device.CRYO800.system.System
        method), 119
getDeviceType() (atto_device.CRYOEVU.system.System
        method), 142
getDeviceType() (atto_device.IDS.system.System
        method), 175
getDiagnosticPower()
    (atto_device.AMC.diagnostic.Diagnostic
    method), 20
getDiagnosticResults()
    (atto_device.AMC.diagnostic.Diagnostic
    method), 20
getDiagnosticStepSize()
    (atto_device.AMC.diagnostic.Diagnostic
    method), 20
getDiagnosticTemperature()
    (atto_device.AMC.diagnostic.Diagnostic
    method), 20
getDnsResolver() (atto_device.AMC.network.Network
        method), 25
getDnsResolver() (atto_device.CRYO2100.network.Network
        method), 67
getDnsResolver() (atto_device.CRYO800.network.Network
        method), 108
getDnsResolver() (atto_device.CRYOEVU.network.Network
        method), 135
getDnsResolver() (atto_device.IDS.network.Network
        method), 164
getDrivenMode() (atto_device.CRYO2100.magnet.Magnet
        method), 60
getDumpPressure() (atto_device.CRYO2100.pressures.Pressures
        method), 70
getDynamicNormalization()
    (atto_device.IDS.nlc.Nlc method), 167
getEnabled() (atto_device.AMC.rotcomp.Rotcomp
        method), 30
getEnabled() (atto_device.IDS.ecu.Ecu method), 160
getEnabled() (atto_device.IDS.pilotlaser.Pilotlaser

```


`method`), 169
`getEnabledDhcpClient()`
 (`atto_device.AMC.network.Network method`), 25
`getEnabledDhcpClient()`
 (`atto_device.CRYO2100.network.Network method`), 67
`getEnabledDhcpClient()`
 (`atto_device.CRYO800.network.Network method`), 109
`getEnabledDhcpClient()`
 (`atto_device.CRYOEVU.network.Network method`), 135
`getEnabledDhcpClient()`
 (`atto_device.IDS.network.Network method`), 164
`getEnabledDhcpServer()`
 (`atto_device.AMC.network.Network method`), 25
`getEnabledDhcpServer()`
 (`atto_device.CRYO2100.network.Network method`), 67
`getEnabledDhcpServer()`
 (`atto_device.CRYO800.network.Network method`), 109
`getEnabledDhcpServer()`
 (`atto_device.CRYOEVU.network.Network method`), 135
`getEnabledDhcpServer()`
 (`atto_device.IDS.network.Network method`), 164
`getExternalSensor()`
 (`atto_device.AMC.control.Control method`), 14
`getFeatures()` (`atto_device.CRYO2100.system.System method`), 84
`getFeatures()` (`atto_device.CRYO800.system.System method`), 119
`getFeatures()` (`atto_device.CRYOEVU.system.System method`), 142
`getFeaturesActivated()`
 (`atto_device.AMC.description.Description method`), 19
`getFeaturesActivated()`
 (`atto_device.CRYO2100.system.System method`), 84
`getFeaturesActivated()`
 (`atto_device.CRYO800.system.System method`), 120
`getFeaturesActivated()`
 (`atto_device.CRYOEVU.system.System method`), 142
`getFeaturesJSON()` (`atto_device.CRYO2100.system.System method`), 85
`getFeaturesJSON()` (`atto_device.CRYO800.system.System method`), 120
`getFeaturesJSON()` (`atto_device.CRYOEVU.system.System method`), 142
`getFeaturesName()` (`atto_device.IDS.system.System method`), 175
`getFieldControl()` (`atto_device.CRYO2100.magnet.Magnet method`), 61
`getFieldsInLeads()` (`atto_device.CRYO2100.magnet.Magnet method`), 61
`getFinePositioningRange()`
 (`atto_device.AMC.control.Control method`), 14
`getFinePositioningSlewRate()`
 (`atto_device.AMC.control.Control method`), 14
`getFirmwareVersion()`
 (`atto_device.AMC.system_service.System_service method`), 38
`getFirmwareVersion()`
 (`atto_device.CRYO2100.system_service.System_service method`), 89
`getFirmwareVersion()`
 (`atto_device.CRYO800.system_service.System_service method`), 124
`getFirmwareVersion()`
 (`atto_device.CRYOEVU.system_service.System_service method`), 147
`getFirmwareVersion()`
 (`atto_device.IDS.system_service.System_service method`), 177
`getFluxCode()` (`atto_device.AMC.system_service.System_service method`), 39
`getFluxCode()` (`atto_device.CRYO2100.system_service.System_service method`), 89
`getFluxCode()` (`atto_device.CRYO800.system_service.System_service method`), 125
`getFluxCode()` (`atto_device.CRYOEVU.system_service.System_service method`), 147
`getFluxCode()` (`atto_device.IDS.system_service.System_service method`), 177
`getFpgaVersion()` (`atto_device.IDS.system.System method`), 175
`getFrequency()` (`atto_device.CRYO2100.scrollPump.Scrollpump method`), 76
`getFrequency()` (`atto_device.CRYOEVU.scrollPump.Scrollpump method`), 139
`getFrequency()` (`atto_device.CRYOEVU.turboPump.Turbopump method`), 152
`getFullCombinedStatus()`
 (`atto_device.AMC.status.Status method`), 36
`getGaugeType()` (`atto_device.CRYOEVU.pressures.Pressures method`), 138
`getGoToBaseRampRateSetting()`
 (`atto_device.CRYO2100.action.Action method`), 46

```

getGoToBaseRampRateSetting()
    (atto_device.CRYO800.action.Action method),
    102
getGpioMode() (atto_device.AMC.rtin.Rtin method), 31
getGroundAxis() (atto_device.AMC.move.Move
    method), 22
getGroundAxisAutoOnTarget()
    (atto_device.AMC.move.Move method), 22
getGroundTargetRange()
    (atto_device.AMC.move.Move method), 22
getH() (atto_device.CRYO2100.magnet.Magnet
    method), 61
getHeaterAllZoneRampRates()
    (atto_device.CRYO2100.condenser.Condenser
    method), 48
getHeaterAllZoneRampRates()
    (atto_device.CRYO2100.exchange.Exchange
    method), 55
getHeaterAllZoneRampRates()
    (atto_device.CRYO2100.sample.Sample
    method), 71
getHeaterAllZoneRampRates()
    (atto_device.CRYO2100.stage40k.Stage40k
    method), 77
getHeaterAllZoneRampRates()
    (atto_device.CRYO2100.vti.Vti method),
    95
getHeaterAllZoneRampRates()
    (atto_device.CRYO800.exchange.Exchange
    method), 104
getHeaterAllZoneRampRates()
    (atto_device.CRYO800.sample.Sample
    method), 112
getHeaterDefaultZoneSettings()
    (atto_device.CRYO2100.condenser.Condenser
    method), 48
getHeaterDefaultZoneSettings()
    (atto_device.CRYO2100.exchange.Exchange
    method), 55
getHeaterDefaultZoneSettings()
    (atto_device.CRYO2100.sample.Sample
    method), 71
getHeaterDefaultZoneSettings()
    (atto_device.CRYO2100.stage40k.Stage40k
    method), 77
getHeaterDefaultZoneSettings()
    (atto_device.CRYO2100.vti.Vti method),
    95
getHeaterDefaultZoneSettings()
    (atto_device.CRYO800.exchange.Exchange
    method), 104
getHeaterDefaultZoneSettings()
    (atto_device.CRYO800.sample.Sample
    method), 112
getHeaterHeatingMode()
    (atto_device.CRYO2100.condenser.Condenser
    method), 48
getHeaterHeatingMode()
    (atto_device.CRYO2100.exchange.Exchange
    method), 55
getHeaterHeatingMode()
    (atto_device.CRYO2100.sample.Sample
    method), 71
getHeaterHeatingMode()
    (atto_device.CRYO2100.stage40k.Stage40k
    method), 77
getHeaterHeatingMode()
    (atto_device.CRYO2100.vti.Vti method),
    95
getHeaterHeatingMode()
    (atto_device.CRYO800.exchange.Exchange
    method), 104
getHeaterHeatingMode()
    (atto_device.CRYO800.sample.Sample
    method), 112
getHeaterNumberOfDefaultZones()
    (atto_device.CRYO2100.condenser.Condenser
    method), 48
getHeaterNumberOfDefaultZones()
    (atto_device.CRYO2100.exchange.Exchange
    method), 55
getHeaterNumberOfDefaultZones()
    (atto_device.CRYO2100.sample.Sample
    method), 71
getHeaterNumberOfDefaultZones()
    (atto_device.CRYO2100.stage40k.Stage40k
    method), 77
getHeaterNumberOfDefaultZones()
    (atto_device.CRYO2100.vti.Vti method),
    95
getHeaterNumberOfDefaultZones()
    (atto_device.CRYO800.exchange.Exchange
    method), 105
getHeaterNumberOfDefaultZones()
    (atto_device.CRYO800.sample.Sample
    method), 112
getHeaterNumberOfZones()
    (atto_device.CRYO2100.condenser.Condenser
    method), 49
getHeaterNumberOfZones()
    (atto_device.CRYO2100.exchange.Exchange
    method), 55
getHeaterNumberOfZones()
    (atto_device.CRYO2100.sample.Sample
    method), 71
getHeaterNumberOfZones()
    (atto_device.CRYO2100.stage40k.Stage40k
    method), 77

```

`getHeaterNumberOfZones()`
 (*atto_device.CRYO2100.vti.Vti method*), 96
`getHeaterNumberOfZones()`
 (*atto_device.CRYO800.exchange.Exchange method*), 105
`getHeaterNumberOfZones()`
 (*atto_device.CRYO800.sample.Sample method*), 113
`getHeaterPIDChannel()`
 (*atto_device.CRYO2100.tboard.Tboard method*), 91
`getHeaterPIDChannel()`
 (*atto_device.CRYO800.tboard.Tboard method*), 126
`getHeaterPIDChannel()`
 (*atto_device.CRYOEVU.tboard.Tboard method*), 149
`getHeaterPower()` (*atto_device.CRYO2100.condenser.Condenser method*), 49
`getHeaterPower()` (*atto_device.CRYO2100.exchange.Exchange method*), 55
`getHeaterPower()` (*atto_device.CRYO2100.sample.Sample method*), 72
`getHeaterPower()` (*atto_device.CRYO2100.stage40k.Stage40k method*), 78
`getHeaterPower()` (*atto_device.CRYO2100.vti.Vti method*), 96
`getHeaterPower()` (*atto_device.CRYO800.exchange.Exchange method*), 105
`getHeaterPower()` (*atto_device.CRYO800.sample.Sample method*), 113
`getHeaterPower()` (*atto_device.CRYOEVU.heater.Heater method*), 133
`getHeaterPowerChannel()`
 (*atto_device.CRYO2100.tboard.Tboard method*), 91
`getHeaterPowerChannel()`
 (*atto_device.CRYO800.tboard.Tboard method*), 127
`getHeaterPowerChannel()`
 (*atto_device.CRYOEVU.tboard.Tboard method*), 149
`getHeaterRampData()`
 (*atto_device.CRYO2100.condenser.Condenser method*), 49
`getHeaterRampData()`
 (*atto_device.CRYO2100.exchange.Exchange method*), 56
`getHeaterRampData()`
 (*atto_device.CRYO2100.sample.Sample method*), 72
`getHeaterRampData()`
 (*atto_device.CRYO2100.stage40k.Stage40k method*), 78
`getHeaterRampData()` (*atto_device.CRYO2100.vti.Vti method*), 96
`getHeaterRampData()` (*atto_device.CRYO800.sample.Sample method*), 113
`getHeaterRes()` (*atto_device.CRYO2100.condenser.Condenser method*), 49
`getHeaterRes()` (*atto_device.CRYO2100.exchange.Exchange method*), 56
`getHeaterRes()` (*atto_device.CRYO2100.sample.Sample method*), 72
`getHeaterRes()` (*atto_device.CRYO2100.stage40k.Stage40k method*), 78
`getHeaterRes()` (*atto_device.CRYO2100.vti.Vti method*), 96
`getHeaterRes()` (*atto_device.CRYO800.exchange.Exchange method*), 105
`getHeaterRes()` (*atto_device.CRYO800.sample.Sample method*), 113
`getHeaterSetpointChannel()`
 (*atto_device.CRYO2100.tboard.Tboard method*), 92
`getHeaterSetpointChannel()`
 (*atto_device.CRYO800.tboard.Tboard method*), 127
`getHeaterSetpointChannel()`
 (*atto_device.CRYOEVU.tboard.Tboard method*), 149
`getHeaterStatus()` (*atto_device.CRYO2100.condenser.Condenser method*), 49
`getHeaterStatus()` (*atto_device.CRYO2100.exchange.Exchange method*), 56
`getHeaterStatus()` (*atto_device.CRYO2100.sample.Sample method*), 72
`getHeaterStatus()` (*atto_device.CRYO2100.stage40k.Stage40k method*), 78
`getHeaterStatus()` (*atto_device.CRYO2100.vti.Vti method*), 96
`getHeaterStatus()` (*atto_device.CRYO800.exchange.Exchange method*), 105
`getHeaterStatus()` (*atto_device.CRYO800.sample.Sample method*), 113
`getHeaterWireRes()` (*atto_device.CRYO2100.condenser.Condenser method*), 49
`getHeaterWireRes()` (*atto_device.CRYO2100.exchange.Exchange method*), 56
`getHeaterWireRes()` (*atto_device.CRYO2100.sample.Sample method*), 72
`getHeaterWireRes()` (*atto_device.CRYO2100.stage40k.Stage40k method*), 78
`getHeaterWireRes()` (*atto_device.CRYO2100.vti.Vti method*), 96
`getHeaterWireRes()` (*atto_device.CRYO800.sample.Sample method*), 113

```

        method), 113
getHeaterZoneSettings()
    (atto_device.CRYO2100.condenser.Condenser
    method), 49
getHeaterZoneSettings()
    (atto_device.CRYO2100.exchange.Exchange
    method), 56
getHeaterZoneSettings()
    (atto_device.CRYO2100.sample.Sample
    method), 72
getHeaterZoneSettings()
    (atto_device.CRYO2100.stage40k.Stage40k
    method), 78
getHeaterZoneSettings()
    (atto_device.CRYO2100.vti.Vti
    method),
    96
getHeaterZoneSettings()
    (atto_device.CRYO800.exchange.Exchange
    method), 105
getHeaterZoneSettings()
    (atto_device.CRYO800.sample.Sample
    method), 113
getHeaterZoneSettingsChannel()
    (atto_device.CRYO2100.tboard.Tboard
    method), 92
getHeaterZoneSettingsChannel()
    (atto_device.CRYO800.tboard.Tboard
    method),
    127
getHeaterZoneSettingsChannel()
    (atto_device.CRYOEVU.tboard.Tboard
    method), 149
getHighPassCutOffFreq()
    (atto_device.IDS.realtime.Realtime
    method),
    171
getHistogram() (atto_device.IDS.nlc.Nlc
    method), 167
getHostname() (atto_device.AMC.system_service.System_service
    method), 39
getHostname() (atto_device.CRYO2100.system_service.System_service
    method), 89
getHostname() (atto_device.CRYO800.system_service.System_service
    method), 125
getHostname() (atto_device.CRYOEVU.system_service.System_service
    method), 147
getHostname() (atto_device.IDS.system_service.System_service
    method), 177
getHSetPoint() (atto_device.CRYO2100.magnet.Magnet
    method), 61
getHSetPoint3D() (atto_device.CRYO2100.magnet.Magnet
    method), 61
getHState() (atto_device.CRYO2100.magnet.Magnet
    method), 62
getHumidityInPercent() (atto_device.IDS.ecu.Ecu
    method), 160
getHumidityInPercent()
    (atto_device.IDS.manual.Manual
    method),
    162
getInitMode() (atto_device.IDS.system.System
    method), 175
getInputFilterSettings()
    (atto_device.CRYO2100.condenser.Condenser
    method), 50
getInputFilterSettings()
    (atto_device.CRYO2100.exchange.Exchange
    method), 56
getInputFilterSettings()
    (atto_device.CRYO2100.magnet.Magnet
    method), 62
getInputFilterSettings()
    (atto_device.CRYO2100.sample.Sample
    method), 72
getInputFilterSettings()
    (atto_device.CRYO2100.stage40k.Stage40k
    method), 78
getInputFilterSettings()
    (atto_device.CRYO2100.vti.Vti
    method),
    97
getInputFilterSettings()
    (atto_device.CRYO800.exchange.Exchange
    method), 105
getInputFilterSettings()
    (atto_device.CRYO800.sample.Sample
    method), 113
getInstalledPackages()
    (atto_device.AMC.about.About
    method),
    8
getInstalledPackages()
    (atto_device.CRYO2100.about.About
    method),
    45
getInstalledPackages()
    (atto_device.CRYO800.about.About
    method),
    101
getInstalledPackages()
    (atto_device.CRYOEVU.about.About
    method),
    131
getInstalledPackages()
    (atto_device.IDS.about.About
    method), 155
getIntervalLongTermEntriesJSONd()
    (atto_device.CRYO2100.system.System
    method), 85
getIntervalLongTermEntriesJSONd()
    (atto_device.CRYO800.system.System
    method),
    120
getIntervalLongTermEntriesJSONd()
    (atto_device.CRYOEVU.system.System
    method), 142
getIntervalShortTermEntriesJSONd()
    (atto_device.CRYO2100.system.System
    method), 85

```

```

getIntervalShortTermEntriesJSONd()
    (atto_device.CRYO800.system.System method), 120
getIntervalShortTermEntriesJSONd()
    (atto_device.CRYOEVU.system.System method), 143
getIpAddress() (atto_device.AMC.network.Network method), 25
getIpAddress() (atto_device.CRYO2100.network.Network method), 67
getIpAddress() (atto_device.CRYO800.network.Network method), 109
getIpAddress() (atto_device.CRYOEVU.network.Network method), 135
getIpAddress() (atto_device.IDS.network.Network method), 164
getIsInQuenchState()
    (atto_device.CRYO2100.magnet.Magnet method), 62
getLastError() (atto_device.CRYO2100.system.System method), 85
getLastError() (atto_device.CRYO800.system.System method), 120
getLastError() (atto_device.CRYOEVU.system.System method), 143
getLastErrorAge() (atto_device.CRYO2100.system.System method), 85
getLastErrorAge() (atto_device.CRYO800.system.System method), 120
getLastErrorAge() (atto_device.CRYOEVU.system.System method), 143
getLastErrorCommand()
    (atto_device.CRYO2100.system.System method), 85
getLastErrorCommand()
    (atto_device.CRYO800.system.System method), 121
getLastErrorCommand()
    (atto_device.CRYOEVU.system.System method), 143
getLastErrorComponent()
    (atto_device.CRYO2100.system.System method), 86
getLastErrorComponent()
    (atto_device.CRYO800.system.System method), 121
getLastErrorComponent()
    (atto_device.CRYOEVU.system.System method), 143
getLastErrorNumber()
    (atto_device.CRYO2100.system.System method), 86
getLastErrorNumber()
    (atto_device.CRYO800.system.System method), 121
getLastErrorNumber()
    (atto_device.CRYOEVU.system.System method), 143
getLastErrorRecommendation()
    (atto_device.CRYO2100.system.System method), 86
getLastErrorRecommendation()
    (atto_device.CRYO800.system.System method), 121
getLastErrorRecommendation()
    (atto_device.CRYOEVU.system.System method), 144
getLastErrorsJSONd()
    (atto_device.CRYO2100.system.System method), 86
getLastErrorsJSONd()
    (atto_device.CRYO800.system.System method), 121
getLastErrorsJSONd()
    (atto_device.CRYOEVU.system.System method), 144
getLastErrorsPickled()
    (atto_device.CRYO2100.system.System method), 86
getLastErrorsPickled()
    (atto_device.CRYO800.system.System method), 121
getLastErrorsPickled()
    (atto_device.CRYOEVU.system.System method), 144
getLastLongTermEntriesJSONd()
    (atto_device.CRYO2100.system.System method), 86
getLastLongTermEntriesJSONd()
    (atto_device.CRYO800.system.System method), 122
getLastLongTermEntriesJSONd()
    (atto_device.CRYOEVU.system.System method), 144
getLastShortTermEntriesJSONd()
    (atto_device.CRYO2100.system.System method), 87
getLastShortTermEntriesJSONd()
    (atto_device.CRYO800.system.System method), 122
getLastShortTermEntriesJSONd()
    (atto_device.CRYOEVU.system.System method), 144
getLeadsHot() (atto_device.CRYO2100.magnet.Magnet method), 62
getLicenseUpdateProgress()
    (atto_device.AMC.update.Update method), 43

```

```

getLicenseUpdateProgress()
    (atto_device.CRYO2100.update.Update
    method), 94
getLicenseUpdateProgress()
    (atto_device.CRYO800.update.Update
    method), 129
getLicenseUpdateProgress()
    (atto_device.CRYOEVU.update.Update
    method), 152
getLicenseUpdateProgress()
    (atto_device.IDS.update.Update
    method), 178
getLinearization() (atto_device.AMC.res.Res
    method), 28
getLinearRange() (atto_device.IDS.realtime.Realtime
    method), 171
getLockStatus() (atto_device.AMC.access.Access
    method), 8
getLockStatus() (atto_device.CRYO2100.access.Access
    method), 45
getLockStatus() (atto_device.CRYO800.access.Access
    method), 101
getLockStatus() (atto_device.CRYOEVU.access.Access
    method), 132
getLockStatus() (atto_device.IDS.access.Access
    method), 155
getLog() (atto_device.AMC.test.Test method), 41
getLogLines() (atto_device.CRYO2100.system.System
    method), 87
getLogLines() (atto_device.CRYO800.system.System
    method), 122
getLogLines() (atto_device.CRYOEVU.system.System
    method), 144
getLowerSoftLimit()
    (atto_device.AMC.amcids.Amcids
    method), 9
getLUT() (atto_device.AMC.rotcomp.Rotcomp method),
    30
getLut() (atto_device.IDS.nlc.Nlc method), 167
getLutApplied() (atto_device.IDS.nlc.Nlc
    method), 168
getLutSn() (atto_device.AMC.res.Res method), 28
getLutStatus() (atto_device.IDS.nlc.Nlc method), 168
getMacAddress() (atto_device.AMC.system_service.System_service
    method), 39
getMacAddress() (atto_device.CRYO2100.system_service.System_service
    method), 90
getMacAddress() (atto_device.CRYO800.system_service.System_service
    method), 125
getMacAddress() (atto_device.CRYOEVU.system_service.System_service
    method), 147
getMacAddress() (atto_device.IDS.system_service.System_service
    method), 177
getMagnetChannelName()
    (atto_device.CRYO2100.magnet.Magnet
    method), 62
getMagnetStateTimer()
    (atto_device.CRYO2100.magnet.Magnet
    method), 62
getMasterAxis() (atto_device.IDS.axis.Axis
    method), 156
getMaxPower() (atto_device.CRYO2100.condenser.Condenser
    method), 50
getMaxPower() (atto_device.CRYO2100.exchange.Exchange
    method), 56
getMaxPower() (atto_device.CRYO2100.sample.Sample
    method), 73
getMaxPower() (atto_device.CRYO2100.stage40k.Stage40k
    method), 79
getMaxPower() (atto_device.CRYO2100.vti.Vti
    method), 97
getMaxPower() (atto_device.CRYO800.exchange.Exchange
    method), 106
getMaxPower() (atto_device.CRYO800.sample.Sample
    method), 114
getMeasurementEnabled()
    (atto_device.IDS.displacement.Displacement
    method), 158
getMode() (atto_device.AMC.res.Res method), 28
getMode() (atto_device.AMC.rtout.Rtout method), 34
getMotionControlThreshold()
    (atto_device.AMC.control.Control method), 14
getNbrFeaturesActivated()
    (atto_device.IDS.system.System
    method), 175
getNonlinearityEstimation()
    (atto_device.IDS.nlc.Nlc method), 168
getNslMux() (atto_device.AMC.rtin.Rtin method), 31
getNSteps() (atto_device.AMC.move.Move method), 22
getNumberOfMagnetChannels()
    (atto_device.CRYO2100.magnet.Magnet
    method), 63
getNumDefaultRampRates()
    (atto_device.CRYO2100.magnet.Magnet
    method), 63
getNumRampRates() (atto_device.CRYO2100.magnet.Magnet
    method), 63
getOlStatus() (atto_device.AMC.status.Status
    method), 36
getPackageLicense() (atto_device.AMC.about.About
    method), 8
getPackageLicense()
    (atto_device.CRYO2100.about.About
    method), 45
getPackageLicense()
    (atto_device.CRYO800.about.About
    method), 101
getPackageLicense()
    (atto_device.CRYOEVU.about.About
    method), 101

```

(atto_device.CRYOEVU.about.About method), 131
 getPackageLicense() (atto_device.IDS.about.About method), 155
 getPassMode() (atto_device.IDS.axis.Axis method), 157
 getPause3D() (atto_device.CRYO2100.magnet.Magnet method), 63
 getPeriodHsslClk() (atto_device.IDS.realtime.Realtime method), 171
 getPeriodHsslGap() (atto_device.IDS.realtime.Realtime method), 171
 getPeriodSinCosClk() (atto_device.IDS.realtime.Realtime method), 171
 getPersistentMode() (atto_device.CRYO2100.magnet.Magnet method), 63
 getPersistentMode3D() (atto_device.CRYO2100.magnet.Magnet method), 63
 getPersistentSwitchHeaterStatus() (atto_device.CRYO2100.magnet.Magnet method), 64
 getPersistentSwitchPresent() (atto_device.CRYO2100.magnet.Magnet method), 64
 getPID() (atto_device.CRYO2100.condenser.Condenser method), 50
 getPID() (atto_device.CRYO2100.exchange.Exchange method), 57
 getPID() (atto_device.CRYO2100.sample.Sample method), 73
 getPID() (atto_device.CRYO2100.stage40k.Stage40k method), 79
 getPID() (atto_device.CRYO2100.vti.Vti method), 97
 getPID() (atto_device.CRYO800.sample.Sample method), 114
 getPosition() (atto_device.AMC.move.Move method), 22
 getPositionersList() (atto_device.AMC.description.Description method), 19
 positionerSN() (atto_device.AMC.test.Test method), 42
 getPositionsAndVoltages() (atto_device.AMC.control.Control method), 15
 getPressureAndVoltage() (atto_device.CRYOEVU.pressures.Pressures method), 138
 getPressureInHPa() (atto_device.IDS.ecu.Ecu method), 160
 getPressureInHPa() (atto_device.IDS.manual.Manual method), 162
 getPressureVoltage() (atto_device.CRYOEVU.pressures.Pressures method), 138
 getProxyServer() (atto_device.AMC.network.Network method), 25
 getProxyServer() (atto_device.CRYO2100.network.Network method), 68
 getProxyServer() (atto_device.CRYO800.network.Network method), 109
 getProxyServer() (atto_device.CRYOEVU.network.Network method), 136
 getProxyServer() (atto_device.IDS.network.Network method), 164
 getRampControlStatus() (atto_device.CRYO2100.condenser.Condenser method), 50
 getRampControlStatus() (atto_device.CRYO2100.exchange.Exchange method), 57
 getRampControlStatus() (atto_device.CRYO2100.sample.Sample method), 73
 getRampControlStatus() (atto_device.CRYO2100.stage40k.Stage40k method), 79
 getRampControlStatus() (atto_device.CRYO2100.vti.Vti method), 97
 getRampControlStatus() (atto_device.CRYO800.sample.Sample method), 114
 getRampRate() (atto_device.CRYO2100.condenser.Condenser method), 50
 getRampRate() (atto_device.CRYO2100.exchange.Exchange method), 57
 getRampRate() (atto_device.CRYO2100.magnet.Magnet method), 64
 getRampRate() (atto_device.CRYO2100.sample.Sample method), 73
 getRampRate() (atto_device.CRYO2100.stage40k.Stage40k method), 79
 getRampRate() (atto_device.CRYO2100.vti.Vti method), 97
 getRampRate() (atto_device.CRYO800.sample.Sample method), 114
 getRawLut() (atto_device.IDS.nlc.Nlc method), 168
 getRawSensorInput() (atto_device.CRYO2100.tboard.Tboard method), 92
 getRawSensorInput() (atto_device.CRYO800.tboard.Tboard method), 127
 getRawSensorInput() (atto_device.CRYOEVU.tboard.Tboard method), 150

```

getRealIpAddress() (atto_device.AMC.network.Network method), 26
getRealIpAddress() (atto_device.CRYO2100.network.Network method), 68
getRealIpAddress() (atto_device.CRYO800.network.Network method), 109
getRealIpAddress() (atto_device.CRYOEVU.network.Network method), 136
getRealIpAddress() (atto_device.IDS.network.Network method), 164
getRealTimeInChangePerPulse() (atto_device.AMC.rtin.Rtin method), 31
getRealTimeInFeedbackLoopMode() (atto_device.AMC.rtin.Rtin method), 31
getRealTimeInMode() (atto_device.AMC.rtin.Rtin method), 32
getRealTimeInStepsPerPulse() (atto_device.AMC.rtin.Rtin method), 32
getReferencePosition() (atto_device.AMC.control.Control method), 15
getReferencePosition() (atto_device.IDS.displacement.Displacement method), 158
getReferencePositions() (atto_device.IDS.displacement.Displacement method), 159
getRefractiveIndex() (atto_device.IDS.ecu.Ecu method), 160
getRefractiveIndex() (atto_device.IDS.manual.Manual method), 162
getRefractiveIndexCompensationMode() (atto_device.IDS.ecu.Ecu method), 161
getRefractiveIndexForCompensation() (atto_device.IDS.ecu.Ecu method), 161
getReport() (atto_device.AMC.test.Test method), 42
getResistance() (atto_device.CRYO2100.condenser.Condenser method), 50
getResistance() (atto_device.CRYO2100.exchange.Exchange method), 57
getResistance() (atto_device.CRYO2100.magnet.Magnet method), 64
getResistance() (atto_device.CRYO2100.sample.Sample method), 73
getResistance() (atto_device.CRYO2100.stage40k.Stage40k method), 79
getResistance() (atto_device.CRYO2100.vti.Vti method), 97
getResistance() (atto_device.CRYO800.exchange.Exchange method), 106
getResistance() (atto_device.CRYO800.sample.Sample method), 114
getResistance() (atto_device.CRYOEVU.tboard.Tboard method), 150
getResolutionBissC() (atto_device.IDS.realtime.Realtime method), 172
getResolutionHsslHigh() (atto_device.IDS.realtime.Realtime method), 172
getResolutionHsslLow() (atto_device.IDS.realtime.Realtime method), 172
getResolutionSinCos() (atto_device.IDS.realtime.Realtime method), 172
getResponse() (atto_device.AMC.ACS.Device method), 7
getResponse() (atto_device.CRYO2100.ACS.Device method), 44
getResponse() (atto_device.CRYO800.ACS.Device method), 100
getResponse() (atto_device.CRYOEVU.ACS.Device method), 131
getResponse() (atto_device.IDS.ACS.Device method), 154
getRtDistanceMode() (atto_device.IDS.realtime.Realtime method), 172
getRtOutMode() (atto_device.IDS.realtime.Realtime method), 173
getSampleExchangePumpOnSetting() (atto_device.CRYO800.action.Action method), 102
getSampleExchangeRampRateSetting() (atto_device.CRYO2100.action.Action method), 46
getSampleExchangeRampRateSetting() (atto_device.CRYO800.action.Action method), 103
getSampleSpacePressure() (atto_device.CRYO800.pressures.Pressures method), 111
getSecondsFromStartCryostatInstallation() (atto_device.CRYO2100.system.System method), 87
getSecondsFromStartCryostatInstallation() (atto_device.CRYO800.system.System method), 122
getSecondsFromStartCryostatInstallation() (atto_device.CRYOEVU.system.System method), 145
getSensorDirection() (atto_device.AMC.control.Control method), 15
getSensorEnabled() (atto_device.AMC.control.Control method), 15
getSensorStatus() (atto_device.AMC.res.Res method), 28

```

```

getSerialNumber() (atto_device.AMC.system_service.System_service method), 153
method), 39
getSerialNumber() (atto_device.CRYO2100.system_service.System_service method), 90
method), 90
getSerialNumber() (atto_device.CRYO800.system_service.System_service method), 125
method), 125
getSerialNumber() (atto_device.CRYOEUVU.system_service.System_service method), 147
method), 147
getSerialNumber() (atto_device.IDS.system_service.System_service method), 178
method), 178
getSetPoint() (atto_device.CRYO2100.condenser.Condenser method), 50
method), 50
getSetPoint() (atto_device.CRYO2100.exchange.Exchange method), 57
method), 57
getSetPoint() (atto_device.CRYO2100.sample.Sample method), 73
method), 73
getSetPoint() (atto_device.CRYO2100.stage40k.Stage40k method), 79
method), 79
getSetPoint() (atto_device.CRYO2100.vti.Vti method), 97
method), 97
getSetPoint() (atto_device.CRYO800.exchange.Exchange method), 106
method), 106
getSetPoint() (atto_device.CRYO800.sample.Sample method), 114
method), 114
getSignalMode() (atto_device.AMC.rtout.Rtout method), 34
method), 34
getSignalQuality() (atto_device.IDS.displacement.Displacement method), 159
method), 159
getSoftLimitEnabled() (atto_device.AMC.amcids.Amcids method), 9
method), 9
getSoftLimitReached() (atto_device.AMC.amcids.Amcids method), 9
method), 9
getStatus() (atto_device.AMC.test.Test method), 42
method), 42
getStatus() (atto_device.CRYO2100.cryoInValve.Cryoinvalve method), 53
method), 53
getStatus() (atto_device.CRYO2100.cryoOutValve.Cryooutvalve method), 53
method), 53
getStatus() (atto_device.CRYO2100.dumpInValve.Dumpinvalve method), 54
method), 54
getStatus() (atto_device.CRYO2100.dumpOutValve.Dumpoutvalve method), 54
method), 54
getStatus() (atto_device.CRYO2100.scrollPump.Scrollpump method), 76
method), 76
getStatus() (atto_device.CRYOEUVU.compressor.Compressor method), 132
method), 132
getStatus() (atto_device.CRYOEUVU.membranePump.Membranepump method), 134
method), 134
getStatus() (atto_device.CRYOEUVU.scrollPump.Scrollpump method), 139
method), 139
getStatus() (atto_device.CRYOEUVU.turboPump.Turbopump method), 152
method), 152
getStatus() (atto_device.CRYOEUVU.valves.Valves method), 178
method), 178
getStatusConnected() (atto_device.AMC.status.Status method), 36
method), 36
getStatusError() (atto_device.AMC.status.Status method), 36
method), 36
getStatusEntBkwd() (atto_device.AMC.status.Status method), 36
method), 36
getStatusEntFwd() (atto_device.AMC.status.Status method), 37
method), 37
getStatusMoving() (atto_device.AMC.status.Status method), 37
method), 37
getStatusMovingAllAxes() (atto_device.AMC.control.Control method), 15
method), 15
getStatusReference() (atto_device.AMC.status.Status method), 37
method), 37
getStatusString() (atto_device.CRYO2100.scrollPump.Scrollpump method), 76
method), 76
getStatusString() (atto_device.CRYOEUVU.membranePump.Membranepump method), 134
method), 134
getStatusString() (atto_device.CRYOEUVU.scrollPump.Scrollpump method), 139
method), 139
getStatusString() (atto_device.CRYOEUVU.turboPump.Turbopump method), 152
method), 152
getStatusTargetRange() (atto_device.AMC.status.Status method), 37
method), 37
getSubnetMask() (atto_device.AMC.network.Network method), 26
method), 26
getSubnetMask() (atto_device.CRYO2100.network.Network method), 68
method), 68
getSubnetMask() (atto_device.CRYO800.network.Network method), 109
method), 109
getSubnetMask() (atto_device.CRYOEUVU.network.Network method), 136
method), 136
getSubnetMask() (atto_device.IDS.network.Network method), 165
method), 165
getSwUpdateProgress() (atto_device.AMC.update.Update method), 43
method), 43
getSwUpdateProgress() (atto_device.CRYO2100.update.Update method), 94
method), 94
getSwUpdateProgress() (atto_device.CRYO800.update.Update method), 129
method), 129
getSwUpdateProgress() (atto_device.CRYOEUVU.update.Update method), 153
method), 153
getSwUpdateProgress() (atto_device.IDS.update.Update method), 178
method), 178
getSystemError() (atto_device.IDS.system.System

```

```

        method), 176
getTempControlStatus()
    (atto_device.CRYO2100.condenser.Condenser
     method), 50
getTempControlStatus()
    (atto_device.CRYO2100.exchange.Exchange
     method), 57
getTempControlStatus()
    (atto_device.CRYO2100.sample.Sample
     method), 73
getTempControlStatus()
    (atto_device.CRYO2100.stage40k.Stage40k
     method), 79
getTempControlStatus()
    (atto_device.CRYO2100.vti.Vti
     method), 97
getTempControlStatus()
    (atto_device.CRYO800.exchange.Exchange
     method), 106
getTempControlStatus()
    (atto_device.CRYO800.sample.Sample
     method), 114
getTemperature() (atto_device.CRYO2100.condenser.Condenser
                  method), 51
getTemperature() (atto_device.CRYO2100.exchange.Exchange
                  method), 57
getTemperature() (atto_device.CRYO2100.magnet.Magnet
                  method), 64
getTemperature() (atto_device.CRYO2100.sample.Sample
                  method), 73
getTemperature() (atto_device.CRYO2100.stage40k.Stage40k
                  method), 79
getTemperature() (atto_device.CRYO2100.tboard.Tboard
                  method), 92
getTemperature() (atto_device.CRYO2100.vti.Vti
                  method), 98
getTemperature() (atto_device.CRYO800.exchange.Exchange
                  method), 106
getTemperature() (atto_device.CRYO800.sample.Sample
                  method), 114
getTemperature() (atto_device.CRYO800.tboard.Tboard
                  method), 127
getTemperature() (atto_device.CRYOEVU.tboard.Tboard
                  method), 150
getTemperatureInDegrees()
    (atto_device.IDS.ecu.Ecu method), 161
getTemperatureInDegrees()
    (atto_device.IDS.manual.Manual
     method), 162
getTestChannelEnabled()
    (atto_device.IDS.realtime.Realtime
     method), 173
getTestParameters() (atto_device.AMC.test.Test
                     method), 42
getTestplatz() (atto_device.AMC.test.Test
                method), 43
getTriggerConfig() (atto_device.AMC.rtout.Rtout
                   method), 34
getUpperSoftLimit()
    (atto_device.AMC.amcids.Amcids
     method), 10
getUserConfigOverrideFile()
    (atto_device.CRYO2100.system.System
     method), 87
getUserConfigOverrideFile()
    (atto_device.CRYO800.system.System
     method), 122
getUserConfigOverrideFile()
    (atto_device.CRYOEVU.system.System
     method), 145
getVelocityThresholds() (atto_device.IDS.nlc.Nlc
                        method), 168
getVolt() (atto_device.CRYO2100.magnet.Magnet
          method), 65
getWaitForEvent() (atto_device.CRYO2100.action.Action
                  method), 47
getWaitForEvent() (atto_device.CRYO800.action.Action
                  method), 103
getWifiMode() (atto_device.AMC.network.Network
              method), 26
getWifiMode() (atto_device.CRYO2100.network.Network
              method), 68
getWifiMode() (atto_device.CRYO800.network.Network
              method), 109
getWifiMode() (atto_device.CRYOEVU.network.Network
              method), 136
getWifiMode() (atto_device.IDS.network.Network
              method), 165
getWifiPassphrase()
    (atto_device.AMC.network.Network
     method), 26
getWifiPassphrase()
    (atto_device.CRYO2100.network.Network
     method), 68
getWifiPassphrase()
    (atto_device.CRYO800.network.Network
     method), 110
getWifiPassphrase()
    (atto_device.CRYOEVU.network.Network
     method), 136
getWifiPassphrase()
    (atto_device.IDS.network.Network
     method), 165
getWifiPresent() (atto_device.AMC.network.Network
                 method), 26
getWifiPresent() (atto_device.CRYO2100.network.Network
                 method), 68
getWifiPresent() (atto_device.CRYO800.network.Network
                 method), 109

```

`method`), 110
`getWifiPresent()` (`atto_device.CRYOEVU.network.Network`
`method`), 136
`getWifiPresent()` (`atto_device.IDS.network.Network`
`method`), 165
`getWifiSSID()` (`atto_device.AMC.network.Network`
`method`), 26
`getWifiSSID()` (`atto_device.CRYO2100.network.Network`
`method`), 69
`getWifiSSID()` (`atto_device.CRYO800.network.Network`
`method`), 110
`getWifiSSID()` (`atto_device.CRYOEVU.network.Network`
`method`), 137
`getWifiSSID()` (`atto_device.IDS.network.Network`
`method`), 165
`goToBase()` (`atto_device.CRYO2100.action.Action`
`method`), 47
`goToBase()` (`atto_device.CRYO800.action.Action`
`method`), 103
`goToBaseSoft()` (`atto_device.CRYO2100.action.Action`
`method`), 47
`goToBaseSoft()` (`atto_device.CRYO800.action.Action`
`method`), 103
`grantAccess()` (`atto_device.AMC.access.Access`
`method`), 8
`grantAccess()` (`atto_device.CRYO2100.access.Access`
`method`), 45
`grantAccess()` (`atto_device.CRYO800.access.Access`
`method`), 101
`grantAccess()` (`atto_device.CRYOEVU.access.Access`
`method`), 132
`grantAccess()` (`atto_device.IDS.access.Access`
`method`), 155

H

`handleError()` (`atto_device.AMC.ACS.Device`
`method`), 7
`handleError()` (`atto_device.CRYO2100.ACS.Device`
`method`), 44
`handleError()` (`atto_device.CRYO800.ACS.Device`
`method`), 100
`handleError()` (`atto_device.CRYOEVU.ACS.Device`
`method`), 131
`handleError()` (`atto_device.IDS.ACS.Device` `method`),
154
`Heater` (class in `atto_device.CRYOEVU.heater`), 133
`heaterRampOff()` (`atto_device.CRYO2100.condenser.Condenser`
`method`), 51
`heaterRampOff()` (`atto_device.CRYO2100.exchange.Exchange`
`method`), 58
`heaterRampOff()` (`atto_device.CRYO2100.sample.Sample`
`method`), 74
`heaterRampOff()` (`atto_device.CRYO2100.stage40k.Stage40k`
`method`), 80
`heaterRampOff()` (`atto_device.CRYO2100.vti.Vti`
`method`), 98
`heaterRampOff()` (`atto_device.CRYO800.sample.Sample`
`method`), 115
`heaterRampOn()` (`atto_device.CRYO2100.condenser.Condenser`
`method`), 51
`heaterRampOn()` (`atto_device.CRYO2100.exchange.Exchange`
`method`), 58
`heaterRampOn()` (`atto_device.CRYO2100.sample.Sample`
`method`), 74
`heaterRampOn()` (`atto_device.CRYO2100.stage40k.Stage40k`
`method`), 80
`heaterRampOn()` (`atto_device.CRYO2100.vti.Vti`
`method`), 98
`heaterRampOn()` (`atto_device.CRYO800.sample.Sample`
`method`), 115

I

`initializeHeater()` (`atto_device.CRYOEVU.heater.Heater`
`method`), 133
`is_open` (`atto_device.AMC.ACS.Device` attribute), 7
`is_open` (`atto_device.CRYO2100.ACS.Device` attribute),
44
`is_open` (`atto_device.CRYO800.ACS.Device` attribute),
100
`is_open` (`atto_device.CRYOEVU.ACS.Device` attribute),
131
`is_open` (`atto_device.IDS.ACS.Device` attribute), 154
`isHeatingChannel()` (`atto_device.CRYO2100.tboard.Tboard`
`method`), 92
`isHeatingChannel()` (`atto_device.CRYO800.tboard.Tboard`
`method`), 128
`isHeatingChannel()` (`atto_device.CRYOEVU.tboard.Tboard`
`method`), 150

L

`licenseUpdateBase64()`
(`atto_device.AMC.update.Update` `method`),
43
`licenseUpdateBase64()`
(`atto_device.CRYO2100.update.Update`
`method`), 94
`licenseUpdateBase64()`
(`atto_device.CRYO800.update.Update`
`method`), 130
`licenseUpdateBase64()`
(`atto_device.CRYOEVU.update.Update`
`method`), 153
`licenseUpdateBase64()`
(`atto_device.IDS.update.Update` `method`),
179
`linProc()` (`atto_device.IDS.displacement.Displacement`
`method`), 159
`lock()` (`atto_device.AMC.access.Access` `method`), 9

lock() (*atto_device.CRYO2100.access.Access method*), 46
 lock() (*atto_device.CRYO800.access.Access method*), 102
 lock() (*atto_device.CRYOEVU.access.Access method*), 132
 lock() (*atto_device.IDS.access.Access method*), 155
 lowerError() (*atto_device.CRYO2100.system.System method*), 87
 lowerError() (*atto_device.CRYO800.system.System method*), 122
 lowerError() (*atto_device.CRYOEVU.system.System method*), 145

M

Magnet (*class in atto_device.CRYO2100.magnet*), 60
 Main (*class in atto_device.CRYO2100.main*), 66
 Main (*class in atto_device.CRYO800.main*), 108
 Main (*class in atto_device.CRYOEVU.main*), 134
 Manual (*class in atto_device.IDS.manual*), 162
 Membranepump (*class in atto_device.CRYO800.membranePump*), 108
 Membranepump (*class in atto_device.CRYOEVU.membranePump*), 134
 module
 atto_device, 179
 atto_device.AMC, 44
 atto_device.AMC.about, 8
 atto_device.AMC.access, 8
 atto_device.AMC.ACS, 7
 atto_device.AMC.AMC, 8
 atto_device.AMC.amcids, 9
 atto_device.AMC.control, 11
 atto_device.AMC.description, 19
 atto_device.AMC.diagnostic, 20
 atto_device.AMC.functions, 21
 atto_device.AMC.move, 21
 atto_device.AMC.network, 24
 atto_device.AMC.res, 28
 atto_device.AMC.rotcomp, 29
 atto_device.AMC.rtin, 30
 atto_device.AMC.rtout, 33
 atto_device.AMC.status, 36
 atto_device.AMC.system_service, 38
 atto_device.AMC.test, 40
 atto_device.AMC.update, 43
 atto_device.CRYO2100, 100
 atto_device.CRYO2100.about, 45
 atto_device.CRYO2100.access, 45
 atto_device.CRYO2100.ACS, 44
 atto_device.CRYO2100.action, 46
 atto_device.CRYO2100.attoDry2100, 48

atto_device.CRYO2100.condenser, 48
 atto_device.CRYO2100.cryoInValve, 53
 atto_device.CRYO2100.cryoOutValve, 53
 atto_device.CRYO2100.dumpInValve, 54
 atto_device.CRYO2100.dumpOutValve, 54
 atto_device.CRYO2100.exchange, 54
 atto_device.CRYO2100.functions, 60
 atto_device.CRYO2100.magnet, 60
 atto_device.CRYO2100.main, 66
 atto_device.CRYO2100.network, 67
 atto_device.CRYO2100.pressures, 70
 atto_device.CRYO2100.sample, 71
 atto_device.CRYO2100.scrollPump, 76
 atto_device.CRYO2100.stage40k, 77
 atto_device.CRYO2100.system, 82
 atto_device.CRYO2100.system_service, 88
 atto_device.CRYO2100.tboard, 91
 atto_device.CRYO2100.update, 94
 atto_device.CRYO2100.vti, 95
 atto_device.CRYO800, 130
 atto_device.CRYO800.about, 101
 atto_device.CRYO800.access, 101
 atto_device.CRYO800.ACS, 100
 atto_device.CRYO800.action, 102
 atto_device.CRYO800.attoDry800, 104
 atto_device.CRYO800.breakVacuumValve, 104
 atto_device.CRYO800.compressor, 104
 atto_device.CRYO800.exchange, 104
 atto_device.CRYO800.functions, 108
 atto_device.CRYO800.main, 108
 atto_device.CRYO800.membranePump, 108
 atto_device.CRYO800.network, 108
 atto_device.CRYO800.pressures, 111
 atto_device.CRYO800.pumpValve, 112
 atto_device.CRYO800.sample, 112
 atto_device.CRYO800.sampleValve, 117
 atto_device.CRYO800.system, 117
 atto_device.CRYO800.system_service, 124
 atto_device.CRYO800.tboard, 126
 atto_device.CRYO800.turboPump, 129
 atto_device.CRYO800.update, 129
 atto_device.CRYOEVU, 154
 atto_device.CRYOEVU.about, 131
 atto_device.CRYOEVU.access, 132
 atto_device.CRYOEVU.ACS, 130
 atto_device.CRYOEVU.attoDryEVU, 132
 atto_device.CRYOEVU.compressor, 132
 atto_device.CRYOEVU.functions, 133
 atto_device.CRYOEVU.heater, 133
 atto_device.CRYOEVU.main, 134
 atto_device.CRYOEVU.membranePump, 134
 atto_device.CRYOEVU.network, 135
 atto_device.CRYOEVU.pressures, 138
 atto_device.CRYOEVU.scrollPump, 139

- `atto_device.CRYOEVU.system`, 140
 - `atto_device.CRYOEVU.system_service`, 146
 - `atto_device.CRYOEVU.tboard`, 148
 - `atto_device.CRYOEVU.turboPump`, 152
 - `atto_device.CRYOEVU.update`, 152
 - `atto_device.CRYOEVU.valves`, 153
 - `atto_device.IDS`, 179
 - `atto_device.IDS.about`, 155
 - `atto_device.IDS.access`, 155
 - `atto_device.IDS.ACS`, 154
 - `atto_device.IDS.adjustment`, 156
 - `atto_device.IDS.axis`, 156
 - `atto_device.IDS.displacement`, 157
 - `atto_device.IDS.ecu`, 160
 - `atto_device.IDS.functions`, 162
 - `atto_device.IDS.IDS`, 155
 - `atto_device.IDS.manual`, 162
 - `atto_device.IDS.network`, 163
 - `atto_device.IDS.nlc`, 167
 - `atto_device.IDS.pilotlaser`, 169
 - `atto_device.IDS.realtime`, 170
 - `atto_device.IDS.system`, 175
 - `atto_device.IDS.system_service`, 176
 - `atto_device.IDS.update`, 178
 - `Move` (class in `atto_device.AMC.move`), 21
 - `moveReference()` (`atto_device.AMC.move.Move` method), 22
 - `MultiAxisPositioning()` (`atto_device.AMC.control.Control` method), 11
- ## N
- `Network` (class in `atto_device.AMC.network`), 24
 - `Network` (class in `atto_device.CRYO2100.network`), 67
 - `Network` (class in `atto_device.CRYO800.network`), 108
 - `Network` (class in `atto_device.CRYOEVU.network`), 135
 - `Network` (class in `atto_device.IDS.network`), 163
 - `Nlc` (class in `atto_device.IDS.nlc`), 167
- ## O
- `open()` (`atto_device.CRYO2100.cryoInValve.Cryoinvalve` method), 53
 - `open()` (`atto_device.CRYO2100.cryoOutValve.Cryooutvalve` method), 53
 - `open()` (`atto_device.CRYO2100.dumpInValve.Dumpinvalve` method), 54
 - `open()` (`atto_device.CRYO2100.dumpOutValve.Dumpoutvalve` method), 54
 - `open()` (`atto_device.CRYOEVU.valves.Valves` method), 153
- ## P
- `performNSteps()` (`atto_device.AMC.move.Move` method), 23
 - `Pilotlaser` (class in `atto_device.IDS.pilotlaser`), 169
 - `postEvent()` (`atto_device.CRYO2100.action.Action` method), 47
 - `postEvent()` (`atto_device.CRYO800.action.Action` method), 103
 - `prepareDatabaseColumnsIntervalAsync()` (`atto_device.CRYO2100.system.System` method), 87
 - `prepareDatabaseColumnsIntervalAsync()` (`atto_device.CRYO800.system.System` method), 123
 - `prepareDatabaseColumnsIntervalAsync()` (`atto_device.CRYOEVU.system.System` method), 145
 - `prepareDatabaseMinMaxInterval()` (`atto_device.CRYO2100.system.System` method), 88
 - `prepareDatabaseMinMaxInterval()` (`atto_device.CRYO800.system.System` method), 123
 - `prepareDatabaseMinMaxInterval()` (`atto_device.CRYOEVU.system.System` method), 145
 - `Pressures` (class in `atto_device.CRYO2100.pressures`), 70
 - `Pressures` (class in `atto_device.CRYO800.pressures`), 111
 - `Pressures` (class in `atto_device.CRYOEVU.pressures`), 138
 - `printError()` (`atto_device.AMC.ACS.Device` method), 7
 - `printError()` (`atto_device.CRYO2100.ACS.Device` method), 44
 - `printError()` (`atto_device.CRYO800.ACS.Device` method), 100
 - `printError()` (`atto_device.CRYOEVU.ACS.Device` method), 131
 - `printError()` (`atto_device.IDS.ACS.Device` method), 154
 - `Pumpvalve` (class in `atto_device.CRYO800.pumpValve`), 112
- ## R
- `Realtime` (class in `atto_device.IDS.realtime`), 170
 - `rebootSystem()` (`atto_device.AMC.system_service.System_service` method), 39
 - `rebootSystem()` (`atto_device.CRYO2100.system_service.System_service` method), 90
 - `rebootSystem()` (`atto_device.CRYO800.system_service.System_service` method), 125
 - `rebootSystem()` (`atto_device.CRYOEVU.system_service.System_service` method), 147
 - `rebootSystem()` (`atto_device.IDS.system_service.System_service` method), 178
 - `request()` (`atto_device.AMC.ACS.Device` method), 7

`request()` (`atto_device.CRYO2100.ACS.Device` method), 44
`request()` (`atto_device.CRYO800.ACS.Device` method), 100
`request()` (`atto_device.CRYOEVU.ACS.Device` method), 131
`request()` (`atto_device.IDS.ACS.Device` method), 154
`request_id` (`atto_device.AMC.ACS.Device` attribute), 7
`request_id` (`atto_device.CRYO2100.ACS.Device` attribute), 45
`request_id` (`atto_device.CRYO800.ACS.Device` attribute), 101
`request_id` (`atto_device.CRYOEVU.ACS.Device` attribute), 131
`request_id` (`atto_device.IDS.ACS.Device` attribute), 154
`request_id_lock` (`atto_device.AMC.ACS.Device` attribute), 8
`request_id_lock` (`atto_device.CRYO2100.ACS.Device` attribute), 45
`request_id_lock` (`atto_device.CRYO800.ACS.Device` attribute), 101
`request_id_lock` (`atto_device.CRYOEVU.ACS.Device` attribute), 131
`request_id_lock` (`atto_device.IDS.ACS.Device` attribute), 154
`Res` (class in `atto_device.AMC.res`), 28
`resetAxes()` (`atto_device.IDS.system.System` method), 176
`resetAxis()` (`atto_device.IDS.system.System` method), 176
`resetError()` (`atto_device.IDS.system.System` method), 176
`resetIdsAxis()` (`atto_device.AMC.amcids.Amcids` method), 10
`resetQuenchState()` (`atto_device.CRYO2100.magnet.Magnet` method), 65
`resetUserConfigOverrideFile()` (`atto_device.CRYO2100.system.System` method), 88
`resetUserConfigOverrideFile()` (`atto_device.CRYO800.system.System` method), 123
`resetUserConfigOverrideFile()` (`atto_device.CRYOEVU.system.System` method), 145
`response_buffer` (`atto_device.AMC.ACS.Device` attribute), 8
`response_buffer` (`atto_device.CRYO2100.ACS.Device` attribute), 45
`response_buffer` (`atto_device.CRYO800.ACS.Device` attribute), 101
`response_buffer` (`atto_device.CRYOEVU.ACS.Device` attribute), 131
`response_buffer` (`atto_device.IDS.ACS.Device` attribute), 154
`Rotcomp` (class in `atto_device.AMC.rotcomp`), 29
`Rtin` (class in `atto_device.AMC.rtin`), 30
`Rtout` (class in `atto_device.AMC.rtout`), 33

S

`Sample` (class in `atto_device.CRYO2100.sample`), 71
`Sample` (class in `atto_device.CRYO800.sample`), 112
`sampleExchange()` (`atto_device.CRYO2100.action.Action` method), 47
`sampleExchange()` (`atto_device.CRYO800.action.Action` method), 103
`sampleExchangeSoft()` (`atto_device.CRYO2100.action.Action` method), 47
`sampleExchangeSoft()` (`atto_device.CRYO800.action.Action` method), 103
`Samplevalve` (class in `atto_device.CRYO800.sampleValve`), 117
`Scrollpump` (class in `atto_device.CRYO2100.scrollPump`), 76
`Scrollpump` (class in `atto_device.CRYOEVU.scrollPump`), 139
`searchReferencePosition()` (`atto_device.AMC.control.Control` method), 16
`sendRequest()` (`atto_device.AMC.ACS.Device` method), 8
`sendRequest()` (`atto_device.CRYO2100.ACS.Device` method), 45
`sendRequest()` (`atto_device.CRYO800.ACS.Device` method), 101
`sendRequest()` (`atto_device.CRYOEVU.ACS.Device` method), 131
`sendRequest()` (`atto_device.IDS.ACS.Device` method), 154
`setAaf()` (`atto_device.IDS.realtime.Realtime` method), 173
`setActorParametersByName()` (`atto_device.AMC.control.Control` method), 16
`setActorParametersJson()` (`atto_device.AMC.control.Control` method), 16
`setActorSensitivity()` (`atto_device.AMC.control.Control` method), 16
`setAutoMeasure()` (`atto_device.AMC.control.Control` method), 16
`setAverageN()` (`atto_device.IDS.displacement.Displacement` method), 159
`setChainGain()` (`atto_device.AMC.res.Res` method), 29
`setConfigurationFile()` (`atto_device.AMC.res.Res` method), 29
`setConfigurationSetting()` (`atto_device.CRYO2100.system.System`

`method`), 88
`setConfigurationSetting()`
 (`atto_device.CRYO800.system.System` `method`),
 123
`setConfigurationSetting()`
 (`atto_device.CRYOEVU.system.System`
 `method`), 146
`setControlAmplitude()`
 (`atto_device.AMC.control.Control` `method`), 16
`setControlAQuadBInResolution()`
 (`atto_device.AMC.rtin.Rtin` `method`), 32
`setControlAQuadBOutClock()`
 (`atto_device.AMC.rtout.Rtout` `method`), 35
`setControlAQuadBOutResolution()`
 (`atto_device.AMC.rtout.Rtout` `method`), 35
`setControlAutoReset()`
 (`atto_device.AMC.control.Control` `method`), 16
`setControlContinuousBkwd()`
 (`atto_device.AMC.move.Move` `method`), 23
`setControlContinuousFwd()`
 (`atto_device.AMC.move.Move` `method`), 23
`setControlEotOutputDeactive()`
 (`atto_device.AMC.move.Move` `method`), 23
`setControlFixOutputVoltage()`
 (`atto_device.AMC.control.Control` `method`), 17
`setControlFrequency()`
 (`atto_device.AMC.control.Control` `method`), 17
`setControlMove()` (`atto_device.AMC.control.Control`
 `method`), 17
`setControlMoveGPIO()` (`atto_device.AMC.rtin.Rtin`
 `method`), 32
`setControlOutput()` (`atto_device.AMC.control.Control`
 `method`), 17
`setControlReferenceAutoUpdate()`
 (`atto_device.AMC.control.Control` `method`), 17
`setControlTargetPosition()`
 (`atto_device.AMC.move.Move` `method`), 23
`setControlTargetRange()`
 (`atto_device.AMC.control.Control` `method`), 17
`setCrosstalkThreshold()`
 (`atto_device.AMC.control.Control` `method`), 17
`setD()` (`atto_device.CRYO2100.condenser.Condenser`
 `method`), 51
`setD()` (`atto_device.CRYO2100.exchange.Exchange`
 `method`), 58
`setD()` (`atto_device.CRYO2100.sample.Sample` `method`),
 74
`setD()` (`atto_device.CRYO2100.stage40k.Stage40k`
 `method`), 80
`setD()` (`atto_device.CRYO2100.vti.Vti` `method`), 98
`setD()` (`atto_device.CRYO800.sample.Sample` `method`),
 115
`setDefaultGateway()`
 (`atto_device.AMC.network.Network` `method`),
 26
`setDefaultGateway()`
 (`atto_device.CRYO2100.network.Network`
 `method`), 69
`setDefaultGateway()`
 (`atto_device.CRYO800.network.Network`
 `method`), 110
`setDefaultGateway()`
 (`atto_device.CRYOEVU.network.Network`
 `method`), 137
`setDefaultGateway()`
 (`atto_device.IDS.network.Network` `method`),
 165
`setDeviceName()` (`atto_device.AMC.system_service.System_service`
 `method`), 39
`setDeviceName()` (`atto_device.CRYO2100.system_service.System_service`
 `method`), 90
`setDeviceName()` (`atto_device.CRYO800.system_service.System_service`
 `method`), 125
`setDeviceName()` (`atto_device.CRYOEVU.system_service.System_service`
 `method`), 148
`setDeviceName()` (`atto_device.IDS.system_service.System_service`
 `method`), 178
`setDnsResolver()` (`atto_device.AMC.network.Network`
 `method`), 27
`setDnsResolver()` (`atto_device.CRYO2100.network.Network`
 `method`), 69
`setDnsResolver()` (`atto_device.CRYO800.network.Network`
 `method`), 110
`setDnsResolver()` (`atto_device.CRYOEVU.network.Network`
 `method`), 137
`setDnsResolver()` (`atto_device.IDS.network.Network`
 `method`), 165
`setDrivenMode()` (`atto_device.CRYO2100.magnet.Magnet`
 `method`), 65
`setDynamicNormalization()`
 (`atto_device.IDS.nlc.Nlc` `method`), 169
`setEnabled()` (`atto_device.AMC.rotcomp.Rotcomp`
 `method`), 30
`setEnabledDhcpClient()`
 (`atto_device.AMC.network.Network` `method`),
 27
`setEnabledDhcpClient()`
 (`atto_device.CRYO2100.network.Network`
 `method`), 69
`setEnabledDhcpClient()`
 (`atto_device.CRYO800.network.Network`
 `method`), 110
`setEnabledDhcpClient()`
 (`atto_device.CRYOEVU.network.Network`
 `method`), 137
`setEnabledDhcpClient()`
 (`atto_device.IDS.network.Network` `method`),
 166

```

setEnabledDhcpServer()
    (atto_device.AMC.network.Network method),
    27
setEnabledDhcpServer()
    (atto_device.CRYO2100.network.Network
    method), 69
setEnabledDhcpServer()
    (atto_device.CRYO800.network.Network
    method), 110
setEnabledDhcpServer()
    (atto_device.CRYOEVU.network.Network
    method), 137
setEnabledDhcpServer()
    (atto_device.IDS.network.Network method),
    166
setExternalSensor()
    (atto_device.AMC.control.Control method), 18
setFinePositioningRange()
    (atto_device.AMC.control.Control method), 18
setFinePositioningSlewRate()
    (atto_device.AMC.control.Control method), 18
setGaugeType() (atto_device.CRYOEVU.pressures.Pressures
    method), 138
setGoToBaseRampRateSetting()
    (atto_device.CRYO2100.action.Action method),
    47
setGoToBaseRampRateSetting()
    (atto_device.CRYO800.action.Action method),
    103
setGpioMode() (atto_device.AMC.rtin.Rtin method), 32
setGroundAxis() (atto_device.AMC.move.Move
    method), 23
setGroundAxisAutoOnTarget()
    (atto_device.AMC.move.Move method), 23
setGroundTargetRange()
    (atto_device.AMC.move.Move method), 24
setHeaterAllZoneRampRates()
    (atto_device.CRYO2100.condenser.Condenser
    method), 51
setHeaterAllZoneRampRates()
    (atto_device.CRYO2100.exchange.Exchange
    method), 58
setHeaterAllZoneRampRates()
    (atto_device.CRYO2100.sample.Sample
    method), 74
setHeaterAllZoneRampRates()
    (atto_device.CRYO2100.stage40k.Stage40k
    method), 80
setHeaterAllZoneRampRates()
    (atto_device.CRYO2100.vti.Vti method),
    98
setHeaterAllZoneRampRates()
    (atto_device.CRYO800.exchange.Exchange
    method), 106
setHeaterAllZoneRampRates()
    (atto_device.CRYO800.sample.Sample
    method), 115
setHeaterAllZoneRampRates()
    (atto_device.CRYO2100.condenser.Condenser
    method), 51
setHeaterAllZoneRampRates()
    (atto_device.CRYO2100.exchange.Exchange
    method), 58
setHeaterAllZoneRampRates()
    (atto_device.CRYO2100.sample.Sample
    method), 74
setHeaterAllZoneRampRates()
    (atto_device.CRYO2100.stage40k.Stage40k
    method), 80
setHeaterAllZoneRampRates()
    (atto_device.CRYO2100.vti.Vti
    method), 98
setHeaterAllZoneRampRates()
    (atto_device.CRYO800.sample.Sample
    method), 115
setHeaterAllZoneRampRates()
    (atto_device.CRYO2100.tboard.Tboard
    method), 93
setHeaterAllZoneRampRates()
    (atto_device.CRYO800.tboard.Tboard method),
    128
setHeaterAllZoneRampRates()
    (atto_device.CRYOEVU.tboard.Tboard
    method), 150
setHeaterPower() (atto_device.CRYO2100.condenser.Condenser
    method), 51
setHeaterPower() (atto_device.CRYO2100.exchange.Exchange
    method), 58
setHeaterPower() (atto_device.CRYO2100.sample.Sample
    method), 74
setHeaterPower() (atto_device.CRYO2100.stage40k.Stage40k
    method), 80
setHeaterPower() (atto_device.CRYO2100.vti.Vti
    method), 98
setHeaterPower() (atto_device.CRYO800.exchange.Exchange
    method), 106
setHeaterPower() (atto_device.CRYO800.sample.Sample
    method), 115
setHeaterRes() (atto_device.CRYO2100.condenser.Condenser
    method), 51
setHeaterRes() (atto_device.CRYO2100.exchange.Exchange
    method), 58
setHeaterRes() (atto_device.CRYO2100.sample.Sample
    method), 74
setHeaterRes() (atto_device.CRYO2100.stage40k.Stage40k
    method), 80
setHeaterRes() (atto_device.CRYO2100.vti.Vti
    method), 98
setHeaterRes() (atto_device.CRYO800.sample.Sample
    method), 115
setHeaterSetpointChannel()
    (atto_device.CRYO2100.tboard.Tboard
    method), 93
setHeaterSetpointChannel()
    (atto_device.CRYO800.tboard.Tboard method),
    128
setHeaterSetpointChannel()
    (atto_device.CRYOEVU.tboard.Tboard
    method), 150
setHeaterWireRes() (atto_device.CRYO2100.condenser.Condenser
    method), 51
setHeaterWireRes() (atto_device.CRYO2100.exchange.Exchange
    method), 58
setHeaterWireRes() (atto_device.CRYO2100.sample.Sample
    method), 74
setHeaterWireRes() (atto_device.CRYO2100.stage40k.Stage40k
    method), 80

```


`method)`, 80
`setHeaterWireRes()` (`atto_device.CRYO2100.vti.Vti` `method)`, 98
`setHeaterWireRes()` (`atto_device.CRYO800.sample.Sample` `method)`, 115
`setHeaterZoneSettings()` (`atto_device.CRYO2100.condenser.Condenser` `method)`, 51
`setHeaterZoneSettings()` (`atto_device.CRYO2100.exchange.Exchange` `method)`, 58
`setHeaterZoneSettings()` (`atto_device.CRYO2100.sample.Sample` `method)`, 74
`setHeaterZoneSettings()` (`atto_device.CRYO2100.stage40k.Stage40k` `method)`, 80
`setHeaterZoneSettings()` (`atto_device.CRYO2100.vti.Vti` `method)`, 98
`setHeaterZoneSettings()` (`atto_device.CRYO800.exchange.Exchange` `method)`, 107
`setHeaterZoneSettings()` (`atto_device.CRYO800.sample.Sample` `method)`, 115
`setHeaterZoneSettingsChannel()` (`atto_device.CRYO2100.tboard.Tboard` `method)`, 93
`setHeaterZoneSettingsChannel()` (`atto_device.CRYO800.tboard.Tboard` `method)`, 128
`setHeaterZoneSettingsChannel()` (`atto_device.CRYOEVU.tboard.Tboard` `method)`, 151
`setHighPassCutOffFreq()` (`atto_device.IDS.realtime.Realtime` `method)`, 173
`setHSetPoint()` (`atto_device.CRYO2100.magnet.Magnet` `method)`, 65
`setHSetPoint3D()` (`atto_device.CRYO2100.magnet.Magnet` `method)`, 65
`setHumidityInPercent()` (`atto_device.IDS.manual.Manual` `method)`, 162
`setI()` (`atto_device.CRYO2100.condenser.Condenser` `method)`, 52
`setI()` (`atto_device.CRYO2100.exchange.Exchange` `method)`, 58
`setI()` (`atto_device.CRYO2100.sample.Sample` `method)`, 74
`setI()` (`atto_device.CRYO2100.stage40k.Stage40k` `method)`, 80
`setI()` (`atto_device.CRYO2100.vti.Vti` `method)`, 99
`setI()` (`atto_device.CRYO800.sample.Sample` `method)`, 116
`setInitMode()` (`atto_device.IDS.system.System` `method)`, 176
`setInputFilterSettings()` (`atto_device.CRYO2100.condenser.Condenser` `method)`, 52
`setInputFilterSettings()` (`atto_device.CRYO2100.exchange.Exchange` `method)`, 59
`setInputFilterSettings()` (`atto_device.CRYO2100.magnet.Magnet` `method)`, 65
`setInputFilterSettings()` (`atto_device.CRYO2100.sample.Sample` `method)`, 75
`setInputFilterSettings()` (`atto_device.CRYO2100.stage40k.Stage40k` `method)`, 81
`setInputFilterSettings()` (`atto_device.CRYO2100.vti.Vti` `method)`, 99
`setInputFilterSettings()` (`atto_device.CRYO800.exchange.Exchange` `method)`, 107
`setInputFilterSettings()` (`atto_device.CRYO800.sample.Sample` `method)`, 116
`setIpAddress()` (`atto_device.AMC.network.Network` `method)`, 27
`setIpAddress()` (`atto_device.CRYO2100.network.Network` `method)`, 69
`setIpAddress()` (`atto_device.CRYO800.network.Network` `method)`, 111
`setIpAddress()` (`atto_device.CRYOEVU.network.Network` `method)`, 137
`setIpAddress()` (`atto_device.IDS.network.Network` `method)`, 166
`setLinearization()` (`atto_device.AMC.res.Res` `method)`, 29
`setLinearRange()` (`atto_device.IDS.realtime.Realtime` `method)`, 173
`setLowerSoftLimit()` (`atto_device.AMC.amcids.Amcids` `method)`, 10
`setLUT()` (`atto_device.AMC.rotcomp.Rotcomp` `method)`, 30
`setLut()` (`atto_device.IDS.nlc.Nlc` `method)`, 169
`setLutApplied()` (`atto_device.IDS.nlc.Nlc` `method)`, 169
`setMasterAxis()` (`atto_device.IDS.axis.Axis` `method)`, 157
`setMaxPower()` (`atto_device.CRYO2100.condenser.Condenser` `method)`, 52

setMaxPower() (*atto_device.CRYO2100.exchange.Exchange method*), 163
 setMaxPower() (*atto_device.CRYO2100.sample.Sample method*), 75
 setMaxPower() (*atto_device.CRYO2100.stage40k.Stage40k method*), 81
 setMaxPower() (*atto_device.CRYO2100.vti.Vti method*), 99
 setMaxPower() (*atto_device.CRYO800.sample.Sample method*), 116
 setMode() (*atto_device.AMC.res.Res method*), 29
 setMode() (*atto_device.AMC.rtout.Rtout method*), 35
 setMotionControlThreshold() (*atto_device.AMC.control.Control method*), 18
 setNslMux() (*atto_device.AMC.rtin.Rtin method*), 32
 setNSteps() (*atto_device.AMC.move.Move method*), 24
 setP() (*atto_device.CRYO2100.condenser.Condenser method*), 52
 setP() (*atto_device.CRYO2100.exchange.Exchange method*), 59
 setP() (*atto_device.CRYO2100.sample.Sample method*), 75
 setP() (*atto_device.CRYO2100.stage40k.Stage40k method*), 81
 setP() (*atto_device.CRYO2100.vti.Vti method*), 99
 setP() (*atto_device.CRYO800.sample.Sample method*), 116
 setPassMode() (*atto_device.IDS.axis.Axis method*), 157
 setPause3D() (*atto_device.CRYO2100.magnet.Magnet method*), 65
 setPeriodHsslClk() (*atto_device.IDS.realtime.Realtime method*), 173
 setPeriodHsslGap() (*atto_device.IDS.realtime.Realtime method*), 174
 setPeriodSinCosClk() (*atto_device.IDS.realtime.Realtime method*), 174
 setPersistentMode3D() (*atto_device.CRYO2100.magnet.Magnet method*), 65
 setPID() (*atto_device.CRYO2100.condenser.Condenser method*), 52
 setPID() (*atto_device.CRYO2100.exchange.Exchange method*), 59
 setPID() (*atto_device.CRYO2100.sample.Sample method*), 75
 setPID() (*atto_device.CRYO2100.stage40k.Stage40k method*), 81
 setPID() (*atto_device.CRYO2100.vti.Vti method*), 99
 setPID() (*atto_device.CRYO800.sample.Sample method*), 116
 setPositionerSN() (*atto_device.AMC.test.Test method*), 43
 setPressureInHPa() (*atto_device.IDS.manual.Manual method*), 163
 setProxyServer() (*atto_device.AMC.network.Network method*), 27
 setProxyServer() (*atto_device.CRYO2100.network.Network method*), 69
 setProxyServer() (*atto_device.CRYO800.network.Network method*), 111
 setProxyServer() (*atto_device.CRYOEUVU.network.Network method*), 137
 setProxyServer() (*atto_device.IDS.network.Network method*), 166
 setRampRate() (*atto_device.CRYO2100.condenser.Condenser method*), 52
 setRampRate() (*atto_device.CRYO2100.exchange.Exchange method*), 59
 setRampRate() (*atto_device.CRYO2100.magnet.Magnet method*), 66
 setRampRate() (*atto_device.CRYO2100.sample.Sample method*), 75
 setRampRate() (*atto_device.CRYO2100.stage40k.Stage40k method*), 81
 setRampRate() (*atto_device.CRYO2100.vti.Vti method*), 99
 setRampRate() (*atto_device.CRYO800.sample.Sample method*), 116
 setRealTimeInChangePerPulse() (*atto_device.AMC.rtin.Rtin method*), 32
 setRealTimeInFeedbackLoopMode() (*atto_device.AMC.rtin.Rtin method*), 33
 setRealTimeInMode() (*atto_device.AMC.rtin.Rtin method*), 33
 setRealTimeInStepsPerPulse() (*atto_device.AMC.rtin.Rtin method*), 33
 setRefractiveIndex() (*atto_device.IDS.manual.Manual method*), 163
 setRefractiveIndexCompensationMode() (*atto_device.IDS.ecu.Ecu method*), 161
 setReset() (*atto_device.AMC.control.Control method*), 18
 setResolutionBissC() (*atto_device.IDS.realtime.Realtime method*), 174
 setResolutionHsslHigh() (*atto_device.IDS.realtime.Realtime method*), 174
 setResolutionHsslLow() (*atto_device.IDS.realtime.Realtime method*), 174
 setResolutionSinCos() (*atto_device.IDS.realtime.Realtime method*), 174
 setRtDistanceMode() (*atto_device.IDS.realtime.Realtime method*), 174

174
 setRtOutMode() (atto_device.IDS.realtime.Realtime method), 174
 setSampleExchangePumpOnSetting() (atto_device.CRYO800.action.Action method), 103
 setSampleExchangeRampRateSetting() (atto_device.CRYO2100.action.Action method), 47
 setSampleExchangeRampRateSetting() (atto_device.CRYO800.action.Action method), 103
 setSensorDirection() (atto_device.AMC.control.Control method), 18
 setSensorEnabled() (atto_device.AMC.control.Control method), 18
 setSetPoint() (atto_device.CRYO2100.condenser.Condenser method), 52
 setSetPoint() (atto_device.CRYO2100.exchange.Exchange method), 59
 setSetPoint() (atto_device.CRYO2100.sample.Sample method), 75
 setSetPoint() (atto_device.CRYO2100.stage40k.Stage40k method), 81
 setSetPoint() (atto_device.CRYO2100.vti.Vti method), 99
 setSetPoint() (atto_device.CRYO800.exchange.Exchange method), 107
 setSetPoint() (atto_device.CRYO800.sample.Sample method), 116
 setSignalMode() (atto_device.AMC.rtout.Rtout method), 35
 setSingleStep() (atto_device.AMC.move.Move method), 24
 setSoftLimitEnabled() (atto_device.AMC.amcids.Amcids method), 10
 setSubnetMask() (atto_device.AMC.network.Network method), 27
 setSubnetMask() (atto_device.CRYO2100.network.Network method), 69
 setSubnetMask() (atto_device.CRYO800.network.Network method), 111
 setSubnetMask() (atto_device.CRYOEUVU.network.Network method), 137
 setSubnetMask() (atto_device.IDS.network.Network method), 166
 setTemperatureInDegrees() (atto_device.IDS.manual.Manual method), 163
 setTestplatz() (atto_device.AMC.test.Test method), 43
 setTime() (atto_device.AMC.system_service.System_service method), 39
 setTime() (atto_device.CRYO2100.system_service.System_service method), 90
 setTime() (atto_device.CRYO800.system_service.System_service method), 125
 setTime() (atto_device.CRYOEUVU.system_service.System_service method), 148
 setTime() (atto_device.IDS.system_service.System_service method), 178
 setTriggerConfig() (atto_device.AMC.rtout.Rtout method), 35
 setUpperSoftLimit() (atto_device.AMC.amcids.Amcids method), 10
 setUserConfigOverrideFile() (atto_device.CRYO2100.system.System method), 88
 setUserConfigOverrideFile() (atto_device.CRYO800.system.System method), 123
 setUserConfigOverrideFile() (atto_device.CRYOEUVU.system.System method), 146
 setVelocityThresholds() (atto_device.IDS.nlc.Nlc method), 169
 setWifiMode() (atto_device.AMC.network.Network method), 27
 setWifiMode() (atto_device.CRYO2100.network.Network method), 69
 setWifiMode() (atto_device.CRYO800.network.Network method), 111
 setWifiMode() (atto_device.CRYOEUVU.network.Network method), 137
 setWifiMode() (atto_device.IDS.network.Network method), 166
 setWifiPassphrase() (atto_device.AMC.network.Network method), 27
 setWifiPassphrase() (atto_device.CRYO2100.network.Network method), 70
 setWifiPassphrase() (atto_device.CRYO800.network.Network method), 111
 setWifiPassphrase() (atto_device.CRYOEUVU.network.Network method), 138
 setWifiPassphrase() (atto_device.IDS.network.Network method), 166
 setWifiSSID() (atto_device.AMC.network.Network method), 27
 setWifiSSID() (atto_device.CRYO2100.network.Network method), 70
 setWifiSSID() (atto_device.CRYO800.network.Network method), 111

`method`), 111
`setWifiSSID()` (`atto_device.CRYOEVU.network.Network`
`method`), 138
`setWifiSSID()` (`atto_device.IDS.network.Network`
`method`), 166
`shutdown()` (`atto_device.CRYO2100.action.Action`
`method`), 47
`shutdown()` (`atto_device.CRYO800.action.Action`
`method`), 103
`softReset()` (`atto_device.AMC.system_service.System_service`
`method`), 39
`softReset()` (`atto_device.CRYO2100.system_service.System_service`
`method`), 90
`softReset()` (`atto_device.CRYO800.system_service.System_service`
`method`), 125
`softReset()` (`atto_device.CRYOEVU.system_service.System_service`
`method`), 148
`softReset()` (`atto_device.IDS.system_service.System_service`
`method`), 178
`softwareUpdateBase64()`
(`atto_device.AMC.update.Update` `method`), 43
`softwareUpdateBase64()`
(`atto_device.CRYO2100.update.Update`
`method`), 94
`softwareUpdateBase64()`
(`atto_device.CRYO800.update.Update`
`method`), 130
`softwareUpdateBase64()`
(`atto_device.CRYOEVU.update.Update`
`method`), 153
`softwareUpdateBase64()`
(`atto_device.IDS.update.Update` `method`), 179
`Stage40k` (class in `atto_device.CRYO2100.stage40k`), 77
`start()` (`atto_device.CRYOEVU.compressor.Compressor`
`method`), 132
`start()` (`atto_device.CRYOEVU.membranePump.Membranepump`
`method`), 134
`start()` (`atto_device.CRYOEVU.turboPump.Turbopump`
`method`), 152
`startDiagnostic()` (`atto_device.AMC.diagnostic.Diagnostic`
`method`), 20
`startFieldControl()`
(`atto_device.CRYO2100.magnet.Magnet`
`method`), 66
`startFullSpeed()` (`atto_device.CRYO2100.scrollPump.Scrollpump`
`method`), 76
`startFullSpeed()` (`atto_device.CRYOEVU.scrollPump.Scrollpump`
`method`), 139
`startHeaterOpenLoopPower()`
(`atto_device.CRYO2100.condenser.Condenser`
`method`), 52
`startHeaterOpenLoopPower()`
(`atto_device.CRYO2100.exchange.Exchange`
`method`), 59
`startHeaterOpenLoopPower()`
(`atto_device.CRYO2100.sample.Sample`
`method`), 75
`startHeaterOpenLoopPower()`
(`atto_device.CRYO2100.stage40k.Stage40k`
`method`), 81
`startHeaterOpenLoopPower()`
(`atto_device.CRYO2100.vti.Vti` `method`), 99
`startHeaterOpenLoopPower()`
(`atto_device.CRYO800.exchange.Exchange`
`method`), 107
`startHeaterOpenLoopPower()`
(`atto_device.CRYO800.sample.Sample`
`method`), 116
`startHeaterOpenLoopPower()`
(`atto_device.CRYOEVU.heater.Heater`
`method`), 133
`startHeaterZoneMode()`
(`atto_device.CRYO2100.condenser.Condenser`
`method`), 52
`startHeaterZoneMode()`
(`atto_device.CRYO2100.exchange.Exchange`
`method`), 59
`startHeaterZoneMode()`
(`atto_device.CRYO2100.sample.Sample`
`method`), 75
`startHeaterZoneMode()`
(`atto_device.CRYO2100.stage40k.Stage40k`
`method`), 81
`startHeaterZoneMode()`
(`atto_device.CRYO2100.vti.Vti` `method`), 99
`startHeaterZoneMode()`
(`atto_device.CRYO800.exchange.Exchange`
`method`), 107
`startHeaterZoneMode()`
(`atto_device.CRYO800.sample.Sample`
`method`), 116
`startHeatingChannel()`
(`atto_device.CRYO2100.tboard.Tboard`
`method`), 93
`startHeatingChannel()`
(`atto_device.CRYO800.tboard.Tboard` `method`), 128
`startHeatingChannel()`
(`atto_device.CRYOEVU.tboard.Tboard`
`method`), 151
`startHeatingOpenLoopChannel()`
(`atto_device.CRYO2100.tboard.Tboard`
`method`), 93
`startHeatingOpenLoopChannel()`

(atto_device.CRYO800.tboard.Tboard method), 129
 startHeatingOpenLoopChannel()
 (atto_device.CRYOEVU.tboard.Tboard method), 151
 startHeatingZoneModeChannel()
 (atto_device.CRYO2100.tboard.Tboard method), 94
 startHeatingZoneModeChannel()
 (atto_device.CRYO800.tboard.Tboard method), 129
 startHeatingZoneModeChannel()
 (atto_device.CRYOEVU.tboard.Tboard method), 151
 startMeasurement() (atto_device.IDS.system.System method), 176
 startOpticsAlignment()
 (atto_device.IDS.system.System method), 176
 startRampControl() (atto_device.CRYO2100.condenser.Condenser method), 52
 startRampControl() (atto_device.CRYO2100.exchange.Exchange method), 59
 startRampControl() (atto_device.CRYO2100.sample.Sample method), 75
 startRampControl() (atto_device.CRYO2100.stage40k.Stage40k method), 81
 startRampControl() (atto_device.CRYO2100.vti.Vti method), 99
 startRampControl() (atto_device.CRYO800.sample.Sample method), 116
 startStandbySpeed()
 (atto_device.CRYO2100.scrollPump.Scrollpump method), 76
 startStandbySpeed()
 (atto_device.CRYOEVU.scrollPump.Scrollpump method), 139
 startTempControl() (atto_device.CRYO2100.condenser.Condenser method), 53
 startTempControl() (atto_device.CRYO2100.exchange.Exchange method), 59
 startTempControl() (atto_device.CRYO2100.sample.Sample method), 75
 startTempControl() (atto_device.CRYO2100.stage40k.Stage40k method), 81
 startTempControl() (atto_device.CRYO2100.vti.Vti method), 100
 startTempControl() (atto_device.CRYO800.exchange.Exchange method), 107
 startTempControl() (atto_device.CRYO800.sample.Sample method), 117
 Status (class in atto_device.AMC.status), 36
 stop() (atto_device.CRYO2100.scrollPump.Scrollpump method), 76
 stop() (atto_device.CRYOEVU.compressor.Compressor method), 133
 stop() (atto_device.CRYOEVU.membranePump.Membranepump method), 134
 stop() (atto_device.CRYOEVU.scrollPump.Scrollpump method), 139
 stop() (atto_device.CRYOEVU.turboPump.Turbopump method), 152
 stopCurrentTest() (atto_device.AMC.test.Test method), 43
 stopFieldControl() (atto_device.CRYO2100.magnet.Magnet method), 66
 stopHeater() (atto_device.CRYOEVU.heater.Heater method), 133
 stopHeatingChannel()
 (atto_device.CRYO2100.tboard.Tboard method), 94
 stopHeatingChannel()
 (atto_device.CRYO800.tboard.Tboard method), 129
 stopHeatingChannel()
 (atto_device.CRYOEVU.tboard.Tboard method), 151
 stopMagnet3D() (atto_device.CRYO2100.magnet.Magnet method), 66
 stopMeasurement() (atto_device.IDS.system.System method), 176
 stopOpticsAlignment()
 (atto_device.IDS.system.System method), 176
 stopRampControl() (atto_device.CRYO2100.condenser.Condenser method), 53
 stopRampControl() (atto_device.CRYO2100.exchange.Exchange method), 60
 stopRampControl() (atto_device.CRYO2100.sample.Sample method), 76
 stopRampControl() (atto_device.CRYO2100.stage40k.Stage40k method), 82
 stopRampControl() (atto_device.CRYO2100.vti.Vti method), 100
 stopRampControl() (atto_device.CRYO800.sample.Sample method), 117
 stopTempControl() (atto_device.CRYO2100.condenser.Condenser method), 53
 stopTempControl() (atto_device.CRYO2100.exchange.Exchange method), 60
 stopTempControl() (atto_device.CRYO2100.sample.Sample method), 76
 stopTempControl() (atto_device.CRYO2100.stage40k.Stage40k method), 82
 stopTempControl() (atto_device.CRYO2100.vti.Vti method), 100
 stopTempControl() (atto_device.CRYO800.exchange.Exchange method), 107

stopTempControl() (*atto_device.CRYO800.sample.Sample*
 method), 117
 switchMagnetsOff() (*atto_device.CRYO2100.action.Action*
 method), 47
 switchMagnetsOff() (*atto_device.CRYO800.action.Action*
 method), 103
 System (*class in atto_device.CRYO2100.system*), 82
 System (*class in atto_device.CRYO800.system*), 117
 System (*class in atto_device.CRYOEVU.system*), 140
 System (*class in atto_device.IDS.system*), 175
 System_service (*class in atto_device.AMC.system_service*), 38
 System_service (*class in atto_device.CRYO2100.system_service*),
 88
 System_service (*class in atto_device.CRYO800.system_service*), 124
 System_service (*class in atto_device.CRYOEVU.system_service*), 146
 System_service (*class in atto_device.IDS.system_service*), 176

T
 Tboard (*class in atto_device.CRYO2100.tboard*), 91
 Tboard (*class in atto_device.CRYO800.tboard*), 126
 Tboard (*class in atto_device.CRYOEVU.tboard*), 148
 TCP_PORT (*atto_device.AMC.ACS.Device attribute*), 7
 TCP_PORT (*atto_device.CRYO2100.ACS.Device attribute*), 44
 TCP_PORT (*atto_device.CRYO800.ACS.Device attribute*),
 100
 TCP_PORT (*atto_device.CRYOEVU.ACS.Device attribute*), 130
 TCP_PORT (*atto_device.IDS.ACS.Device attribute*), 154
 Test (*class in atto_device.AMC.test*), 40
 Turbopump (*class in atto_device.CRYO800.turboPump*),
 129
 Turbopump (*class in atto_device.CRYOEVU.turboPump*),
 152

U
 unlock() (*atto_device.AMC.access.Access method*), 9
 unlock() (*atto_device.CRYO2100.access.Access*
 method), 46
 unlock() (*atto_device.CRYO800.access.Access*
 method), 102
 unlock() (*atto_device.CRYOEVU.access.Access*
 method), 132
 unlock() (*atto_device.IDS.access.Access method*), 156
 Update (*class in atto_device.AMC.update*), 43
 Update (*class in atto_device.CRYO2100.update*), 94
 Update (*class in atto_device.CRYO800.update*), 129
 Update (*class in atto_device.CRYOEVU.update*), 152
 Update (*class in atto_device.IDS.update*), 178

 updateOffsets() (*atto_device.AMC.rotcomp.Rotcomp*
 method), 30
 updateTimeFromInternet() (*atto_device.AMC.system_service.System_service*
 method), 40
 updateTimeFromInternet() (*atto_device.CRYO2100.system_service.System_service*
 method), 90
 updateTimeFromInternet() (*atto_device.CRYO800.system_service.System_service*
 method), 126
 updateTimeFromInternet() (*atto_device.CRYOEVU.system_service.System_service*
 method), 148
 updateTimeFromInternet() (*atto_device.IDS.system_service.System_service*
 method), 178
 uploadCalibrationCurve() (*atto_device.CRYO2100.condenser.Condenser*
 method), 53
 uploadCalibrationCurve() (*atto_device.CRYO2100.exchange.Exchange*
 method), 60
 uploadCalibrationCurve() (*atto_device.CRYO2100.magnet.Magnet*
 method), 66
 uploadCalibrationCurve() (*atto_device.CRYO2100.sample.Sample*
 method), 76
 uploadCalibrationCurve() (*atto_device.CRYO2100.stage40k.Stage40k*
 method), 82
 uploadCalibrationCurve() (*atto_device.CRYO2100.tboard.Tboard*
 method), 94
 uploadCalibrationCurve() (*atto_device.CRYO2100.vti.Vti method*),
 100
 uploadCalibrationCurve() (*atto_device.CRYO800.exchange.Exchange*
 method), 107
 uploadCalibrationCurve() (*atto_device.CRYO800.sample.Sample*
 method), 117
 uploadCalibrationCurve() (*atto_device.CRYO800.tboard.Tboard method*),
 129
 uploadCalibrationCurve() (*atto_device.CRYOEVU.tboard.Tboard*
 method), 151
 uploadCalibrationCurve340() (*atto_device.CRYO2100.exchange.Exchange*
 method), 60
 uploadLicenseBase64()

(*atto_device.AMC.update.Update method*),
 44
 uploadLicenseBase64()
 (*atto_device.CRYO2100.update.Update method*), 94
 uploadLicenseBase64()
 (*atto_device.CRYO800.update.Update method*), 130
 uploadLicenseBase64()
 (*atto_device.CRYOEVU.update.Update method*), 153
 uploadLicenseBase64()
 (*atto_device.IDS.update.Update method*),
 179
 uploadSoftwareImageBase64()
 (*atto_device.AMC.update.Update method*),
 44
 uploadSoftwareImageBase64()
 (*atto_device.CRYO2100.update.Update method*), 94
 uploadSoftwareImageBase64()
 (*atto_device.CRYO800.update.Update method*), 130
 uploadSoftwareImageBase64()
 (*atto_device.CRYOEVU.update.Update method*), 153
 uploadSoftwareImageBase64()
 (*atto_device.IDS.update.Update method*),
 179

V

Valves (*class in atto_device.CRYOEVU.valves*), 153
 verify() (*atto_device.AMC.network.Network method*),
 27
 verify() (*atto_device.CRYO2100.network.Network method*), 70
 verify() (*atto_device.CRYO800.network.Network method*), 111
 verify() (*atto_device.CRYOEVU.network.Network method*), 138
 verify() (*atto_device.IDS.network.Network method*),
 166
 Vti (*class in atto_device.CRYO2100.vti*), 95

W

writeNSteps() (*atto_device.AMC.move.Move method*),
 24